MLOps Portfolio

From Scratch to Advanced: A Complete Study & Interview Guide

A complete, from-scratch to advanced guide to the project and every tool it uses - written for deep understanding and technical interview preparation.

Project: MLOps Portfolio - operating one spam classifier five ways (serve, stream, provision, deploy, explain).
Repository: github.com/Swapnil-byte-798/mlops-portfolio

Table of Contents

1. Introduction: What This MLOps Portfolio Is 7

1.1 What is MLOps, in one paragraph? . . . 7
1.2 The core idea: one model, operated five ways . . . 7
1.3 A plain-English tour of the five modules . . . 8
1.4 Why "container-first" and why reproducibility matters . . . 9
1.5 The three pitches (know all three cold) . . . 10
1.6 Roadmap of the rest of this guide . . . 10

2. MLOps Fundamentals From Zero 12

2.1 What is a "model," really? . . . 12
2.2 The gap: why MLOps exists . . . 12
2.3 The ML lifecycle: data -> train -> package -> serve -> monitor -> retrain . . . 13
2.4 Dev vs Ops - and the extra weight ML adds . . . 14
2.5 The model vs. the system that runs the model . . . 14
2.6 The five pillars this project touches - and why each matters . . . 15
2.7 Reproducibility, containers, and why "it works on my machine" is banned . . . 16
2.8 The 12-Factor App and configuration via environment variables . . . 16
2.9 The mental model to carry into the tool chapters . . . 17

3. The Model: Spam Classification, NLP, and scikit-learn 18

3.1 What is text classification, and what is spam detection? . . . 18
3.2 The core problem: computers do math, not words . . . 18
3.3 Bag-of-words: the simplest text-to-numbers scheme . . . 19
3.4 Two text-cleaning steps: lowercasing and stop words . . . 19
3.5 TF-IDF, explained intuitively . . . 20
3.6 Logistic Regression: turning the vector into a decision . . . 21
3.7 scikit-learn Pipelines: chaining vectorizer + classifier . . . 21
3.8 fit, predict, and predict_proba . . . 22
3.9 Model persistence with joblib and .pkl . . . 23
3.10 Evaluating a spam classifier: accuracy is not enough . . . 23
3.11 Putting it together . . . 24

4. Containers & Docker From Scratch 26

4.1 The problem containers solve: "it works on my machine" . . . 26
4.2 Containers vs. virtual machines . . . 26

4.3 Images vs. containers . . . 27
4.4 Layers and the union filesystem . . . 27
4.5 The build cache - and why we copy requirements before source . . . 28
4.6 Reading serving/Dockerfile directive by directive . . . 29
4.7 CMD vs. ENTRYPOINT, and ENV . . . 31
4.8 Training the model at build time - a self-contained image . . . 31
4.9 The kafka-streaming Dockerfile: one image, two roles . . . 32
4.10 Working with Docker: the core commands . . . 32
4.11 Why bind to 0.0.0.0 , not 127.0.0.1 , inside a container . . . 33
4.12 Registries and tags, conceptually . . . 33
4.13 Chapter summary . . . 34

5. Docker Compose & Container Networking 35

5.1 What Docker Compose is . . . 35
5.2 Anatomy of a Compose file . . . 35
5.3 Container networking - the top interview topic . . . 38
5.4 depends_on vs. true readiness . . . 40
5.5 Volumes and environment, briefly . . . 40
5.6 Chapter summary . . . 41

6. Serving the Model: FastAPI, Uvicorn, Pydantic, REST 42

6.1 What an API is, and what "serving a model" means . . . 42
6.2 REST basics: resources, methods, JSON, status codes . . . 42
6.3 FastAPI: what it is and why it's used here . . . 43
6.4 Automatic OpenAPI docs . . . 44
6.5 Pydantic: request models and validation . . . 45
6.6 ASGI and Uvicorn: what actually runs the server . . . 46
6.7 The two endpoints in depth . . . 46
6.8 Lazy model loading with a cached global . . . 47
6.9 How the rest of the repo consumes this API . . . 48

7. Real-Time Streaming: Apache Kafka 50

7.1 Batch vs. stream processing (from zero) . . . 50
7.2 What Kafka actually is . . . 50
7.3 Kafka core concepts . . . 51
7.4 Serialization: getting Python objects onto the wire . . . 51
7.5 Consumer groups: scalable, ordered-per-partition consumption . . . 52

7.6 Offsets and delivery semantics . . . 53
7.7 KRaft mode vs. the old ZooKeeper . . . 54
7.8 Listeners vs. advertised listeners - the classic container gotcha . . . 54
7.9 Replication factor 1 - single-node reality . . . 55
7.10 Why inference is a separate service from the consumer . . . 56
7.11 Resilience: retry and backoff for startup ordering . . . 56
7.12 The whole pipeline, end to end . . . 57

8. Orchestration: Kubernetes 59

8.1 What "orchestration" means and why it exists . . . 59
8.2 The object hierarchy: Pod -> ReplicaSet -> Deployment . . . 59
8.3 Labels and selectors - how objects find each other . . . 61
8.4 Services - a stable address for moving Pods . . . 62
8.5 Health probes - readiness vs liveness . . . 63
8.6 Resource requests vs limits . . . 64
8.7 imagePullPolicy: IfNotPresent . . . 65
8.8 Declarative apply and the reconciliation loop . . . 66
8.9 kubectl basics . . . 66
8.10 Chapter summary . . . 67

9. Infrastructure as Code: Terraform 68

9.1 What "Infrastructure as Code" means, and why anyone bothers . . . 68
9.2 Terraform, declarative vs. imperative . . . 69
9.3 HCL: the language you write Terraform in . . . 69
9.4 State: the file that makes Terraform work . . . 72
9.5 The core loop: init -> plan -> apply -> destroy . . . 73
9.6 Retargeting the provider: how one config hits LocalStack instead of AWS . . . 74
9.7 DynamoDB specifics: billing mode, hash key, attributes . . . 74
9.8 Putting it together . . . 75

10. Cloud & AWS Emulation: LocalStack, S3, DynamoDB, IAM 77

10.1 What "the cloud" and AWS actually are . . . 77
10.2 What LocalStack is and why it is valuable . . . 77
10.3 The endpoint override - how you point tools at LocalStack . . . 78
10.4 The skip_* flags - telling Terraform not to phone home . . . 79
10.5 AWS S3 - object storage for the model artifacts . . . 79
10.6 AWS DynamoDB - a managed NoSQL database for predictions . . . 80

10.7 AWS credentials & IAM - who are you, and what may you do . . . 82
10.8 Regions and endpoints . . . 83
10.9 The full picture . . . 84

11. The GenAI Layer: AWS Bedrock, boto3, and LLMs 85

11.1 What a Large Language Model is (from scratch) . . . 85
11.2 Managed hosting vs self-hosting - and what AWS Bedrock is . . . 85
11.3 boto3 and the bedrock-runtime client . . . 86
11.4 invoke_model and the Anthropic messages shape . . . 87
11.5 Prompt design: how the explanation is steered . . . 88
11.6 Credentials from the environment - never hard-coded . . . 89
11.7 Regions and model IDs - the two things you must get right . . . 89
11.8 Composing classical ML with GenAI - Explainable AI . . . 90

12. Putting It All Together: Architecture, Flow, and Running It 92

12.1 The one-model, five-ways idea . . . 92
12.2 The end-to-end architecture . . . 92
12.3 The "same image everywhere" principle . . . 93
12.4 The recurring design principles . . . 94
12.5 Running each module (command recap) . . . 95
12.6 What is real vs. what is demo . . . 96
12.7 How you would harden it for production (the system-design bridge) . . . 97
12.8 Tying the modules together . . . 97

13. Interview Prep I - Tool Question Bank (Zero to Advanced) 99

13.1 Docker . . . 99
13.2 Docker Compose & Container Networking . . . 100
13.3 FastAPI / Uvicorn / Pydantic & REST . . . 101
13.4 scikit-learn / ML Serving . . . 103
13.5 Apache Kafka . . . 103
13.6 Kubernetes . . . 105
13.7 Terraform / Infrastructure as Code . . . 106
13.8 AWS (S3, DynamoDB, IAM, Bedrock) & LocalStack . . . 108
13.9 General MLOps . . . 109

14. Interview Prep II - Project Deep-Dive, System Design, War Stories, Glossary 111

14.1 The 30-second, 2-minute, and 5-minute pitch . . . 111

14.2 Project-specific Q&A; 112
14.3 System-design questions built on the repo 113
14.4 Behavioral "war stories" (real gotchas from this repo) 115
14.5 One-page cheat sheet 116
14.6 Comprehensive glossary 117

1. Introduction: What This MLOps Portfolio Is

This guide teaches you one project, deeply, from the ground up - and along the way it teaches you the entire toolchain of modern **MLOps**. By the end you should be able to walk into an interview, describe this project in 30 seconds or 20 minutes, and answer any follow-up question about the tools inside it. This first chapter is the map. It tells you what the project is, what MLOps means, the single idea that ties everything together, and how to talk about it. Every later chapter drills into one tool.

1.1 What is MLOps, in one paragraph?

MLOps (short for "Machine Learning Operations") is the discipline of taking a machine-learning model out of a data scientist's experimental notebook and turning it into a reliable, reproducible, monitored service that real users or systems can depend on. A **model** here just means a saved mathematical function that was *trained* on data - you feed it an input (a text message) and it returns an output (a label like "spam" or "ham"). The hard part of ML in industry is almost never the math; it is everything *around* the model: packaging it so it runs the same way everywhere, exposing it so other software can call it, scaling it when traffic grows, provisioning the cloud resources it needs, deploying new versions without downtime, and being able to explain and audit what it did. MLOps is essentially **DevOps applied to machine learning** - where **DevOps** (Development + Operations) is the older practice of automating how software gets built, tested, shipped, and run. If a data scientist's job ends at "the model works on my laptop," the MLOps engineer's job is everything from there to "the model works, at scale, for everyone, forever, and we can prove it."

> **Interview Q:** What is MLOps and how is it different from data science?
> **A:** Data science produces a trained model - it answers "does this model predict well?" MLOps takes that model and makes it a dependable production system: it answers "does this model run reliably, reproducibly, and at scale for real traffic, and can we deploy, monitor, and roll it back safely?" MLOps is DevOps practices - containers, infrastructure-as-code, orchestration, CI/CD - specialized for the ML lifecycle, which adds concerns like model artifacts, retraining, and prediction auditing.

1.2 The core idea: one model, operated five ways

Most beginner portfolios show five *different* models - a house-price regressor, an image classifier, a chatbot - each a shallow demo. This project does the opposite, and that is deliberate. It takes **one** deliberately simple model and operates it **five different ways**. The model itself is a **spam classifier**: given a short text message, it predicts whether the message is *spam* (unwanted junk) or *ham* (a legitimate message - "ham" is the standard, slightly jokey term for "not spam"). Under the hood it is a **TF-IDF + Logistic Regression** pipeline built with **scikit-learn** (the most widely used classical machine-learning library in Python):

- **TF-IDF** ("Term Frequency-Inverse Document Frequency") turns raw text into numbers by scoring how important each word is - words that appear often in one message but rarely across all messages score high. It is how the model "reads" text.
- **Logistic Regression** is a simple, fast, well-understood classification algorithm that takes those numbers and outputs a probability of "spam."

The model is intentionally boring. That is the point: because the model is trivial, nothing distracts from the *operations* around it - which is exactly what an MLOps role is about. The portfolio's narrative is that **the interesting engineering is not the model; it is the five ways you can put a model to work**:

#	Verb	Module	What it demonstrates
1	**Serve**	`serving/`	Wrap the model in a REST API and containerize it
2	**Stream**	`kafka-streaming/`	Run real-time inference over a message pipeline
3	**Provision**	`terraform-localstack/`	Create cloud infrastructure as code
4	**Deploy**	`kubernetes/`	Orchestrate the container with replicas and health checks
5	**Explain**	`bedrock-explainer/`	Use a large language model to explain predictions

Together these five verbs trace the **ML lifecycle** end to end: build -> serve -> integrate -> provision -> deploy -> observe/explain. This is a powerful portfolio story because it shows breadth (you know the whole stack) *and* coherence (it all revolves around one artifact you understand completely).

> **Key insight:** "One model, five ways" is a stronger interview narrative than five separate demos because it proves you understand that in production the model is a small, stable core and the *operational surface around it* is where the real engineering lives. You can pivot to any tool - Kafka, Terraform, Kubernetes, Bedrock - and it is all the same spam classifier, so the conversation stays focused on operations, not on ML trivia.

1.3 A plain-English tour of the five modules

Here is what each module actually does and how they connect. Each has its own chapter later; this is the orientation.

1. `serving/` **- the model as a REST API (Ch. 3).** This is the foundation everything else builds on. A **REST API** ("REpresentational State Transfer Application Programming Interface") is just a way for one program to call another over HTTP, the same protocol your browser uses - you send a request to a URL and get a structured response back. Here, the model is wrapped in **FastAPI** (a modern Python web framework) and served by **Uvicorn** (the web server that runs it). It exposes two endpoints: `GET /health` (a heartbeat that says "I'm alive") and `POST /predict` (send a message, get back spam/ham). The trained model is saved to disk as `model.pkl` using **joblib** and loaded when needed. The whole thing is packaged in a **Docker** container so it runs identically anywhere. This module is the actual spam-classifier service; the other four modules stream to it, deploy it, provision for it, or explain it.

2. `kafka-streaming/` **- real-time inference pipeline (Ch. 6). Apache Kafka** is a high-throughput **message broker** - think of it as a super-durable, replayable conveyor belt that many programs can put messages onto and read messages off of, at massive scale. In this module a **producer** program streams incoming messages onto a Kafka topic called `messages`; a **consumer** program reads each one, calls the serving API's `/predict` endpoint, and publishes the result onto a second topic called `predictions`. This shows how a model plugs into a real-time data flow rather than waiting for someone to call it manually. It reuses the exact `serving/` container, proving the value of the serve-first design.

3. `terraform-localstack/` **- infrastructure as code (Ch. 7). Infrastructure as Code (IaC)** means describing your cloud resources - servers, storage, databases - in text files instead of clicking around a web console, so the setup is version-controlled, reviewable, and repeatable. **Terraform** is the leading IaC tool. Here it provisions an **S3 bucket** (cloud file storage, e.g. for model artifacts) and a **DynamoDB table** (a NoSQL database, e.g. for storing predictions). Crucially it runs against **LocalStack** - a program that emulates real AWS cloud services on your own laptop - so you can practice cloud engineering for free, with no AWS account and no risk of a surprise bill.

4. `kubernetes/` **- orchestration and deployment (Ch. 8). Kubernetes** (often abbreviated **K8s**) is the industry-standard system for running containers in production: it keeps them alive, restarts them when they crash, runs multiple copies for reliability, and routes traffic to healthy ones. This module deploys the `serving/` container as a **Deployment** with **2 replicas** (two identical copies for redundancy), health probes so K8s knows when each copy is ready and alive, and a **Service** that exposes them on a network port. This is "deploy" done the way real companies deploy.

5. `bedrock-explainer/` **- a GenAI explanation layer (Ch. 9).** The spam classifier gives a verdict but no reasoning. This module adds a **large language model (LLM)** - Anthropic's **Claude 3 Haiku**, accessed through **AWS Bedrock** (Amazon's managed service for calling foundation models) via the **boto3** Python SDK - to explain in plain English *why* a message looks like spam or ham. This demonstrates the increasingly common pattern of pairing a fast, cheap classical model with a GenAI layer for human-readable explanations. It is the one module that needs real AWS access to run.

Visually, the pieces connect like this: messages flow through **Kafka** into the **spam-classifier API**, which is **deployed by Kubernetes**, running on **infrastructure provisioned by Terraform**, with **Bedrock** available to explain any prediction.

> **Interview Q:** Walk me through how the modules fit together.
> **A:** The `serving/` FastAPI container is the core - the model as an HTTP service. Kafka streaming wraps it in a real-time pipeline: producer to a `messages` topic, consumer calls `/predict`, results to a `predictions` topic. Kubernetes deploys that same container with two replicas and health probes. Terraform provisions the cloud storage and database it would use, tested locally against LocalStack. And Bedrock adds an LLM layer to explain predictions. One model, five operational surfaces, all reusing the same container.

1.4 Why "container-first" and why reproducibility matters

The single most important architectural decision in this project is that **everything runs in Docker**. A **container** is a lightweight, self-contained package that bundles an application together with *everything* it needs to run - the exact Python version, the exact libraries, the model file, the startup command. **Docker** is the tool that builds and runs these packages. The classic problem it solves is *"it works on my machine"*: software that runs on the author's laptop but breaks on a colleague's or on a server because some version differs. A container freezes the entire environment, so it runs the same way on your laptop, your teammate's, a CI server, and a cloud host.

Reproducibility is the property that follows: anyone with Docker can rebuild and run this project identically, with no manual "install Python 3.11, then Java, then these 14 libraries" ritual. As the README puts it, the project is "reproducible on any machine with Docker installed - no local Python/Java version juggling required." This matters because reproducibility is the bedrock of everything else in MLOps: you cannot reliably deploy, scale, roll back, or debug a system whose behavior depends on invisible details of the machine it happens to run on. Container-first is also *why*

the "one model, five ways" story holds together - the same serving container is what Kafka streams to and what Kubernetes deploys, byte-for-byte identical, because it is one image.

> **Interview Q:** Why is this project "container-first," and what problem does that solve?
> **A:** Containers package the app with its exact dependencies, so it runs identically everywhere - solving the "works on my machine" problem. That gives reproducibility, which is the foundation of MLOps: you can only deploy, scale, and roll back confidently if behavior doesn't depend on the host. It's also why one build serves everywhere - Kafka streams to and Kubernetes deploys the *same* image.

> **Gotcha:** Reproducibility is not automatic just because you use Docker. If a `Dockerfile` installs "the latest" of a library instead of a pinned version, two builds a month apart can differ. This portfolio pins dependencies and trains the model at build time, so the image is deterministic - a detail worth mentioning in an interview to show depth.

1.5 The three pitches (know all three cold)

Interviewers ask "tell me about a project" at different depths. Prepare all three.

The 30-second pitch:

> "It's a container-first MLOps portfolio built around one spam classifier - a TF-IDF plus logistic-regression model in scikit-learn - that I operate five different ways: I serve it as a Dockerized FastAPI REST API, stream real-time inference through it with Kafka, provision its cloud infrastructure with Terraform against LocalStack, deploy it on Kubernetes with replicas and health checks, and add an AWS Bedrock LLM layer to explain the predictions. The point isn't the model - it's demonstrating the full ML lifecycle around one artifact."

The 2-minute pitch: Expand each of the five verbs into one or two sentences - the serving container and its `/health` and `/predict` endpoints; the Kafka producer/consumer topology feeding the same API; Terraform creating an S3 bucket and DynamoDB table on LocalStack so it's free and safe; the Kubernetes Deployment with two replicas, readiness/liveness probes, and a NodePort Service; and the Bedrock explainer using Claude 3 Haiku via boto3. Close on the theme: everything is Dockerized for reproducibility, and reusing one container across streaming and deployment is what makes the story coherent.

The deep dive: This is the rest of this guide. For any module the interviewer picks, you can explain the code file by file, justify each design choice (why lazy model loading, why KRaft-mode Kafka with no ZooKeeper, why `PAY_PER_REQUEST` DynamoDB, why readiness *and* liveness probes, why credentials come from the environment and are never hard-coded), and discuss trade-offs and what you'd change for real production.

> **Key insight:** Always land the 30-second pitch on the *theme* ("the point isn't the model - it's the operations around it"). It signals seniority: you understand that in industry, model code is a small fraction of a working ML system, and you've deliberately built to show the other 90%.

1.6 Roadmap of the rest of this guide

Here is how the remaining chapters build from absolute zero to advanced. Read in order for a first pass; jump directly to a chapter to prep for a question on that tool.

Ch.	Topic	The tools you'll master
2	ML foundations & the spam model	scikit-learn, TF-IDF, logistic regression, joblib
3	Serving the model as an API	FastAPI, Uvicorn, Pydantic, REST
4	Containerization	Docker, Dockerfiles, images, layer caching
5	(Docker Compose / multi-container)	orchestrating linked containers
6	Real-time streaming	Apache Kafka, KRaft mode, producers & consumers
7	Infrastructure as code	Terraform, LocalStack, S3, DynamoDB
8	Container orchestration	Kubernetes: Deployments, Services, probes
9	GenAI explanation layer	AWS Bedrock, boto3, Claude 3 Haiku

Each chapter defines every term from scratch, quotes the real repository files, and is packed with interview questions and model answers. The repository lives at `/Users/cook/mlops-portfolio` (modules `serving`, `kafka-streaming`, `terraform-localstack`, `kubernetes`, `bedrock-explainer`) and on GitHub at `github.com/Swapnil-byte-798/mlops-portfolio`.

You now have the map. The next chapter starts at the very center of it - the spam classifier itself - and from there we build outward, one operational surface at a time, until you can operate one model five ways and explain every line of it.

2. MLOps Fundamentals From Zero

Before we touch a single tool, you need a mental model of *what problem all these tools are solving*. This chapter builds that model from absolute zero. By the end you will understand what MLOps is, why it exists, the life story of a machine-learning model from raw data to a live service, and how the five modules in this project (see Ch. 3-7) each attack one slice of the problem. Everything after this chapter is detail hanging off the skeleton you learn here.

2.1 What is a "model," really?

A **machine-learning model** is a file. That is the single most important thing to internalize. When people say "we trained a model," what they physically produced is a blob of numbers saved to disk - in this project a file called `model.pkl` (see Ch. 3). That file, plus a tiny bit of code that knows how to feed input into it, can take a piece of text like "WIN a FREE iPhone, click here!!!" and output a label: `spam` or `ham` (ham is the standard jargon for "not spam" - a legitimate message).

The specific model in this project is a **TF-IDF + Logistic Regression spam classifier** built with **scikit-learn** (the most popular classical-ML library in Python). Let's demystify each term:

- **Classifier** - a model whose job is to sort inputs into a fixed set of categories (here: two categories, spam or ham).
- **TF-IDF** (Term Frequency-Inverse Document Frequency) - a recipe for turning words into numbers. Computers can't do math on the word "free"; TF-IDF converts each message into a long list of numbers where words that are common *in this message* but rare *across all messages* get high scores. It's how "the model reads text." Think of it as a fingerprint of a message's vocabulary.
- **Logistic Regression** - a simple, fast, well-understood math formula that takes those numbers and produces a probability between 0 and 1 (e.g. "0.93 chance this is spam"). If the probability crosses a threshold, it outputs `spam`.

You do **not** need to master the math. For MLOps, the model is a black box that eats input and produces output. Our entire job is everything *around* that box.

> **Key insight:** The model is deliberately *boring* in this project - a decades-old algorithm on a two-way classification. That's intentional. The portfolio isn't about a fancy model; it's about the **engineering system** that operates a model in production. Interviewers for MLOps roles care far more about the system than the model.

2.2 The gap: why MLOps exists

Here is the situation that created the entire MLOps discipline. A data scientist opens a Jupyter notebook (an interactive Python coding environment), loads a spreadsheet of example messages, runs some code, and produces a model that is 98% accurate. Everyone celebrates. Then someone asks the real question:

> "Great. Now how do 10,000 users per second actually *use* this from their phones, forever, without it breaking, and how do we know when it stops working?"

The notebook cannot answer that. A trained model sitting in a notebook is like a brilliant chef who has cooked one perfect meal alone in a test kitchen. Running a *restaurant* - taking orders at scale, consistent quality every time, staff, supply chains, health inspections, staying open through a rush - is

a completely different discipline. **MLOps is the discipline of turning a trained model into a reliable, running restaurant.**

MLOps stands for **Machine Learning Operations**. Formally: the set of practices, tools, and cultural habits for deploying and *maintaining* machine-learning models in production reliably and efficiently. It is the ML-specific cousin of **DevOps** (which we define below).

The famous, well-supported observation behind the field: **the model code is a tiny fraction of a real ML system.** A landmark Google paper ("Hidden Technical Debt in Machine Learning Systems") illustrated this with a diagram where the little "ML Code" box is dwarfed by boxes for data collection, serving infrastructure, monitoring, configuration, and more. The model is maybe 5% of the work. The other 95% - the plumbing - is what this portfolio teaches.

> **Interview Q:** In one sentence, what is MLOps and why does it exist?
> **A:** MLOps is the practice of reliably deploying, running, and maintaining ML models in production; it exists because a trained model in a notebook is only about 5% of a real system - the other 95% is the data pipelines, serving infrastructure, orchestration, and monitoring needed to make that model actually usable, reproducible, and trustworthy at scale.

2.3 The ML lifecycle: data -> train -> package -> serve -> monitor -> retrain

Every ML system moves through a cycle. Learn this loop cold; interviewers use it as scaffolding for half their questions.

Stage	What happens	In this project
Data	Collect and clean example inputs with known answers (labeled data)	Example spam/ham messages used to train
Train	Run an algorithm over the data to produce the model file	`train.py` runs scikit-learn, outputs `model.pkl` (Ch. 3)
Package	Wrap the model + code + dependencies into a shippable unit	Docker image built from `serving/` (Ch. 3)
Serve	Expose the model so other software can send inputs and get predictions	FastAPI `POST /predict` endpoint (Ch. 3)
Monitor	Watch whether it's healthy, fast, and still accurate in the real world	`GET /health`, health probes (Ch. 3, Ch. 6)
Retrain	When accuracy decays, gather fresh data and loop back to Train	(Conceptual here; the loop is the point)

The arrow from **Retrain** back to **Data** is what makes this a *cycle*, not a line. Why must you retrain? Because the world changes. Spammers invent new tricks; last year's model gets stale. This decay is called **model drift** (or **data drift**): the live data slowly stops resembling the data the model was trained on, so accuracy silently rots. A model is not a build-once artifact like a bridge - it's a perishable good with a shelf life.

> **Gotcha:** Software has bugs that are usually *loud* - it crashes, throws errors. Model decay is **silent**. The service returns 200 OK and confident-looking answers while quietly becoming wrong. That's why *monitoring* is a first-class MLOps stage, not an afterthought. A candidate who says "we deployed it and moved on" is showing they've never run ML in production.

2.4 Dev vs Ops - and the extra weight ML adds

To understand MLOps you must first understand **DevOps**, its parent.

Historically two teams were at war. **Developers ("Dev")** write new features and want to ship changes *fast*. **Operations ("Ops")** run the servers and want *stability* - every change is a risk. Dev throws code "over the wall"; Ops catches the fallout at 3 a.m. **DevOps** is the cultural and technical movement that tore down that wall: the same people (and, crucially, *automation*) handle both building and running the software. Its core practices are **CI/CD** (Continuous Integration / Continuous Delivery - automatically testing and shipping every code change), **infrastructure as code** (Ch. 5), and heavy monitoring.

DevOps assumes your system is made of **one** thing that changes: **code**. You version code in **Git**, and if you check out an old commit you get *exactly* the old behavior. Reproducibility is (relatively) solved.

ML breaks that assumption by adding **two more moving parts**:

1. **Data** - the training data can change, and different data produces a different model even with identical code.
2. **The model artifact** - the trained `model.pkl` itself, which is neither code nor data but a *product* of running code over data.

So where DevOps versions **code**, MLOps must version **code + data + model** together to get reproducibility. "It works on my machine" becomes "it works with *this code*, trained on *this data*, producing *this exact model file*." This three-way versioning problem is the single biggest thing that makes MLOps harder than plain DevOps.

> **Interview Q:** How is MLOps different from DevOps?
> **A:** DevOps versions and ships one artifact - code - and Git gives you reproducibility. MLOps has three coupled artifacts: code, data, and the trained model. The same code on different data yields a different model, and accuracy decays over time via data drift, so MLOps adds data/model versioning, experiment tracking, a retraining loop, and model-quality monitoring on top of everything DevOps already does.

2.5 The model vs. the system that runs the model

This is the conceptual spine of the whole portfolio, so we'll state it plainly. There are two distinct things:

- **The model** - the `model.pkl` artifact. Small. Portable. Dumb on its own; it's just weights.
- **The platform around it** - the API server, the container it runs in, the streaming pipeline that feeds it, the cloud infrastructure it stores things in, the orchestrator that keeps copies alive, the LLM that explains it. Large. This is where all the engineering lives.

An analogy: the model is an **engine**; the platform is the **entire car** - chassis, wheels, dashboard, fuel system, brakes. A brilliant engine on a workbench moves nobody. This project takes *one modest engine* and builds *five different vehicles* around it to show you the platform-building skills. The same `model.pkl` gets served, streamed, provisioned-for, orchestrated, and explained - the model is the constant; the *system* is what varies.

> **Key insight:** When an interviewer asks "walk me through your project," the strong answer leads with the *system*, not the model: "It's one spam classifier operated five ways to demonstrate serving, streaming, IaC, orchestration, and GenAI explainability." That framing signals you think like an engineer, not just a modeler.

2.6 The five pillars this project touches - and why each matters

Each module is one industrial-strength pattern for operating a model. Here's *why* each exists, one paragraph each. The deep dives are the chapters that follow.

1. Model serving (Ch. 3 - `serving/`). Serving means wrapping the model behind a network endpoint so *any* program can get a prediction by sending an HTTP request - the model becomes a service. Without serving, the model is unreachable; with it, a website, phone app, or another backend can ask "spam or ham?" and get an answer in milliseconds. This project uses **FastAPI** (a Python web framework) running under **Uvicorn** (the server that actually handles network traffic), exposing `GET /health` and `POST /predict`, all packaged in Docker. Serving is the foundational pillar - every other module ultimately talks to this API.

2. Real-time streaming (Ch. 4 - `kafka-streaming/`). Request/response serving is great when a user is waiting for one answer. But many real systems have a *firehose* of events arriving continuously - transactions, clicks, incoming messages - that must be scored the instant they appear. **Streaming** connects a never-ending flow of data to the model. This project uses **Apache Kafka** (an industrial event pipeline) so a *producer* streams messages onto a topic, a *consumer* pulls each one, asks the serving API to classify it, and publishes the result to a `predictions` topic. This matters because real-time fraud detection, content moderation, and alerting all live or die on this pattern.

3. Infrastructure as Code (Ch. 5 - `terraform-localstack/`). Production models need cloud resources - a place to store the model artifact, a database to log predictions. Clicking buttons in a cloud console to create those is slow, error-prone, and impossible to reproduce. **Infrastructure as Code (IaC)** means you *describe* the resources you want in a text file and a tool creates them for you, identically, every time. This project uses **Terraform** to provision an **S3** bucket (cloud file storage) and a **DynamoDB** table (a cloud database) - but points them at **LocalStack**, a fake AWS that runs on your laptop, so you learn real cloud tooling for free. IaC matters because it makes infrastructure versioned, reviewable, and reproducible - the same discipline Git gave to code, now applied to servers.

4. Container orchestration (Ch. 6 - `kubernetes/`). One container serving one model is a single point of failure - if it crashes, or gets overwhelmed, service dies. **Orchestration** is automated management of many containers: keeping the right number alive, restarting the dead ones, load-balancing traffic, and rolling out new versions without downtime. This project uses **Kubernetes** (the industry-standard orchestrator) to run **2 replicas** of the serving container behind a single network address, with **readiness/liveness probes** that automatically detect and heal unhealthy copies. This matters because it's how you get reliability and scale - the difference between a demo and a production service.

5. GenAI / explainability (Ch. 7 - `bedrock-explainer/`). The spam model outputs a bare label - `spam` - but not *why*. For trust, debugging, and compliance, humans often need a plain-language reason. This project adds a **GenAI** (generative AI - large language models that produce human-like text) layer: **boto3** (the AWS Python SDK) calls **AWS Bedrock** (Amazon's managed LLM service) to have **Anthropic's Claude 3 Haiku** explain, in English, why a message looks like spam. This matters because **explainability** and **human-in-the-loop trust** are increasingly required - and because bolting

an LLM onto a classical model is a very current, very hireable pattern.

> **Interview Q:** Why operate one model five different ways?
> **A:** Each way isolates one production concern so it can be learned cleanly: serving is request/response access, Kafka is high-throughput streaming, Terraform is reproducible infrastructure, Kubernetes is reliability and scale through orchestration, and Bedrock is GenAI explainability. Keeping the model trivial and constant makes the *engineering patterns* the star - which is exactly what MLOps interviews probe.

2.7 Reproducibility, containers, and why "it works on my machine" is banned

Reproducibility means: anyone, on any machine, can rebuild and run your system and get the *same* result. It is the north star of both DevOps and MLOps, and the README states it as the project's core promise - *"reproducible on any machine with Docker installed - no local Python/Java version juggling required."*

The classic enemy of reproducibility is the phrase **"it works on my machine."** Your laptop has Python 3.11, a specific scikit-learn version, some system libraries; a colleague's has different ones; the production server has yet another set. The same code behaves differently in each. This is called **environment drift**, and it wastes enormous engineering time.

The solution used everywhere in this project is the **container** (specifically **Docker**). A container packages your code *together with* the exact Python version, the exact library versions, and the exact system dependencies into one sealed, portable box. Whatever runs inside the container runs the same on your laptop, your teammate's laptop, and the cloud. The analogy that gives the technology its name: **shipping containers**. Before standardized steel containers, loading a ship meant hand-stacking mismatched barrels and crates; after, any container fits any ship, crane, and truck. Docker did that for software.

> **Key insight:** "Everything runs in Docker" isn't a technical footnote - it's the *thesis* of the portfolio. It's what lets a reviewer clone the repo and have the whole thing work, which is precisely the reproducibility MLOps demands. Ch. 3 shows the actual `Dockerfile`.

2.8 The 12-Factor App and configuration via environment variables

How should software be built so it's easy to deploy, scale, and reproduce? The widely-cited answer is the **Twelve-Factor App** - a set of twelve principles, originally from the team at Heroku, for building modern, deployable services. You don't need all twelve for interviews, but a few show up constantly in this project:

- **Config in the environment, not the code.** Anything that differs between environments (which cloud endpoint to hit, which AWS credentials, which port) must come from **environment variables** - named values injected into the program from *outside* at run time - never hard-coded. This is why `bedrock-explainer/` reads AWS credentials **from the environment and never hard-codes them** (Ch. 7), and why `terraform-localstack/` swaps the real AWS endpoint for LocalStack's `http://localhost:4566` via configuration (Ch. 5). The same image can run against a fake cloud or a real one purely by changing config.

- **Strict separation of build and run.** You *build* the artifact once (e.g., train the model into the Docker image, see Ch. 3), then *run* that identical artifact everywhere.
- **Port binding / stateless processes.** The service exposes itself on a port and holds no critical state in memory, so you can run many identical copies - which is exactly what makes Kubernetes' 2 replicas (Ch. 6) safe.

> **Interview Q:** Why put configuration in environment variables instead of the code?
> **A:** So one identical build runs unchanged across dev, staging, and production - you only swap the injected config. It keeps secrets like API keys out of source control, makes the app portable and reproducible, and follows the Twelve-Factor "config in the environment" principle. In this project it's why the same serving image can point at LocalStack or real AWS just by changing env vars.

> **Gotcha:** Hard-coding a secret (an AWS key, a password) into your code and committing it to Git is one of the most common and dangerous mistakes in the industry - secrets in Git history are extremely hard to fully purge and are routinely scraped by attackers. "Credentials come from the environment, never hard-coded" is a rule you should state proudly in any interview.

2.9 The mental model to carry into the tool chapters

Zoom out. Here is the whole picture in one frame:

```
ONE model artifact  -->  wrapped by a PLATFORM  -->  operated FIVE ways

model.pkl                Docker (reproducibility)      1. Serve   (Ch.3)
(TF-IDF + LogReg         + config via env vars         2. Stream  (Ch.4)
 spam classifier)        + the ML lifecycle loop:      3. Provision(Ch.5)
                         data->train->package->serve      4. Orchestrate(Ch.6)
                         ->monitor->retrain             5. Explain (Ch.7)
```

Hold these five ideas as you read the rest of the guide:

1. **The model is a file; the system is everything around it.** The chapters are about the system.
2. **The lifecycle is a loop, not a line** - monitoring and retraining close it because models decay.
3. **MLOps = DevOps + data + model versioning**, because ML has three moving parts, not one.
4. **Containers + env-var config = reproducibility**, the non-negotiable foundation under every module.
5. **Each module isolates one production concern** so you can learn it cleanly, then combine them.

With that skeleton in place, the tool chapters stop feeling like five unrelated technologies and start feeling like five answers to one question: *how do you turn a file full of numbers into a service the world can depend on?*

3. The Model: Spam Classification, NLP, and scikit-learn

Every one of the five modules in this project exists to move, serve, provision for, or explain **one** machine-learning model: a spam classifier. Before you can talk intelligently about serving it (Ch. 4), streaming to it (Ch. 5), or deploying it (Ch. 7), you need to understand what the model actually *is* and how it makes decisions. This chapter builds that understanding from absolute zero - from "what is text classification?" up to the exact 41 lines of `serving/train.py` that produce `model.pkl`.

3.1 What is text classification, and what is spam detection?

Machine learning (ML) is the practice of writing programs that learn patterns from examples instead of following rules a human typed out by hand. Rather than a programmer writing `if message contains "free iPhone" then spam`, we show the computer hundreds of labeled examples and let it *discover* which words signal spam.

Classification is the branch of ML where the answer is a *category* (a discrete label) rather than a number. "Is this email spam or not?" is classification. "How much will this house sell for?" would be **regression** (a continuous number). Our task has exactly two categories, so it is **binary classification**.

Text classification is classification where the input is a piece of natural-language text. **Spam detection** is the textbook example: given a message, output one of two labels. In this repo the two labels are:

- `spam` - unwanted junk ("Congratulations you won a free iPhone claim now")
- `ham` - legitimate messages. "Ham" is the traditional, slightly jokey term the anti-spam community uses for "not spam" (e.g. "Hey are we still meeting for lunch tomorrow").

You can see the exact training data in `serving/data/messages.csv`: 20 spam rows followed by 20 ham rows, each a `label,text` pair.

> **Key insight:** Spam detection is a *supervised* learning problem. "Supervised" means every training example comes with the correct answer (its label) already attached, and the model learns by comparing its guesses to those known answers. The alternative, *unsupervised* learning, has no labels and instead finds structure on its own (e.g. clustering).

> **Interview Q:** Classification vs. regression - one sentence each?
> **A:** Classification predicts a discrete category (spam vs. ham, cat vs. dog); regression predicts a continuous numeric value (price, temperature). Our spam model is binary classification because there are exactly two categories.

3.2 The core problem: computers do math, not words

A machine-learning algorithm like Logistic Regression can only do arithmetic on numbers. It has no concept of the *word* "free". So the central challenge of any NLP (**Natural Language Processing** - the field of getting computers to work with human language) task is **feature extraction**: turning raw text into a fixed grid of numbers called a **feature vector**.

A **feature** is a single measurable property of an input. If we decide our features are "how often does the word *free* appear", "how often does *lunch* appear", and so on for every word in our vocabulary,

then each message becomes a row of numbers - one number per word. The full set of numbers for one message is its **vector** (an ordered list of numbers). Turning text into vectors is called **vectorization**, and the component that does it is a **vectorizer**.

Let's build up the two vectorization ideas the project uses: bag-of-words, then TF-IDF.

3.3 Bag-of-words: the simplest text-to-numbers scheme

Bag-of-words (BoW) treats each message as a *bag* (an unordered collection) of its words. It throws away word order entirely - "dog bites man" and "man bites dog" look identical - and just counts how many times each vocabulary word appears.

First you build the **vocabulary**: the set of all distinct words across the whole training set. Suppose, for a tiny 3-message corpus, the vocabulary is `[claim, free, lunch, meeting, prize, won]`. Then each message becomes a count vector over those six columns:

message	claim	free	lunch	meeting	prize	won
"free prize won"	0	1	0	0	1	1
"claim free prize"	1	1	0	0	1	0
"lunch meeting"	0	0	1	1	0	0

That grid of numbers is something Logistic Regression can chew on. But raw counts have a weakness, and TF-IDF fixes it.

> **Gotcha:** Bag-of-words discards word order and grammar. That is fine for spam (the *presence* of "free", "winner", "claim" is what matters) but a real limitation for tasks like sentiment analysis where "not good" != "good". Fixes include **n-grams** (treating "not good" as a single two-word feature).

3.4 Two text-cleaning steps: lowercasing and stop words

Before counting, we clean the text. `train.py` configures the vectorizer with `lowercase=True` and `stop_words="english"`.

Lowercasing converts everything to lower case so that "Free", "FREE", and "free" are treated as the *same* word instead of three different vocabulary entries. Without it you'd split the model's evidence across near-duplicate columns and waste data.

Stop words are extremely common words that carry little discriminating meaning: "the", "is", "a", "to", "and", "your". They appear in spam and ham alike, so they add noise without helping the model tell the classes apart. `stop_words="english"` tells scikit-learn to drop words on its built-in English stop-word list before vectorizing. Notice this shrinks the vocabulary: in a message like "You have been selected for a $1000 gift card", words like "you", "have", "been", "for", "a" are removed, leaving the informative tokens like "selected", "gift", "card".

> **Interview Q:** Why remove stop words?
> **A:** Stop words like "the" and "is" occur in nearly every document regardless of class, so they carry almost no signal about spam vs. ham. Removing them shrinks the feature space, reduces noise, and lets the model focus on discriminating words. TF-IDF (next) also *automatically* down-weights common words, so stop-word removal and TF-IDF are complementary, not redundant.

3.5 TF-IDF, explained intuitively

TF-IDF stands for **Term Frequency - Inverse Document Frequency**. It is a smarter alternative to raw counts. The intuition in one sentence: *a word is important to a document if it appears often in that document but rarely across all the other documents.*

It multiplies two pieces:

Term Frequency (TF) - how often a word appears *in this one message*. If "free" shows up a lot in a message, it's probably relevant to what that message is about. More occurrences -> higher TF.

Inverse Document Frequency (IDF) - how *rare* the word is across the entire collection. A word that appears in almost every message (like "click" if half your data is spam links, or ordinary filler words) is not distinctive, so IDF pushes its weight *down*. A word that appears in only a few messages is distinctive, so IDF pushes its weight *up*. The "inverse" is the key: common across documents -> small weight; rare across documents -> large weight. The standard formula is roughly:

```
idf(term) = log( total_documents / documents_containing_term )
tfidf(term, doc) = tf(term, doc) * idf(term)
```

Worked example. Say we have 100 messages. The word "the" appears in 90 of them; the word "lottery" appears in 3.

- IDF("the") = log(100 / 90) ~ log(1.11) ~ 0.10 -> tiny weight, because it's everywhere.
- IDF("lottery") = log(100 / 3) ~ log(33.3) ~ 3.5 -> large weight, because it's rare and therefore distinctive.

So even if both words appear once in a message, "lottery" gets a TF-IDF score ~35x larger than "the". The vectorizer has *learned* - purely from statistics, no human rules - that "lottery" is a meaningful signal and "the" is not. (In our repo "the" is already gone as a stop word, but the same mechanism down-weights any common surviving word.) scikit-learn's `TfidfVectorizer` also **L2-normalizes** each row by default, meaning it scales every message's vector to unit length so that long messages don't automatically get bigger numbers than short ones.

> **Key insight:** TF-IDF is *fit* on the training data. During `fit`, the vectorizer learns two things and stores them: (1) the vocabulary - which word maps to which column - and (2) each word's IDF value. At prediction time it reuses exactly those learned values. This is why the *same fitted object* must be used for training and serving (see §3.8).

> **Interview Q:** In plain English, what does TF-IDF do?
> **A:** It converts text into numbers by scoring each word high if it's frequent in the current document but rare across the whole corpus. This surfaces distinctive words and suppresses ubiquitous ones, giving a downstream classifier a much cleaner signal than raw word counts.

> **Gotcha:** A word the vectorizer never saw during training (**out-of-vocabulary**) is simply ignored at predict time - it has no column. If a brand-new spam campaign uses vocabulary absent from the training set, the model can miss it. That's an argument for periodic retraining (Ch. 6 covers retraining/CI ideas).

3.6 Logistic Regression: turning the vector into a decision

Now each message is a vector of TF-IDF numbers. **Logistic Regression (LR)** is the classifier that turns that vector into a labeled prediction with a probability.

Despite the word "regression" in its name, Logistic Regression is a *classification* algorithm. Here's the mental model:

1. It assigns a **weight** (a number, positive or negative) to every feature/word. During training it learns these weights. A word like "won" or "prize" earns a large *positive* weight (pushes toward spam); a word like "meeting" or "lunch" earns a *negative* weight (pushes toward ham).
2. For a new message it computes a **weighted sum**: multiply each word's TF-IDF value by that word's learned weight, add them all up, plus a constant called the **bias/intercept**. Call this sum `z`.
3. It squashes `z` through the **sigmoid** (also called logistic) function, `sigmoid(z) = 1 / (1 + e^(-z))`, which maps any number to a value between 0 and 1. That output is a **probability** - the model's estimated chance the message is spam.
4. If that probability is above the **decision threshold** (0.5 by default), predict `spam`; otherwise `ham`.

The **decision boundary** is the surface where the probability equals exactly 0.5 - the dividing line between the two classes. Because LR's `z` is a straight weighted sum, that boundary is *linear*. LR is a **linear classifier**: it draws a (high-dimensional) straight hyperplane separating spam from ham. It cannot learn curvy boundaries, but for TF-IDF text features a linear boundary works remarkably well, which is why the TF-IDF + LR combination is a classic, strong, cheap baseline for text.

> **Interview Q:** Why is Logistic Regression a good fit for TF-IDF text features?
> **A:** Text vectorized with TF-IDF is very high-dimensional and sparse (thousands of possible words, most zero per message), and the classes are often close to linearly separable in that space. Logistic Regression is linear, fast to train, memory-light, gives calibrated probabilities via the sigmoid, and its per-word weights are directly interpretable - you can read off which words push toward spam. That interpretability also feeds the GenAI explainer in Ch. 8.

> **Interview Q:** Is Logistic Regression a regression or a classification model, and why the confusing name?
> **A:** It's a classification model. The name comes from the underlying *logistic function* (sigmoid) it uses to model the probability, and from its lineage in statistical regression on log-odds. But its job is to output a class label, so we use it as a classifier.

On `max_iter=1000`. LR learns its weights by an iterative optimization procedure (gradient-based) that takes repeated steps to minimize prediction error. `max_iter` caps the number of steps. scikit-learn's default is 100, which can be too few to fully **converge** (settle on stable weights) and emits a warning. Setting `max_iter=1000` in `train.py` gives the optimizer enough iterations to converge quietly on this dataset.

3.7 scikit-learn Pipelines: chaining vectorizer + classifier

scikit-learn (imported as `sklearn`) is the standard Python library for classical (non-deep-learning) ML. It gives us `TfidfVectorizer`, `LogisticRegression`, and the glue that binds them: `Pipeline`.

A **Pipeline** chains multiple processing steps into a single object that behaves like one model. Here is the exact definition from `serving/train.py`:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

clf = Pipeline([
    ("tfidf", TfidfVectorizer(lowercase=True, stop_words="english")),
    ("lr", LogisticRegression(max_iter=1000)),
])
clf.fit(texts, labels)
```

The pipeline has two named steps: `"tfidf"` (the vectorizer) and `"lr"` (the classifier). When you call `clf.fit(texts, labels)`, scikit-learn does the right thing automatically:

1. Calls `fit_transform` on the TF-IDF step - learns the vocabulary + IDF weights *and* converts the raw training texts into vectors.
2. Feeds those vectors into `LogisticRegression.fit`, which learns the per-word weights.

At prediction time, `clf.predict(["some text"])` runs the *same* two steps in order: vectorize the raw text with the already-learned vocabulary/IDF, then classify. The crucial point is that **the pipeline accepts raw text at both train and predict time** and hides the vectorization inside itself.

> **Key insight - why Pipeline matters for train/serve consistency:** The vectorizer and classifier are inseparable - the classifier's weights only mean anything relative to the exact vocabulary and IDF values the vectorizer learned. If you saved them as two separate files and someone loaded the wrong vectorizer version, "column 5" might mean "free" in one and "lunch" in the other, and predictions would be silently garbage. Bundling both into one `Pipeline` object that you fit, save, and load as a unit guarantees the serving code (Ch. 4) applies *exactly* the same transformation as training. This is the single most important reason to use a Pipeline in an MLOps setting.

> **Interview Q:** What problem does a scikit-learn Pipeline solve beyond convenience?
> **A:** It prevents **train/serve skew** - the bug where preprocessing at inference time differs from training time. By wrapping preprocessing and the model in one fitted object, the same learned transformation is applied everywhere, and there's exactly one artifact to version and deploy. It also prevents **data leakage** during cross-validation, because the vectorizer is re-fit only on each training fold rather than on the whole dataset.

3.8 fit, predict, and predict_proba

These three methods make up scikit-learn's core API, and `app.py` uses two of them.

- `fit(X, y)` - the *training* call. `X` is the inputs (our list of message strings), `y` is the labels. It learns everything and mutates the model in place. Called once, in `train.py`.
- `predict(X)` - returns the predicted *label* for each input: an array like `["spam"]`. It applies the 0.5 threshold internally.
- `predict_proba(X)` - returns the predicted *probabilities* for each class, as an array of arrays, e.g. `[[0.08, 0.92]]` meaning 8% ham, 92% spam. The columns follow `clf.classes_` (alphabetical: `ham` then `spam`).

Here's how `serving/app.py` uses them to produce a labeled prediction plus a confidence score:

```
label = clf.predict([msg.text])[0]
confidence = float(max(clf.predict_proba([msg.text])[0]))
return {"text": msg.text, "label": label, "confidence": round(confidence, 4)}
```

Note the inputs are always *lists* (`[msg.text]`) because scikit-learn works on batches; `[0]` pulls out the single result. **Confidence** here is defined as `max(...)` of the probability row - the probability of the *winning* class. If the model says 92% spam / 8% ham, the reported confidence is 0.92. That's a simple, honest way to express "how sure was the model."

> **Gotcha:** `confidence` is *not* the probability of spam specifically - it's the probability of whichever class won. A confident ham prediction also reports a high number (e.g. 0.97 ham). Don't misread the confidence field as a spam score.

3.9 Model persistence with joblib and .pkl

Training is expensive and happens once; serving happens millions of times. So after `fit` , we **persist** (save to disk) the trained pipeline and reload it later instead of retraining. `train.py` does:

```
import joblib
joblib.dump(clf, OUT)   # OUT = ".../serving/model.pkl"
```

and `app.py` reloads it:

```
_clf = joblib.load(MODEL_PATH)
```

Serialization means turning an in-memory Python object into a stream of bytes that can be written to a file; **deserialization** is reading it back. Python's built-in serialization format is called **pickle**, and files conventionally use the `.pkl` extension - hence `model.pkl` . **joblib** is a library that does pickling but is optimized for objects containing large NumPy arrays (exactly what ML models are - big grids of learned weights). `joblib.dump` writes the whole fitted `Pipeline` - vocabulary, IDF values, LR weights, and all - into one file; `joblib.load` reconstructs the live object, ready to `.predict` .

> **Interview Q:** Why joblib instead of plain `pickle` for scikit-learn models?
> **A:** joblib uses the same pickle protocol but is optimized for objects dominated by large NumPy arrays, which it stores more efficiently and faster. scikit-learn's own docs recommend joblib for persisting estimators. Functionally it produces the same kind of `.pkl` artifact.

> **Gotcha - pickling security:** Unpickling executes arbitrary code embedded in the file, so **never load a `.pkl` from an untrusted source** - a malicious pickle can run anything on your machine. In this project it's safe because `train.py` builds `model.pkl` at Docker build time from our own code (Ch. 4), so we control the artifact end-to-end. In production you'd store models in a trusted, access-controlled registry and verify integrity (checksums/signing).

> **Gotcha - version pinning:** A pickle is tied to the library versions that created it. Load a scikit-learn model with a different scikit-learn version and you may get warnings or subtle breakage. That's why the serving Docker image pins its dependencies (Ch. 4) - the environment that loads `model.pkl` must match the one that produced it.

3.10 Evaluating a spam classifier: accuracy is not enough

The repo intentionally keeps training simple - it trains on all 40 rows and doesn't compute metrics - because its focus is the *MLOps plumbing*, not squeezing out model quality. But you must be able to discuss evaluation in an interview.

To measure a classifier honestly you split data into a **training set** (used to `fit`) and a held-out **test set** (used only to score), so you measure how it does on messages it has never seen. From the test predictions you build a **confusion matrix**:

	predicted spam	predicted ham
actually spam	True Positive (TP)	False Negative (FN)
actually ham	False Positive (FP)	True Negative (TN)

From it come the key metrics:

- **Accuracy** = (TP + TN) / everything. The fraction of predictions that were correct. Intuitive, but *misleading on imbalanced data*: if 95% of real email is ham, a lazy model that labels *everything* ham scores 95% accuracy while catching zero spam.
- **Precision** = TP / (TP + FP). Of the messages we *flagged as spam*, how many really were? Low precision means legitimate mail lands in the spam folder.
- **Recall** = TP / (TP + FN). Of all the *actual* spam, how much did we catch? Low recall means spam reaches the inbox.
- **F1 score** = the harmonic mean of precision and recall - a single number balancing the two.

> **Key insight - the precision/recall trade-off in spam:** For spam, a **false positive** (a real message from your boss wrongly sent to the junk folder) is usually far more costly than a **false negative** (one spam that slips into your inbox). So spam filters typically favor **high precision**, even at some cost to recall. You tune this by moving the decision threshold: raise it above 0.5 to flag spam only when very confident (^precision, vrecall); lower it to be aggressive (^recall, vprecision).

> **Interview Q:** Why is accuracy a poor metric for spam detection?
> **A:** Spam data is typically class-imbalanced, so a trivial model that predicts the majority class scores high accuracy while being useless. You need precision and recall - and because misclassifying legitimate mail is worse than missing spam, precision is usually the priority. Report the confusion matrix, precision, recall, F1, and ideally ROC-AUC rather than accuracy alone.

> **Interview Q:** How would you improve this specific model for production?
> **A:** Train on far more than 40 examples with a proper train/test split and cross-validation; report precision/recall/F1 instead of nothing; consider n-grams and tuning `TfidfVectorizer` (`min_df`, `max_df`, `ngram_range`) and LR's regularization strength `C`; handle class imbalance with `class_weight="balanced"`; calibrate the decision threshold to the business cost of false positives; and set up periodic retraining as new spam patterns emerge, since TF-IDF ignores out-of-vocabulary words.

3.11 Putting it together

The entire model lifecycle for this project fits in one mental picture:

```
messages.csv (label,text)
      |  load_dataset()
      v
Pipeline.fit(texts, labels)
   +- TfidfVectorizer  -> learns vocabulary + IDF weights, vectorizes text
```

```
   +- LogisticRegression -> learns per-word weights, sigmoid -> probability
      |  joblib.dump
      v
   model.pkl   -- the single deployable artifact
      |  joblib.load (lazily, in app.py)
      v
POST /predict -> clf.predict + clf.predict_proba -> {label, confidence}
```

That `model.pkl` is the star of the whole repo. The serving module (Ch. 4) wraps it in an HTTP API and a Docker image; the Kafka module (Ch. 5) streams messages through that API; Kubernetes (Ch. 7) runs several copies of it; and the Bedrock explainer (Ch. 8) uses an LLM to narrate *why* the classifier - with its readable per-word weights - flagged a given message. Everything downstream assumes the model behaves exactly as this chapter described, which is precisely why bundling vectorizer and classifier into one persisted Pipeline matters so much.

4. Containers & Docker From Scratch

Every one of this project's five modules ships as a **container**, and four of them (serving, kafka-streaming, kubernetes, and the compose stacks) are literally built from `Dockerfile`s in the repo. If you understand Docker, you understand roughly half of MLOps tooling - it is the single most-asked topic in interviews for these roles. This chapter teaches it from zero using the two real Dockerfiles in `serving/` and `kafka-streaming/`.

4.1 The problem containers solve: "it works on my machine"

Imagine you write a Python program that trains a spam classifier. It needs Python 3.11, scikit-learn, FastAPI, and a trained `model.pkl` file. On your laptop it runs. You hand it to a colleague - their Python is 3.9, their scikit-learn is a different version, the model file doesn't deserialize, and it crashes. Multiply that across a team, a CI server, and a production cluster, and you have chaos.

A **container** packages your application *together with everything it needs to run* - a specific Python version, exact library versions, system libraries, config, even data - into one sealed, portable unit. Ship that unit anywhere Docker runs (your Mac, a Linux server, a Kubernetes cluster) and it behaves identically. This property is called **reproducibility**, and it is the whole reason MLOps leans so heavily on Docker.

> **Interview Q:** What problem does Docker solve?
> **A:** It solves environment drift - the "works on my machine" problem. A container bundles the application with its exact runtime, dependencies, and OS libraries, so it runs identically across a developer laptop, CI, and production. That gives you reproducibility and portability, which are prerequisites for reliable deployment.

Docker is the most popular tool for building and running containers. You describe your environment declaratively in a text file called a `Dockerfile`; Docker turns it into an **image**; and you run instances of that image as **containers**.

4.2 Containers vs. virtual machines

Newcomers confuse containers with **virtual machines (VMs)**. Both isolate software, but they work very differently.

A **VM** virtualizes *hardware*. A program called a **hypervisor** carves your physical machine into fake computers, and each VM runs a *complete guest operating system* (its own Linux kernel, boot process, the works) on top. That is heavy: each VM is gigabytes in size and takes minutes to boot.

A **container** virtualizes the *operating system* instead. All containers on a host **share the host's Linux kernel** - there is no guest OS. The kernel just uses two features, **namespaces** (which give each container its own isolated view of processes, network, and filesystem - so it *thinks* it's alone on the machine) and **cgroups** (which cap how much CPU and memory it can use), to keep containers separated. The result is dramatically lighter: containers are tens or hundreds of megabytes, and they start in milliseconds.

	Virtual Machine	Container
Isolates	Hardware	Operating system
Contains a full guest OS?	Yes	No - shares host kernel

	Virtual Machine	Container
Typical size	Gigabytes	Megabytes
Startup time	Seconds to minutes	Milliseconds
Overhead	High	Low

Analogy: A VM is like giving each tenant a separate detached *house* with its own foundation, plumbing, and electrical system. A container is like an *apartment* in one building - each unit is private and lockable (namespaces) with its own metered utilities (cgroups), but they all share the building's foundation and structure (the kernel). Apartments are far cheaper and faster to build.

> **Interview Q:** How is a container different from a VM?
> **A:** A VM runs a full guest OS on virtualized hardware via a hypervisor, so it's heavy - gigabytes, slow to boot. A container shares the host kernel and uses Linux namespaces and cgroups for isolation and resource limits, so it's lightweight - megabytes, boots in milliseconds. Containers give you density and speed; VMs give you stronger, hardware-level isolation and can run a different OS kernel.

> **Gotcha:** Because containers share the *host* kernel, a Linux container needs a Linux kernel. On a Mac or Windows machine, Docker Desktop quietly runs a tiny Linux VM in the background and starts your containers inside it. So on non-Linux hosts you actually get both - but you still only manage containers.

4.3 Images vs. containers

These two words are constantly mixed up. Keep them straight:

- An **image** is a *read-only template* - a frozen snapshot of a filesystem plus metadata (which command to run, which port to expose). It is like a *class* in programming, or a cake recipe. Nothing is running.
- A **container** is a *running (or stopped) instance* of an image - like an *object* instantiated from that class, or an actual baked cake. Docker takes the read-only image and adds a thin **writable layer** on top so the running process can create and modify files.

You can start many containers from one image, each isolated with its own writable layer. In `kubernetes/` (see Ch. 8) the Deployment runs **2 replicas** - that is two containers from the *same* image.

> **Interview Q:** What's the difference between an image and a container?
> **A:** An image is an immutable, read-only template - a packaged filesystem plus metadata. A container is a runnable instance of an image with an added writable layer. One image can spawn many containers, just like one class can instantiate many objects.

4.4 Layers and the union filesystem

Here is the mechanism that makes Docker fast and space-efficient, and it comes up constantly in interviews.

An image is not one monolithic blob - it is a **stack of layers**. Each instruction in a Dockerfile that changes the filesystem (`FROM` , `COPY` , `RUN`) creates a new **layer**: a read-only diff describing

what changed relative to the layer below. Docker stacks these layers and presents them to the container as a single, unified filesystem using a **union filesystem** (a **union mount** - technically OverlayFS on modern Docker), which overlays the layers so the container sees one merged directory tree.

Two big wins come from layers:

1. **Sharing.** Layers are content-addressed (identified by a hash of their contents) and shared across images. If ten of your images all start `FROM python:3.11-slim`, that base layer is stored on disk *once*. Pulling a new image only downloads layers you don't already have.
2. **Caching** during builds - the topic of the next section.

> **Key insight:** Layers are immutable and content-addressed. That's why identical layers are deduplicated on disk and over the network, and why a container's changes go into a separate writable layer instead of mutating the image.

4.5 The build cache - and why we copy requirements *before* source

This is the single most important Dockerfile idiom in the repo, and a classic interview question. Look at the ordering in `serving/Dockerfile`:

```
FROM python:3.11-slim
WORKDIR /app

# Install dependencies first so Docker can cache this layer.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy source + data, then train the model into the image.
COPY train.py app.py ./
COPY data ./data
RUN python train.py
```

When Docker builds an image, it processes instructions top to bottom, producing one layer each. For every instruction it computes a cache key. If nothing that affects that instruction has changed since a previous build, Docker **reuses the cached layer** instead of re-running the step. Crucially, **the moment one layer's cache is invalidated, every layer after it must be rebuilt too** - the cache is a chain, and a break anywhere busts everything downstream.

Now think about what changes most often. Your **application code** (`app.py`, `train.py`) changes many times a day. Your **dependencies** (`requirements.txt`) change rarely. `pip install` is the slow step - it can take minutes downloading and compiling scikit-learn, FastAPI, and friends.

So the Dockerfile deliberately copies **only** `requirements.txt` first and runs `pip install`, *then* copies the source. Because of cache-chain invalidation:

- You edit `app.py` -> the `COPY train.py app.py` layer's cache breaks, but everything *above* it (including the slow `pip install`) is unchanged -> Docker **reuses** the dependency layer and skips reinstalling. Rebuild takes seconds.
- If instead you had written `COPY . .` before `pip install`, then *any* code edit would change that copied layer and invalidate the `pip install` layer beneath it, forcing a full reinstall every single build. Slow and wasteful.

> **Interview Q:** Why copy `requirements.txt` and run `pip install` *before* copying the application source?
> **A:** To maximize Docker's layer cache. Dependencies change rarely and installing them is slow; source changes constantly. By isolating `pip install` into its own layer that only depends on `requirements.txt`, a source-code edit doesn't invalidate that layer, so Docker reuses the cached dependencies and rebuilds finish in seconds instead of minutes. Ordering Dockerfile instructions from least- to most-frequently-changing is the general rule.

> **Gotcha:** `pip install --no-cache-dir` and Docker's *layer* cache are unrelated. `--no-cache-dir` tells **pip** not to keep its own download cache *inside the image* - it keeps the image smaller. Docker's build-layer cache is a separate mechanism that lives on the build host. The Dockerfile uses both: `--no-cache-dir` for a slim image, layer ordering for fast rebuilds.

4.6 Reading `serving/Dockerfile` directive by directive

Every directive in this file matters. Here is what each does, taught from scratch.

`FROM python:3.11-slim` - the base image

Every image is built *on top of* another image, the **base image**, named by `FROM`. `python:3.11-slim` is an official image from Docker Hub containing a minimal Linux plus Python 3.11 preinstalled. The **tag** after the colon (`3.11-slim`) pins the version.

Why **slim**? The default `python:3.11` image is ~1 GB because it bundles build tools, compilers, and extra libraries. The **-slim** variant strips those out to ~150 MB - smaller images pull faster, use less disk, and expose a smaller **attack surface** (fewer packages means fewer potential vulnerabilities). There's an even smaller `-alpine` variant, but Alpine uses a different C library (musl instead of glibc) that can break Python packages with compiled extensions like scikit-learn, so `-slim` is the safe, popular choice.

> **Interview Q:** Why `python:3.11-slim` instead of `python:3.11` or `python:3.11-alpine` ?
> **A:** `-slim` drops build tools and docs to shrink the image (~150 MB vs ~1 GB) and reduce attack surface, while staying on glibc so wheels for scientific packages install cleanly. Alpine is smaller still but uses musl libc, which often forces slow source builds or breaks packages like NumPy/scikit-learn - so slim is the pragmatic sweet spot.

`WORKDIR /app` - set the working directory

`WORKDIR` sets the current directory for every following instruction (and for the container at runtime). If `/app` doesn't exist Docker creates it. It's cleaner and safer than `RUN cd /app`, because `cd` in a `RUN` wouldn't persist to the next instruction - each `RUN` is a fresh shell.

`COPY` - copy files from build context into the image

`COPY <src> <dest>` copies files from the **build context** (the directory you point `docker build` at, usually `.`) into the image. `COPY requirements.txt .` copies one file into `/app`. Later, `COPY train.py app.py ./` and `COPY data ./data` bring in the source and the training data.

> **Gotcha:** `COPY` sources must live *inside* the build context - you can't `COPY ../something`. Use a `.dockerignore` file to exclude junk (like `.git`, `__pycache__`, local `venv`) from the context, which speeds up builds and avoids leaking secrets into the image.

`RUN` - execute a command at *build* time

`RUN` runs a command while the image is being *built*, and the resulting filesystem changes become a new layer. This file has two: `RUN pip install ...` (install deps) and `RUN python train.py` (train the model - see 4.7). Contrast with `CMD`, which runs at *container start* time, not build time.

`EXPOSE 8000` - document the port

`EXPOSE 8000` declares that the app inside listens on port 8000. Critically, `EXPOSE` **does not actually publish or open the port** - it is documentation/metadata for humans and tools. To reach the port from your host you must *publish* it at run time with `-p` (see 4.8).

> **Interview Q:** Does `EXPOSE` make a port reachable from my host?
> **A:** No. `EXPOSE` is purely documentation of which port the container listens on - it doesn't open anything. You make a port reachable by publishing it at runtime with `docker run -p host:container`, which sets up the actual port mapping. `EXPOSE` and `-p` are independent.

`HEALTHCHECK` - teach Docker to check the app is alive

```
HEALTHCHECK --interval=15s --timeout=5s --start-period=90s --retries=5 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')" || e
xit 1
```

A **health check** is a command Docker runs periodically *inside* the container to decide whether it's genuinely healthy - not just "the process exists" but "the app responds." Here it hits the API's `/health` endpoint (see Ch. 5) every 15 seconds. Exit code 0 = healthy, non-zero = unhealthy; the container's status shows `healthy` / `unhealthy` in `docker ps`, and orchestrators can restart or stop routing traffic to unhealthy ones.

The flags:

Flag	Meaning
`--interval=15s`	Run the check every 15 seconds
`--timeout=5s`	A check taking longer than 5s counts as a failure
`--start-period=90s`	Grace window at startup - failures here don't count against the container
`--retries=5`	Mark unhealthy only after 5 consecutive failures

`--start-period=90s` is the interesting one, and the comment in the file explains why: **scikit-learn's first import is slow** on some machines. During that cold start the app isn't answering yet. Without a start period, early failed checks would pile up and the orchestrator might kill the container before it ever finished booting. The 90-second grace period says "don't judge me until I've had time to warm up."

> **Interview Q:** What is `HEALTHCHECK` 's `start-period` for?
> **A:** It's a startup grace window. During `start-period` , failing checks are ignored so a slow-booting app - here, slow because scikit-learn's first import is expensive - isn't prematurely marked unhealthy and killed. After it elapses, normal `interval` / `retries` logic applies. It prevents crash-loops on services with heavy cold starts.

`CMD` - the default command at container start

```
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

`CMD` specifies the process to run when a container starts. This launches **Uvicorn** (the web server, Ch. 5) serving the FastAPI app. It's written in **exec form** - a JSON array - rather than **shell form** (`CMD uvicorn app:app ...`). Exec form runs the binary *directly* as PID 1 with no shell wrapper, so it receives OS signals like `SIGTERM` correctly and shuts down cleanly. Shell form wraps the command in `/bin/sh -c` , which can swallow signals and prevent graceful shutdown.

4.7 CMD vs. ENTRYPOINT, and ENV

The repo uses `CMD` but not `ENTRYPOINT` or `ENV` , and interviewers love asking about all three.

- `CMD` sets the *default* command, and it's **easily overridden**: `docker run myimage python train.py` replaces the whole `CMD` . Good for a default you might want to swap.
- `ENTRYPOINT` sets a *fixed* command that always runs; arguments you pass at `docker run` are *appended* to it rather than replacing it. The common pattern is `ENTRYPOINT ["python"]` + `CMD ["app.py"]` - `ENTRYPOINT` is the executable, `CMD` supplies default args you can override.
- `ENV KEY=value` sets an environment variable baked into the image and visible to the running process. The bedrock module (Ch. 9) reads AWS credentials from the environment - but those are passed at *runtime* (`docker run -e`), never baked in with `ENV` , precisely because secrets must not live inside an image.

> **Interview Q:** `CMD` vs `ENTRYPOINT` ?
> **A:** `CMD` provides a default command/arguments that a `docker run` argument fully overrides. `ENTRYPOINT` defines a fixed executable that always runs, with `docker run` arguments appended to it. Use `CMD` alone for a simple, swappable default; combine `ENTRYPOINT` (the binary) with `CMD` (default args) when you want the container to always run one program but accept argument overrides.

> **Gotcha:** Never `ENV` a secret. Environment variables set in a Dockerfile are stored in image metadata and visible via `docker inspect` and image history to anyone who pulls the image. Pass secrets at runtime with `-e VAR=...` or a secrets manager instead.

4.8 Training the model at build time - a self-contained image

Notice `RUN python train.py` runs *during the build*. The comment says it plainly: "Trains the model at build time so the image is self-contained and reproducible." `train.py` fits the TF-IDF + Logistic Regression classifier on the bundled `data/` and writes `model.pkl` *into the image*. (See Ch. 3 for what the model does and Ch. 5 for how it's served.)

Why bake training into the build rather than train at runtime?

- **Self-contained:** the shipped image already carries a trained `model.pkl`. Start a container and it can predict immediately - no download, no external model store, no network dependency.
- **Reproducible:** the model artifact is pinned to the exact code and data present at build time. Every container from that image has *the identical* model. No "which version of the model is running?" ambiguity.
- **Fast startup:** no training work at boot, so replicas (the 2 in Kubernetes) come up quickly and identically.

> **Interview Q:** What are the trade-offs of training the model inside the Docker build?
> **A:** Baking training into the build makes the image fully self-contained and reproducible - every container ships the exact same model artifact and can serve instantly, which is great for a portfolio or small model. The trade-off is that model and code lifecycles are coupled: retraining means rebuilding the image, and the image is only as fresh as its last build. For large models or frequent retraining you'd instead train separately and load the artifact from a registry or object store (e.g., the S3 bucket in Ch. 7) at startup.

4.9 The kafka-streaming Dockerfile: one image, two roles

`kafka-streaming/Dockerfile` is deliberately minimal:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY producer.py consumer.py ./
```

Same base image, same cache-friendly ordering (`requirements.txt` first). But note there is **no** `CMD`. This single image contains *both* `producer.py` and `consumer.py`, and the compose file (Ch. 6) decides which one each container runs by supplying a `command:` at run time. One image, two roles - this keeps the build simple and guarantees producer and consumer share identical dependencies. There's also no `EXPOSE` or `HEALTHCHECK` because these are background worker scripts, not an HTTP server.

> **Key insight:** A Dockerfile's `CMD` is only a *default*. Omitting it and letting the orchestrator (compose/Kubernetes) inject the command is a clean way to run several different processes from one shared image.

4.10 Working with Docker: the core commands

You now understand the files. Here are the commands that turn them into running software.

```
# Build an image from the Dockerfile in the current directory (.)
# and tag it with a name. The final "." is the build context.
docker build -t spam-api:latest .

# Run a container from that image, publishing container port 8000
# to host port 8000 so you can reach it at http://localhost:8000
docker run -p 8000:8000 spam-api:latest

# List running containers (add -a to include stopped ones)
docker ps

# Stream logs from a container
docker logs -f <container-id>

# Remove a stopped container
```

```
docker rm <container-id>
```

Building and tagging. `docker build -t name:tag .` reads the `Dockerfile`, executes each instruction, and produces a **tagged image**. A **tag** is a human-readable label (`spam-api:latest`, `spam-api:v1.2`). If you omit the tag it defaults to `latest`. Tags let you version images.

Port publishing with `-p`. `-p 8000:8000` means "map port 8000 on the *host* to port 8000 in the *container*." The format is `-p HOST:CONTAINER`. This - not `EXPOSE` - is what actually lets your browser reach the app. You can remap, e.g. `-p 9000:8000` to reach the container's 8000 at your host's 9000.

> **Interview Q:** Walk me through `docker build -t spam-api .` then `docker run -p 8000:8000 spam-api`.
> **A:** `build` reads the Dockerfile, executing each instruction into a cached layer, and tags the resulting image `spam-api:latest`; the trailing `.` is the build context sent to the daemon. `run` starts a container from that image, and `-p 8000:8000` publishes container port 8000 to host port 8000 so the API is reachable at `localhost:8000`. The container then executes the image's `CMD`, launching Uvicorn.

4.11 Why bind to `0.0.0.0`, not `127.0.0.1`, inside a container

The `CMD` runs Uvicorn with `--host 0.0.0.0`. This detail trips up newcomers constantly and is a favorite interview question.

- `127.0.0.1` (a.k.a. `localhost`) is the **loopback** address - it accepts connections *only from the same machine*. Inside a container, "the same machine" means *inside that container's network namespace*. So a server bound to `127.0.0.1` is reachable only by other processes in the same container - the port mapping from `-p` can't reach it, and your requests get "connection refused."
- `0.0.0.0` means "listen on **all** network interfaces." That includes the virtual interface Docker connects to the container, so traffic arriving via the `-p` mapping actually reaches the server.

Because a container is network-isolated, "outside the container" is effectively a different machine as far as the network stack is concerned. To accept external traffic you must bind to all interfaces.

> **Interview Q:** Why does the server bind to `0.0.0.0` instead of `127.0.0.1` inside a container?
> **A:** `127.0.0.1` is loopback - reachable only from within the container's own network namespace, so the published port and outside clients can't connect. `0.0.0.0` binds all interfaces, including the one Docker bridges to the host, so requests via `-p` reach the app. Binding to localhost inside a container is the classic cause of "connection refused" despite a correct `-p` mapping.

4.12 Registries and tags, conceptually

A container **registry** is a server that stores and distributes images - think GitHub, but for Docker images. **Docker Hub** is the default public registry; `python:3.11-slim` was *pulled* from it by `FROM`. Others include AWS ECR, Google Artifact Registry, and GitHub Container Registry.

An image's full name is `registry/repository:tag`, e.g. `docker.io/library/python:3.11-slim`. You **push** images you build to a registry so other

machines - a teammate's laptop, a CI runner, a Kubernetes node - can **pull** and run them:

```
docker tag spam-api:latest myregistry.com/team/spam-api:v1.0
docker push myregistry.com/team/spam-api:v1.0
```

> **Gotcha:** The `latest` tag is *not* magic and *not* automatically "the newest." It's just the default tag name applied when you don't specify one. Two different builds can both be tagged `latest`, and a `latest` image can be older than a `v1.2` one. In production, always pin explicit, immutable version tags (or content digests) so deployments are reproducible - relying on `latest` is a classic source of "which version is actually running?" incidents. This is also why `kubernetes/` sets `imagePullPolicy: IfNotPresent` for its locally-built image (see Ch. 8): it uses the image already on the node instead of trying to pull `latest` from a registry.

> **Key insight:** The full lifecycle is Dockerfile -> `docker build` -> **image** -> `docker push` -> **registry** -> `docker pull` on another host -> `docker run` -> **container**. Every claim about reproducibility in this project rests on that chain: the image is the immutable, shareable unit of deployment, and it's identical everywhere it runs.

4.13 Chapter summary

- A **container** shares the host kernel (via namespaces + cgroups) and is lightweight; a **VM** runs a full guest OS and is heavy.
- An **image** is an immutable, layered template; a **container** is a running instance with a writable layer on top.
- Images are stacks of **layers** presented through a **union filesystem**; layers are content-addressed, so they're deduplicated and cached.
- The **build cache** is a chain - order Dockerfile steps least-changing first. That's why `requirements.txt` is copied and installed before source: source edits don't bust the slow `pip install` layer.
- Key directives: `FROM` (base image; `-slim` for size + glibc), `WORKDIR`, `COPY`, `RUN` (build time), `EXPOSE` (docs only), `HEALTHCHECK` (with `--start-period` for slow scikit-learn cold start), `CMD` (default runtime command, overridable) vs `ENTRYPOINT` (fixed), `ENV` (never for secrets).
- Training at **build time** (`RUN python train.py`) makes the image self-contained and reproducible.
- Publish ports with `-p`, not `EXPOSE`; bind the server to `0.0.0.0` so it's reachable across the container boundary.
- **Registries** store images; **tags** version them - pin explicit tags, never trust `latest`.

5. Docker Compose & Container Networking

In Chapter 4 you learned to build and run a *single* container. But real systems are rarely one container. This project's streaming pipeline (Ch. 6) needs **four** programs running at once - a message broker, a model API, a producer, and a consumer - and they must all start together, find each other, and talk. Managing that by hand with a pile of `docker run` commands is painful and error-prone. **Docker Compose** is the tool that solves this.

5.1 What Docker Compose is

Docker Compose is a tool that lets you define and run a *multi-container application* from a single text file. Instead of typing many long `docker run` commands, you write one file - `docker-compose.yml` - that describes every container, how it's built, how it's configured, and how the containers relate. Then a single command brings the whole system up or tears it all down.

> **Analogy:** If a `Dockerfile` is a recipe for one dish, a Compose file is the *menu for a whole dinner* - it lists every dish, says which need to be cooked fresh vs. bought ready-made, in what order to serve them, and how they're plated together.

The file is written in **YAML** (short for "YAML Ain't Markup Language") - a human-readable format for structured data where **indentation** (spaces, never tabs) defines nesting, `key: value` defines settings, and a leading `-` marks a list item. YAML is used across this project (Compose, Kubernetes in Ch. 8, GitHub Actions).

The core commands:

```
docker compose up          # build (if needed) + create + start everything
docker compose up -d       # same, but detached (runs in the background)
docker compose up --build  # force a rebuild of images before starting
docker compose ps          # list the running services
docker compose logs -f     # stream logs from all services
docker compose down        # stop and remove all containers + the network
```

> **Interview Q:** When would you reach for Docker Compose instead of plain `docker run`?
> **A:** Whenever an application is more than one container and those containers need to be configured and networked together on a single host - a dev/test environment, a demo, or a local reproduction of a service and its dependencies. Compose gives you a declarative, version-controlled description of the whole stack, one command to start/stop it, and an automatic private network with DNS so services find each other by name. For multi-host production orchestration you'd move up to Kubernetes (Ch. 8), but Compose is the standard for local multi-container work.

> **Key insight:** Compose is *declarative*. You describe the desired end state (these services, this config) and Compose makes it so - you don't script the individual steps. This is the same philosophy behind Terraform (Ch. 7) and Kubernetes (Ch. 8), which is why understanding it here pays off everywhere.

5.2 Anatomy of a Compose file

Let's read the project's streaming Compose file top to bottom - every field in it is a core interview topic.

```
name: kafka-streaming

services:
```

```
  kafka:
    image: apache/kafka:3.9.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      # ... more Kafka settings (see Ch. 6)

  spam-api:
    build: ../serving
    environment:
      PYTHONUNBUFFERED: "1"

  producer:
    build: .
    command: python producer.py
    depends_on:
      - kafka
    environment:
      KAFKA_BROKER: kafka:9092

  consumer:
    build: .
    command: python consumer.py
    depends_on:
      - kafka
      - spam-api
    environment:
      KAFKA_BROKER: kafka:9092
      MODEL_API: http://spam-api:8000/predict
```

`name` sets the *project name* (`kafka-streaming`). Compose prefixes resources (network, containers) with it, so this stack's objects don't collide with other Compose projects on the same machine.

`services` is the heart of the file. Each key underneath (`kafka`, `spam-api`, `producer`, `consumer`) is one **service** - a container definition. The service *name* is also the container's hostname on the network (crucial - see §5.3). Here is what each field means:

Field	What it does
`image`	Use a **pre-built image** pulled from a registry (e.g. `apache/kafka:3.9.0`). No building.
`build`	**Build an image** from a Dockerfile in the given directory instead of pulling one.
`command`	Override the container's default startup command.
`environment`	Inject **environment variables** into the container (config without hard-coding).
`depends_on`	Declare start-order dependencies on other services.
`ports`	Publish a container port to the host machine.

`build` vs `image`

This is a frequent interview question, and this file shows both sides cleanly:

- `image: apache/kafka:3.9.0` - the `kafka` service uses an off-the-shelf image maintained by someone else. Compose pulls it from Docker Hub. You'd never build Kafka yourself.
- `build: ../serving` - the `spam-api` service is built from *our own* Dockerfile (the FastAPI serving image from Ch. 3), located one directory up in `../serving`. The value of `build` is

the **build context** - the folder Docker sends to the daemon and where it looks for the `Dockerfile`.

- `build: .` - the `producer` and `consumer` both build from the *current* directory (`kafka-streaming/`), which has its own Dockerfile. They share **one image** but run different programs.

> **Interview Q:** The `producer` and `consumer` both say `build: .` - are they the same container?
> **A:** They're built from the same image (same Dockerfile, same code baked in), but they're two *separate* running containers differentiated by their `command`: one runs `python producer.py`, the other `python consumer.py`. This is a common, efficient pattern - one image, multiple roles selected at runtime by the command. It keeps the build simple and guarantees both sides share identical dependencies.

> **Gotcha:** `build` and `image` can appear *together* in one service. When both are set, Compose builds the image and *tags* it with the `image` name. If only `build` is present (as here), Compose auto-generates an image name from the project and service names.

`command`

The container image has a default command (its `CMD` from the Dockerfile). `command:` **overrides it.** The serving image defaults to launching Uvicorn (Ch. 3), but the `producer` / `consumer` image is told explicitly to run `python producer.py` / `python consumer.py`. Without the override they'd both run whatever the shared Dockerfile's default is - the override is what makes one image behave as two different programs.

`ports` - publishing to the host

```
ports:
  - "9092:9092"
```

The format is `"HOST:CONTAINER"`. This maps port 9092 on your laptop (the host) to port 9092 inside the `kafka` container, so a program running *outside* Docker - say a debugging script on your machine - can reach Kafka at `localhost:9092`.

> **Key insight - the single most important networking distinction:** `ports:` is *only* about host<->container access. Containers talking to *each other* do **not** need `ports` and do **not** use `localhost`. Notice that `spam-api`, `producer`, and `consumer` publish **no ports at all** - yet the consumer happily calls the API. That works because of the internal network, covered next. Publishing a port is like drilling a hole through your house's outside wall; rooms inside the house already have doors between them.

`environment` - configuration by injection

Every service is configured through environment variables rather than hard-coded values:

- `KAFKA_BROKER: kafka:9092` tells the producer/consumer where the broker lives.
- `MODEL_API: http://spam-api:8000/predict` tells the consumer where to POST messages.
- `PYTHONUNBUFFERED: "1"` forces Python to flush its output immediately so logs appear in real time.

The application code reads these with `os.environ`. This is the **Twelve-Factor App** principle of "store config in the environment" - the same image can run in dev, test, or prod by changing variables, with no code edits and no secrets baked into the image.

> **Gotcha:** In YAML, `1` is a number and `true` is a boolean, but environment variable *values* must be strings. That's why you see quotes: `PYTHONUNBUFFERED: "1"` and (in Kafka's config) `KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"`. Omitting the quotes can trigger a type error or surprising coercion.

5.3 Container networking - the top interview topic

This is the concept interviewers probe hardest, because it separates people who've *used* Compose from those who understand it.

The default bridge network

When you run `docker compose up`, Compose automatically **creates a private virtual network** for the project (a Docker "bridge" network) and connects every service to it. A **bridge network** is a software-defined LAN that lives inside your machine; containers on it can reach each other, but it's isolated from your host's real network and from other Compose projects.

Every container gets its own internal IP address on this network. You never have to declare the network - Compose does it for you and names it after the project (e.g. `kafka-streaming_default`).

Service discovery by name (DNS)

Here's the magic. Compose runs an **embedded DNS server** on that network. **DNS** (Domain Name System) is the internet's phonebook - it turns a name into an IP address. On the Compose network, **each service name resolves to that service's container IP.** So:

- The consumer reaches the API at `http://spam-api:8000` - `spam-api` is literally the service name from the Compose file.
- The producer/consumer reach the broker at `kafka:9092` - `kafka` is the service name.
- Kafka *advertises itself* to clients as `kafka:9092` (`KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092`) so that when a client connects, Kafka says "reach me at `kafka:9092`," and DNS resolves `kafka` correctly.

You never hard-code an IP address, and you never need to know one. Container IPs are assigned dynamically and can change between runs; the *name* is stable. This is the whole point of service discovery.

Interview Q: Why does the consumer call `http://spam-api:8000` and not `http://localhost:8000`? And why does Kafka advertise `kafka:9092` instead of `localhost:9092`?
A: Because inside a container, `localhost` means *that container itself* - its own loopback interface - not the host machine and not any sibling container. If the consumer called `localhost:8000`, it would be looking for an API running inside the consumer's *own* container, which doesn't exist. On the Compose network, each service is reachable by its **service name**, which Compose's built-in DNS resolves to the right container's IP. So the consumer uses `spam-api`, and Kafka must advertise `kafka:9092` so clients connect to the broker's network name - advertising `localhost` would tell every client to look inside itself and the connection would fail.

Key insight: `localhost` is per-container. Container-to-container = **service name**. Host-to-container = `localhost:<published-port>` (which requires a `ports` entry). Getting these two backwards is the #1 Compose networking bug. In this project, a human debugging from the laptop uses `localhost:9092` (published), but the consumer *inside* Docker uses `kafka:9092` (service name).

Gotcha - the Kafka advertised-listener trap: Kafka is notorious here. A client first connects to the broker, and the broker *replies with the address to actually use* (the "advertised listener"). If that advertised address is wrong for the client's network, the initial connection succeeds but every subsequent one fails mysteriously. Setting `KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092` is exactly what makes in-network clients (producer, consumer) able to talk to the broker. More on Kafka's listener setup in Ch. 6.

The second Compose file - a one-service stack

Not every Compose file is multi-service. The Terraform/LocalStack module (Ch. 7) has a deliberately tiny one:

```
name: terraform-localstack

services:
  localstack:
    image: localstack/localstack:3
    ports:
      - "4566:4566"
    environment:
      SERVICES: s3,dynamodb
      DEBUG: "0"
```

LocalStack is a local emulator of AWS's cloud APIs, packaged as a single image. Here Compose is used not to network several containers, but simply as a clean, declarative way to run *one* container with the right port and config:

- `ports: "4566:4566"` publishes LocalStack's single "edge" port to the host, because the thing that talks to it - the Terraform CLI (Ch. 7) - runs *on the host*, not in a container. So Terraform reaches it at `localhost:4566`. This is the host-to-container case, and it's exactly why a `ports` entry is needed here.
- `SERVICES: s3,dynamodb` tells LocalStack to emulate only the two AWS services this module uses (an S3 bucket and a DynamoDB table), keeping startup fast and lean.
- `DEBUG: "0"` keeps LocalStack's logging quiet (again quoted because it's an env value).

> **Interview Q:** Why use Compose for a single container instead of `docker run` ?
> **A:** Reproducibility and self-documentation. The Compose file is a version-controlled record of exactly how the container should run - image tag, published ports, and environment - so anyone can `docker compose up` and get an identical LocalStack with no memorized flags. It also fits naturally alongside the multi-service files, so the whole repo has one consistent way to start any module.

5.4 `depends_on` vs. true readiness

`depends_on` controls **start order**. In the streaming file:

```
consumer:
  depends_on:
    - kafka
    - spam-api
```

This tells Compose: start `kafka` and `spam-api` *before* starting the `consumer` . Sensible - the consumer needs both.

But here is the crucial subtlety, and a favorite interview trap. By default, `depends_on` only waits for the dependency container to *start* - to be running - **not for the program inside it to be ready to accept requests.** Kafka's broker takes several seconds to elect a controller and open its listener; the FastAPI app takes a moment to boot Uvicorn and load the model. `depends_on` will happily launch the consumer while Kafka is still initializing.

> **Interview Q:** Does `depends_on` guarantee the consumer won't start until Kafka is actually ready to serve traffic?
> **A:** No. Plain `depends_on` only orders *container startup* - it waits for the dependency to be *started*, not *healthy*. The Kafka broker or the API can still be mid-initialization when the dependent service launches. So the application code must be resilient: it retries the connection with backoff until the dependency responds. That's why the producer and consumer in this project wrap their Kafka/API connections in retry loops rather than assuming the first attempt succeeds. You can strengthen `depends_on` with `condition: service_healthy` tied to a container healthcheck, but even then, defensive retry logic in the app is best practice.

> **Key insight:** "Started" != "ready." Distributed systems must assume dependencies are sometimes unreachable and retry. `depends_on` is a convenience for *ordering*, never a substitute for **application-level readiness handling**. (See Ch. 6 for the producer/consumer retry loops, and Ch. 8 for how Kubernetes solves the *ready* problem more rigorously with readiness probes.)

The stronger form, for reference, ties dependency start to a healthcheck:

```
depends_on:
  kafka:
    condition: service_healthy   # waits for kafka's HEALTHCHECK to pass
```

This project keeps `depends_on` in its simple list form and pushes readiness into the code - a legitimate and common choice.

5.5 Volumes and environment, briefly

Volumes are Docker's mechanism for **persistent or shared storage** that lives *outside* a container's own writable layer. A container's filesystem is ephemeral - delete the container and its data is gone. A

volume survives, and can be mounted into a container at a path.

There are two flavors you should be able to name:

Type	Syntax	Use
Named volume	`myvol:/data`	Docker-managed storage that persists across container restarts (e.g. a database's data directory).
Bind mount	`./local/path:/data`	Maps a folder on the *host* into the container - great for live-editing source in dev.

Neither Compose file in this project declares a `volumes:` block - the streaming stack treats Kafka's data as disposable (fine for a demo), and LocalStack keeps its emulated state in-memory for the run. That's a deliberate simplicity choice, but in a real deployment you'd add a named volume for Kafka's log directory so messages survive a restart.

> **Interview Q:** What's the difference between a named volume and a bind mount, and when do you use each?
> **A:** A named volume is storage managed by Docker in its own area - you reference it by name and Docker handles where it physically lives; it's the right choice for persistent app data like a database, and it's portable across machines. A bind mount maps a specific host directory into the container - you control the exact path, so it's ideal in development for mounting your source code so changes appear live without rebuilding. Named volumes for data you want Docker to own and persist; bind mounts for host files you want to share into the container.

On **environment**, note the two ways to supply it: the inline `environment:` block (used throughout this project) and an `env_file:` pointing to a `.env` file - the latter keeps long or secret-ish config out of the Compose file itself. Compose also auto-reads a `.env` file in the project directory for **variable substitution**, letting you write `image: myapp:${TAG}` and set `TAG` externally. This project keeps config inline because the values are non-secret and few.

5.6 Chapter summary

- **Docker Compose** runs multi-container apps from one declarative YAML file; `up` starts everything, `down` tears it down.
- Each **service** is a container; `build` builds your own image, `image` pulls a ready one; both can share `command` overrides (producer vs. consumer from one image).
- Compose auto-creates a **private bridge network** with **DNS by service name** - the reason the consumer uses `spam-api:8000` and Kafka advertises `kafka:9092`, never `localhost`.
- `ports` is *only* for host<->container access (LocalStack's `4566`, Kafka's `9092` for external tools); container-to-container needs no published port.
- `depends_on` orders **startup**, not **readiness** - the code still retries.
- **Volumes** persist data (named) or share host files (bind mounts); **environment** injects config, keeping images generic and secret-free.

The next chapter puts this network to work: the Kafka streaming pipeline that flows through exactly these named-service connections.

6. Serving the Model: FastAPI, Uvicorn, Pydantic, REST

A trained machine-learning model is just a file sitting on disk - in this project, `model.pkl`, a serialized TF-IDF + Logistic Regression spam classifier (how it gets trained is covered in Ch. 5). A file cannot *do* anything by itself. To make the model useful, something has to accept messages, run them through the model, and hand back answers. The standard way to expose that capability to the outside world is an **HTTP API**: a small web server that other programs talk to over the network. This chapter takes you from "what even is an API" to being able to explain every line of the 46-line `serving/app.py` and defend it in an interview.

6.1 What an API is, and what "serving a model" means

An **API** (Application Programming Interface) is a contract: a defined set of requests a program can make and the responses it will get back. When the API is reached over a network using the same protocol web browsers use, we call it a **web API** or **HTTP API**.

HTTP (HyperText Transfer Protocol) is the language of the web. It works as a request/response cycle: a **client** sends a request ("give me this thing" / "here's some data, do something with it"), and a **server** sends back a response. That's it - one round trip per interaction.

Serving a model means wrapping the model in exactly such a server so that any client - a browser, a `curl` command, a Kafka consumer (Ch. 7), another microservice - can send it an input and receive a prediction, without knowing anything about Python, scikit-learn, or pickle files. The model becomes a **service**: an always-on process listening for requests.

> **Interview Q:** Why serve a model behind an HTTP API instead of importing it directly in each application that needs it?
> **A:** Decoupling and reuse. With an API, the model lives in one place, in one language, with one set of dependencies. Any client in any language can use it over HTTP without installing scikit-learn or loading the pickle. You can scale, version, monitor, and redeploy the model independently of its consumers. In this repo the exact same `/predict` endpoint is consumed by a Kafka consumer, a Kubernetes deployment, and Docker healthchecks - one service, many clients.

6.2 REST basics: resources, methods, JSON, status codes

REST (REpresentational State Transfer) is a set of conventions for designing HTTP APIs. You don't need the academic theory; you need four practical ideas.

1. Resources and URLs. A REST API is organized around *things* you can act on, each addressed by a **URL** (Uniform Resource Locator - the address). In this service the URLs are `/health` and `/predict`. The part after the host (`http://localhost:8000`) is the **path**.

2. HTTP methods (verbs). Each request carries a method that says what *kind* of action it is. The common ones:

Method	Meaning	Analogy
`GET`	Read something; must not change server state	Reading a page in a book

Method	Meaning	Analogy
`POST`	Send data to create or process something	Handing in a form
`PUT`	Replace a resource	Overwriting a file
`DELETE`	Remove a resource	Throwing something away

`GET` is meant to be **safe** (no side effects) and **idempotent** (calling it ten times is the same as once). Our `/health` is a `GET` - it only reports status. Our `/predict` is a `POST` because the caller sends a body of data (the message text) to be processed.

3. JSON. The request and response bodies are usually **JSON** (JavaScript Object Notation) - a plain-text format of key/value pairs that every language can read and write. A prediction response looks like:

```
{"text": "you won a prize", "label": "spam", "confidence": 0.9812}
```

4. Status codes. Every HTTP response carries a three-digit **status code** summarizing the outcome:

Range	Meaning	Examples
2xx	Success	`200 OK`
4xx	Client's fault	`404 Not Found` , `422 Unprocessable Entity` (bad input)
5xx	Server's fault	`500 Internal Server Error`

You will see `422` a lot with FastAPI: it's what the framework returns automatically when the incoming JSON fails validation (more in §6.5).

> **Key insight:** The genius of REST is uniformity. Because everyone agrees on methods, JSON, and status codes, a client written years later in a different language can talk to your model with zero custom glue - just standard HTTP.

6.3 FastAPI: what it is and why it's used here

FastAPI is a modern Python **web framework** - a library that handles the tedious machinery of being an HTTP server (parsing requests, routing URLs to your code, serializing responses to JSON) so you only write the interesting part: what each endpoint does. Think of it as the receptionist that takes every incoming call, figures out who it's for, and passes you a clean, typed message.

Here is the entire application from `serving/app.py` , which we'll dissect piece by piece:

```
import os
import joblib
from fastapi import FastAPI
from pydantic import BaseModel

MODEL_PATH = os.path.join(os.path.dirname(__file__), "model.pkl")

app = FastAPI(title="Spam Classifier API", version="1.0.0")

_clf = None

def get_model():
    """Load and cache the model on first use."""
```

```
    global _clf
    if _clf is None:
        _clf = joblib.load(MODEL_PATH)
    return _clf

class Message(BaseModel):
    text: str

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(msg: Message):
    clf = get_model()
    label = clf.predict([msg.text])[0]
    confidence = float(max(clf.predict_proba([msg.text])[0]))
    return {"text": msg.text, "label": label, "confidence": round(confidence, 4)}
```

`app = FastAPI(...)` creates the application object - the central registry that knows about every endpoint. The `title` and `version` are metadata that show up in the auto-generated docs (§6.4).

Path operations. A **path operation** is the pairing of an HTTP method with a URL path, handled by one function. FastAPI wires them up with **decorators** - the `@app.get(...)` / `@app.post(...)` lines directly above a function. A decorator is Python syntax that "tags" the function; here it tells FastAPI "when a GET request for `/health` arrives, call this function." The decorated function is the **path operation function** (or *handler*).

Notice how little there is: no manual JSON parsing, no manual response building. You `return` a Python dict and FastAPI serializes it to JSON with a `200 OK` automatically. That's the framework earning its keep.

> **Interview Q:** What is a path operation in FastAPI?
> **A:** It's an endpoint definition - one HTTP method plus one URL path bound to a Python function via a decorator like `@app.get("/health")` or `@app.post("/predict")`. FastAPI routes matching requests to that function, injects validated parameters, and turns whatever the function returns into a JSON HTTP response.

Type hints do real work. In `def predict(msg: Message)`, the annotation `: Message` isn't documentation - FastAPI reads it and knows the request body must match the `Message` shape. FastAPI is built on Python **type hints** (the `name: type` syntax) and uses them to drive validation and documentation. This is its defining design choice.

6.4 Automatic OpenAPI docs

Because FastAPI knows the exact shape of every request and response from your type hints, it can generate an **OpenAPI** specification for free. **OpenAPI** (formerly Swagger) is a standard, machine-readable description of an HTTP API - every path, method, and data schema in a single JSON document. FastAPI serves it at `/openapi.json` and, on top of it, two interactive documentation pages:

- `/docs` - a Swagger UI page where you can see every endpoint and *click to send test requests* from the browser.
- `/redoc` - a cleaner, read-only reference rendering of the same spec.

You wrote zero lines to get any of this. For an ML service this is a big deal: teammates consuming your model can discover exactly what `/predict` expects and returns without reading your code.

> **Key insight:** "The docs are generated from the same type hints that enforce validation" - so the documentation can never drift out of sync with the actual behavior. That single-source-of-truth property is why FastAPI is popular for ML serving.

6.5 Pydantic: request models and validation

Pydantic is a data-validation library. You declare what your data should look like as a Python class, and Pydantic guarantees that any data you accept actually matches - coercing types where sensible and raising a clear error where not. FastAPI uses Pydantic under the hood for every request body; the two are inseparable.

In this service the entire input contract is:

```
class Message(BaseModel):
    text: str
```

`BaseModel` is Pydantic's base class; subclassing it turns `Message` into a **request model** (also called a *schema*). It declares: a valid request has one field, `text`, which must be a string. When a `POST /predict` arrives, FastAPI:

1. Reads the raw JSON body.
2. Hands it to Pydantic to build a `Message`.
3. If `text` is missing, or is a number/object where a string is required and can't be coerced, Pydantic rejects it and FastAPI returns `422 Unprocessable Entity` with a JSON body pinpointing the offending field - before your `predict` code ever runs.
4. If it's valid, your function receives a fully-typed `msg` object and you access the value as `msg.text`.

So the client sends `{"text": "hello"}` and your handler gets a guaranteed-valid object. You never write `if "text" not in body` checks; the model *is* the validation.

```
# valid -> 200
curl -X POST localhost:8000/predict -H "Content-Type: application/json" \
  -d '{"text": "free prize claim now"}'

# missing field -> 422, handled entirely by FastAPI/Pydantic
curl -X POST localhost:8000/predict -H "Content-Type: application/json" \
  -d '{"message": "oops wrong key"}'
```

> **Interview Q:** How does FastAPI validate incoming requests, and what happens on bad input?
> **A:** It uses Pydantic models. You declare a `BaseModel` subclass - here `Message` with a `text: str` field - and annotate the handler parameter with it. FastAPI parses the JSON body, validates and coerces it against the model, and if it fails returns a `422` with a detailed error automatically. My handler only runs on already-valid, typed data, which eliminates a whole class of defensive boilerplate and injection-y bugs.

> **Gotcha:** This model has no constraints beyond "is a string," so `{"text": ""}` (empty) is accepted and passed to the classifier. If the interviewer asks how you'd harden it, the answer is Pydantic constraints - e.g. `text: str = Field(min_length=1, max_length=10000)` - to reject empty or absurdly large inputs at the edge.

6.6 ASGI and Uvicorn: what actually runs the server

Your `app` object is not a running server - it's just a description of endpoints. Something has to open a network socket, listen on a port, accept incoming TCP connections, and call `app` for each request. That something is an **ASGI server**, and in this project it's **Uvicorn**.

ASGI (Asynchronous Server Gateway Interface) is a standard *interface* between a Python web framework and the server process that runs it. Its older sibling is **WSGI** (used by Flask/Django's classic setup). The crucial difference is the "A" - **Asynchronous**.

Sync vs. async, briefly. A **synchronous** (sync) server handles one request per worker at a time: while one request waits on something slow (a disk read, a network call), that worker is blocked and idle. An **asynchronous** (async) server can juggle many requests on a single thread: when one request is waiting on I/O, the server switches to progress another, then comes back. Analogy: a sync waiter takes your order, walks to the kitchen, stands there until the food is ready, then serves you before taking the next table; an async waiter takes your order, moves on to other tables while the kitchen cooks, and returns when your dish is up. Async shines for **I/O-bound** work (lots of waiting on the network) - exactly the shape of a prediction microservice being called by Kafka, Kubernetes, and healthchecks at once.

Uvicorn is a lightning-fast ASGI server. It's what you point at `app`:

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Read `app:app` as "in the module `app.py`, use the object named `app`." This exact command is the container's `CMD` (Ch. 5), which is why the Dockerfile `EXPOSE`s 8000 and the service binds there.

> **Interview Q:** What is the relationship between FastAPI and Uvicorn?
> **A:** FastAPI is the framework - it defines the endpoints and the request/response logic - but it doesn't listen on a socket. Uvicorn is the ASGI server that actually runs the process, accepts HTTP connections, and invokes the FastAPI app for each request. FastAPI implements the ASGI application interface; Uvicorn implements the ASGI server side. They meet at the ASGI standard, which is why you could swap Uvicorn for another ASGI server like Hypercorn without touching your app code.

> **Gotcha - `0.0.0.0` vs `127.0.0.1`:** The README calls this out. `127.0.0.1` (localhost) accepts connections *only from inside the same machine/container* - useless when the caller is on the host or in another container. `0.0.0.0` means "listen on all network interfaces," so the mapped port (`docker run -p 8000:8000`, or Kubernetes port-forward) can actually reach it. Binding to localhost inside a container is a classic "why can't I connect?" bug.

6.7 The two endpoints in depth

`GET /health` - the liveness probe

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

This is deliberately trivial: it returns a fixed dict, does no work, touches no model. Its job is to answer one question fast - "is this process alive and able to respond?" This is called a **liveness probe** or **health check**. Orchestrators (Docker's `HEALTHCHECK`, Kubernetes' liveness/readiness probes - Ch.

8) hit it on a schedule; if it stops responding, they conclude the container is dead and restart or stop routing traffic to it.

The endpoint **must not block**. If `/health` did anything slow - loaded the model, queried a database - then during that slow work it would fail to answer probes, the orchestrator would wrongly declare the container unhealthy, and you'd get restart loops. A health check that can be slow is a health check that lies. Hence: return `{"status": "ok"}` and nothing more.

`POST /predict` - the real work

```
@app.post("/predict")
def predict(msg: Message):
    clf = get_model()
    label = clf.predict([msg.text])[0]
    confidence = float(max(clf.predict_proba([msg.text])[0]))
    return {"text": msg.text, "label": label, "confidence": round(confidence, 4)}
```

Step by step:

1. `clf = get_model()` - fetch the classifier (loading it the first time; see §6.8).
2. `clf.predict([msg.text])[0]` - scikit-learn's `predict` expects a *list* of inputs and returns a list of outputs, so we wrap the single text in `[...]` and take element `[0]`. The result is the class label, here the string `"spam"` or `"ham"`.
3. `clf.predict_proba([msg.text])[0]` - `predict_proba` returns, for each input, the probability the model assigns to *each* class, e.g. `[0.02, 0.98]`. `max(...)` picks the largest - the model's confidence in the class it actually chose. `float(...)` converts NumPy's numeric type to a plain Python float so it serializes cleanly to JSON, and `round(..., 4)` trims it to four decimals for a tidy response.
4. Return a dict echoing the input `text`, the `label`, and the `confidence`. FastAPI serializes it to JSON with `200 OK`.

> **Interview Q:** What's the difference between `predict` and `predict_proba`, and why use both here?
> **A:** `predict` returns the hard classification - the single most-likely label. `predict_proba` returns the probability distribution over classes. We use `predict` for the label and `predict_proba + max` to report how *confident* the model is in that label. A bare label is far less actionable than "spam at 0.98" vs "spam at 0.51" - the confidence lets downstream consumers set thresholds or flag borderline cases (this is exactly what the Bedrock explainer in Ch. 9 builds on).

6.8 Lazy model loading with a cached global

This is the most interview-worthy design detail in the file:

```
_clf = None

def get_model():
    """Load and cache the model on first use."""
    global _clf
    if _clf is None:
        _clf = joblib.load(MODEL_PATH)
    return _clf
```

What it does. `_clf` starts as `None`. The model is *not* loaded when the process starts. The first time `get_model()` runs (on the first `/predict`), `_clf is None` is true, so `joblib.load` deserializes `model.pkl` from disk into memory and stores it in the global `_clf`. Every subsequent call sees `_clf` is no longer `None` and returns the already-loaded

object instantly. This pattern is **lazy loading** (do expensive work only when first needed) combined with **caching** (do it once, reuse forever). `joblib` is the serialization library scikit-learn recommends for models; `joblib.load` is the counterpart to the `joblib.dump` that `train.py` used to write the file.

Why do it this way - two reasons, both about the health check:

1. **Fast startup.** Loading scikit-learn + the pickled model can take many seconds (the README warns the first `/predict` can be ~40s on a cold start on slower machines). If that happened at import time, the process wouldn't be ready to answer *anything* until it finished. With lazy loading, the process is up and `app` is serving within a fraction of a second.
2. **The probe never blocks.** Because loading is deferred to the first prediction, `GET /health` is answerable immediately - the orchestrator gets its "ok" long before the model is in memory. The heavy work is confined to `/predict`, where a one-time delay is acceptable. If loading happened at startup, the container could fail its early liveness probes and be killed before it ever became useful.

> **Interview Q:** Walk me through the lazy model-loading pattern and why it matters for a containerized service.
> **A:** A module-level `_clf` starts as `None`; `get_model()` loads the pickle with `joblib` on first call and caches it in that global, returning the cached object thereafter. It matters because model loading is slow. Doing it lazily means the process starts almost instantly, so the health/liveness probe returns "ok" right away and the orchestrator won't kill the container during a slow startup. The cost is a one-time latency on the first prediction, which is a fine trade. It's essentially a lightweight singleton cache.

> **Gotcha - the global and concurrency:** With a single Uvicorn worker this is safe and simple. If you ran multiple workers, each is a separate process with its *own* `_clf`, so the model loads once *per worker* - memory scales with worker count. And if you served with many threads, two simultaneous first-requests could both see `None` and load twice (harmless here - it just wastes work once - but worth naming). Interviewers love when you volunteer these limits unprompted. The production-grade alternative is FastAPI's `lifespan` /startup event or a `@lru_cache`-decorated loader, but the manual global is perfectly correct for this scope.

6.9 How the rest of the repo consumes this API

This one service is the hub the whole portfolio revolves around. The `/predict` and `/health` endpoints are contracts other modules depend on:

Consumer	How it uses the API
Kafka consumer (Ch. 7)	Reads each message off the `messages` topic and `POST`s it to `/predict`, then publishes the `{label, confidence}` result to the `predictions` topic. The API is a plain HTTP dependency of the streaming pipeline.
Kubernetes (Ch. 8)	The Deployment's **readiness** and **liveness** probes both hit `GET /health`; the NodePort Service routes external traffic to `/predict` across 2 replicas.
Docker (Ch. 5)	The image's `HEALTHCHECK` curls `/health` so Docker can mark the container healthy/unhealthy.

Consumer	How it uses the API
Bedrock explainer (Ch. 9)	Builds on the `label` + `confidence` output to ask an LLM *why* a message was classified that way.

Every one of those integrations works precisely *because* the model is behind a uniform HTTP contract instead of being imported directly. Change the model internals, retrain it, swap scikit-learn for something else - as long as `POST /predict` still takes `{"text": ...}` and returns `{label, confidence}`, none of the four consumers need to change.

> **Key insight:** The API is the seam. FastAPI + Pydantic define a stable contract; Uvicorn runs it; lazy loading keeps startup and health checks fast; and REST's uniformity lets Kafka, Kubernetes, Docker, and Bedrock all reuse the same 46 lines. That is the entire point of "serving" a model - turning a `.pkl` file into a dependable, discoverable, orchestratable service.

7. Real-Time Streaming: Apache Kafka

In Ch. 3 the spam classifier lived behind a REST API: you send it one request, it hands back one prediction, and nothing happens unless someone calls it. That is fine when a human clicks a button. But real production systems face a firehose - thousands of chat messages, transactions, or log lines arriving every second, continuously, forever. This chapter builds the `kafka-streaming/` module, which turns the one-off REST model into a **real-time inference pipeline**: messages flow in as a live stream, get classified the instant they arrive, and the results flow out as another stream. The tool that makes this possible is **Apache Kafka**, one of the most heavily interviewed technologies in all of MLOps and data engineering.

7.1 Batch vs. stream processing (from zero)

Before Kafka, understand the two fundamental ways to process data.

Batch processing means collecting data into a big pile and processing the whole pile at once, on a schedule. Analogy: doing a week's laundry every Sunday. You wait until you have a full load, run it, and you are done until next week. In ML terms: "every night at 2 a.m., score all the day's messages and write the results to a table." Simple, efficient, but the answer for a message that arrived at 2:01 a.m. is 24 hours late.

Stream processing means handling each item the moment it appears, one at a time, endlessly. Analogy: a dishwasher who washes each dish as it comes off the table instead of waiting for a pile. In ML terms: "the instant a message arrives, classify it." This is **real-time inference**.

	Batch	Stream
Trigger	Schedule (nightly, hourly)	Each event as it arrives
Latency	Minutes to hours	Milliseconds to seconds
Data shape	Bounded (a finite pile)	Unbounded (never-ending)
Example	Nightly churn report	Live fraud / spam blocking

> **Interview Q:** When would you choose streaming over batch for ML inference?
> **A:** When the *decision has to be acted on immediately* and staleness is costly - fraud detection, spam/abuse filtering, real-time recommendations, alerting. If the consumer of the prediction can tolerate a delay (a daily dashboard, a monthly retraining set), batch is cheaper and simpler. The rule of thumb: match the pipeline's latency to the business's tolerance for staleness, and don't pay for streaming complexity you don't need.

7.2 What Kafka actually is

Apache Kafka is a **distributed event streaming platform**. Strip the jargon: it is a durable, replayable, high-throughput **log of messages** that many producers can write to and many consumers can read from, independently, at their own pace.

The single most useful mental model is the **append-only log**, like a shared notebook where you can only ever add new lines at the bottom, never edit or delete in the middle. Writers append events; readers each keep their own bookmark and read forward. Because the log is *durable* (written to disk) and *retained* for a configured time, a reader can rewind and re-read - something a normal REST call or

a traditional message queue cannot do.

Kafka sits between systems as a **buffer and decoupler**. The producer doesn't need to know who reads the data, how many readers there are, or whether they are online right now. It just appends. This decoupling is the whole point.

> **Key insight:** Kafka is not a database and not a plain queue. Unlike a queue (RabbitMQ, SQS) where reading a message typically removes it, Kafka *keeps* messages after they're read. Consumers track their own position, so ten different teams can each read the same stream for different purposes without interfering with one another.

7.3 Kafka core concepts

These terms come up in nearly every Kafka interview. Learn all of them.

- **Broker** - a single Kafka server process. It stores data and serves reads/writes. A production Kafka **cluster** is several brokers working together; our demo runs exactly one broker (the `kafka` service in the compose file).
- **Topic** - a named stream/category of messages, like a table name or a folder. Producers write *to* a topic; consumers read *from* it. This module uses two: `messages` (raw incoming text) and `predictions` (classified results).
- **Record / message** - one event in a topic. It has a **key** (optional), a **value** (the payload), a timestamp, and metadata. In `producer.py` each value is a small JSON object like `{"id": 3, "text": "..."}`.
- **Partition** - a topic is split into one or more partitions, and *each partition is its own independent append-only log*. Partitions are Kafka's unit of parallelism and ordering. Analogy: a topic is a supermarket queue that's been split into several checkout lanes; more lanes means more shoppers served at once.
- **Offset** - the position number of a record *within a partition*, starting at 0 and only increasing. It is the reader's bookmark. "Consumer X is at offset 47 in partition 2" means it has processed records 0-46 there.
- **Producer** - a client that appends records to a topic (`producer.py`).
- **Consumer** - a client that reads records from a topic (`consumer.py`).
- **Consumer group** - a set of consumers that cooperate to read a topic, sharing the partitions among themselves (detailed in 7.5).

> **Gotcha:** Kafka guarantees ordering **only within a single partition**, never across a whole topic. If total global ordering matters, you need a single partition (which caps throughput), or you route related events to the same partition using a shared key. A classic interview trap is claiming "Kafka keeps everything in order" - it keeps each partition in order.

Here is how those pieces connect in this module:

```
producer.py --> [ messages topic ]  --> consumer.py --> POST /predict --> [ predictions topic ]
                  (append-only log)         |                                  (results log)
                                      spam-api (the FastAPI model from Ch. 3)
```

7.4 Serialization: getting Python objects onto the wire

Kafka itself only moves **bytes** - it has no idea what a Python dict is. **Serialization** is turning an in-memory object into bytes to transmit; **deserialization** is the reverse on the way out. This project uses **JSON** (human-readable text) as the format.

In `producer.py`, the producer is configured to serialize every value by JSON-encoding it and then UTF-8 encoding the resulting string into bytes:

```
KafkaProducer(
    bootstrap_servers=BROKER,
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),
)
```

The consumer does the mirror image - read bytes, decode UTF-8, parse JSON back into a dict:

```
KafkaConsumer(
    IN_TOPIC,
    ...
    value_deserializer=lambda v: json.loads(v.decode("utf-8")),
)
```

> **Interview Q:** JSON vs. a binary format like Avro or Protobuf for Kafka - trade-offs?
> **A:** JSON is human-readable, schema-free, and trivial to debug, which is why demos and low-volume pipelines use it. But it's verbose (field names repeat in every message) and has no enforced schema, so a producer can silently break consumers. At scale, teams prefer **Avro or Protobuf** with a **Schema Registry**: compact binary payloads, enforced and versioned schemas, and safe schema evolution. The cost is added infrastructure and less human-readable messages on the wire.

7.5 Consumer groups: scalable, ordered-per-partition consumption

This is the concept interviewers probe hardest, because it's where Kafka's scaling model lives.

A **consumer group** is identified by a `group_id`. In `consumer.py`:

```
KafkaConsumer(
    IN_TOPIC,
    bootstrap_servers=BROKER,
    auto_offset_reset="earliest",
    group_id="spam-inference",
    value_deserializer=...,
)
```

The rule: **within one group, each partition is assigned to exactly one consumer.** Kafka spreads the topic's partitions across the group's members. This gives you two things at once:

1. **Scalable throughput** - start more consumer instances (up to the number of partitions) and Kafka automatically hands each one a share of the partitions to process in parallel. This automatic reassignment is called a **rebalance**.
2. **Ordered-per-partition processing** - because a partition goes to a single consumer, that consumer sees those records strictly in offset order.

The ceiling: **the number of partitions caps useful parallelism.** If a topic has 3 partitions, a 4th consumer in the group sits idle - there's no free partition to give it. Our demo topics are auto-created with the broker default (typically 1 partition), so scaling the consumer beyond one instance wouldn't add parallelism here; the code is nonetheless written the way a scalable system would be.

Different groups are fully independent: two groups reading `predictions` each get *every* message and track *separate* offsets. That's how one stream feeds a dashboard and an archiver simultaneously.

> **Interview Q:** You have a topic with 6 partitions and start 10 consumers in one group. What happens?
> **A:** Six consumers each get one partition; the other four stay idle with no partitions assigned. To use all ten you'd need at least ten partitions. Partition count is the upper bound on parallelism within a group, so you plan partition count for your peak expected consumer parallelism - bearing in mind that repartitioning a live topic is disruptive.

> **Key insight:** Scale a consumer group *horizontally* by adding instances, not by threading inside one process. Because Kafka assigns whole partitions and preserves per-partition order, adding pods is safe and requires no coordination code from you - the broker handles the rebalance.

7.6 Offsets and delivery semantics

Because each consumer group stores its position (offset) per partition, a consumer that crashes and restarts resumes where it left off - it **commits** offsets as it makes progress. The `auto_offset_reset` setting decides what to do only when a group has *no committed offset yet* (first run):

- `earliest` - start from the oldest retained record (offset 0). Used in `consumer.py`, so on a fresh start it processes the entire backlog and misses nothing.
- `latest` - start from new records only, ignoring history.

This connects directly to **delivery semantics**, the guarantees a pipeline makes about how many times a message is processed:

Semantic	Meaning	Risk
At-most-once	Each message processed 0 or 1 times	May **lose** messages
At-least-once	Each message processed 1 or more times	May **duplicate** messages
Exactly-once	Each message effect happens once	Complex; needs transactions/idempotence

The default consumer behavior here - read, act, then let offsets commit - is **at-least-once**: if the consumer classifies a message, publishes the prediction, but crashes before its offset commits, on restart it reprocesses that message and classifies it twice. For spam classification that's harmless (the same text yields the same label). For something like "charge a credit card," duplicates are dangerous and you'd need idempotency (dedupe by message `id`) or Kafka's transactional exactly-once features.

> **Interview Q:** How do you get exactly-once processing in Kafka?
> **A:** True exactly-once across arbitrary external systems is impossible, so you engineer around it. Two practical paths: (1) Kafka's built-in **transactions / idempotent producer** for Kafka-to-Kafka flows, which atomically commit output records and consumer offsets together; or (2) design consumers to be **idempotent** - key each event by a unique id and dedupe or upsert on that id, so reprocessing a message is a no-op. Most real "exactly-once" systems are actually at-least-once delivery plus idempotent handling.

> **Gotcha:** `auto_offset_reset` is *not* "where do I always start." It only applies when there's no valid committed offset for the group. If the `spam-inference` group already has committed offsets, restarting resumes from them and `earliest` is ignored. Candidates who say "earliest re-reads everything on every restart" get this wrong.

7.7 KRaft mode vs. the old ZooKeeper

Historically, Kafka could not run alone. It needed a *second* distributed system, **Apache ZooKeeper**, to store cluster metadata - which brokers exist, who's the controller, topic configs, and partition leadership. Running Kafka meant running and operating two clusters, which was a well-known source of operational pain.

KRaft (Kafka Raft) removes ZooKeeper. Kafka now manages its own metadata internally using the **Raft** consensus algorithm, with special **controller** nodes electing a leader and replicating metadata among themselves. One system to deploy, faster failover, and support for far more partitions.

You can see KRaft directly in the compose file:

```
KAFKA_PROCESS_ROLES: broker,controller
KAFKA_NODE_ID: 1
KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
```

`KAFKA_PROCESS_ROLES: broker,controller` means this single node plays **both** roles - it stores data (broker) *and* manages metadata (controller) - which is exactly how you run a one-node KRaft demo. `KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093` names the controller quorum: node id `1`, reachable at `kafka:9093`. No `zookeeper` service appears anywhere in the file - that absence is the point.

> **Interview Q:** What is KRaft mode and why did Kafka move to it?
> **A:** KRaft lets Kafka manage its own cluster metadata via the Raft consensus protocol instead of depending on ZooKeeper. It removes an entire external system from the deployment, simplifies operations and security, speeds up controller failover and recovery, and scales to millions of partitions. ZooKeeper is deprecated and KRaft is the default for modern Kafka - as of 3.x it's production-ready and 4.0 removes ZooKeeper entirely.

7.8 Listeners vs. advertised listeners - the classic container gotcha

This trips up almost everyone the first time, and interviewers love it because it reveals whether you actually understand how Kafka connections work.

Connecting to Kafka is a **two-step handshake**:

1. A client contacts a **bootstrap server** to ask "where are the brokers?"
2. The broker replies with its **advertised address**, and the client makes a *brand-new connection* to that advertised address to do real work.

So the address a client ultimately uses is the one the broker *advertises* in step 2 - not the one it dialed in step 1. Two config keys control this:

- `KAFKA_LISTENERS` - the addresses/ports the broker actually **binds and listens on** locally (the sockets it opens). Here: `PLAINTEXT://:9092,CONTROLLER://:9093` - listen on 9092 for clients, 9093 for controller traffic.
- `KAFKA_ADVERTISED_LISTENERS` - the address the broker **hands back to clients** to reconnect on. Here: `PLAINTEXT://kafka:9092`.

Why `kafka:9092` and not `localhost`? Because the producer and consumer run in *their own containers*. On Docker's internal network, `kafka` is the DNS hostname of the broker container (the compose service name). If the broker advertised `localhost:9092`, each client would reconnect to *its own* container's localhost, find nothing there, and fail. By advertising `kafka:9092`, every container is told to reach the broker at the hostname that resolves correctly across the Docker network.

```
KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
```

And the clients use exactly that hostname:

```
BROKER = os.environ.get("KAFKA_BROKER", "kafka:9092")   # producer.py and consumer.py
```

with `KAFKA_BROKER: kafka:9092` injected via the compose `environment:` block.

> **Interview Q:** Kafka connects fine from inside Docker but a client on the host machine times out (or vice-versa). Why?
> **A:** Almost always an advertised-listener mismatch. The broker binds on `KAFKA_LISTENERS` but tells clients to reconnect using `KAFKA_ADVERTISED_LISTENERS`. If it advertises the internal hostname `kafka:9092`, only containers on that Docker network can resolve it; a host process reaching in via a published port needs a *separate* advertised listener (e.g. `localhost:9092` on another port). The fix is multiple named listeners - one internal (`kafka:9092`), one external (`localhost:29092`) - each advertised with the address valid for that audience. It is essentially never a firewall issue; it's what the broker advertised.

> **Key insight:** The port mapping `"9092:9092"` in the compose file publishes the broker to the host, but because this demo only advertises `kafka:9092` (a Docker-internal name), the intended clients are the containerized producer and consumer. Understanding *who resolves which hostname* is the entire lesson here.

7.9 Replication factor 1 - single-node reality

Replication means keeping copies of each partition on multiple brokers so that if one broker dies, a copy survives. The **replication factor** is how many copies exist. The compose file sets several internal topics to factor 1:

```
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
```

Factor 1 means **one copy, no redundancy** - the only sane choice here because there is only one broker; you cannot replicate across brokers you don't have. It is perfect for local development and completely wrong for production, where a factor of **3** is standard so the cluster survives losing a broker.

> **Interview Q:** Why is replication factor 3 the production standard?
> **A:** With factor 3, each partition has a leader plus two followers on different brokers. You can lose one broker and still have a full in-sync copy for both reads and writes; combined with `min.insync.replicas=2` you get durability without halting on a single failure. Factor 1 has no redundancy - losing that broker loses the data - so it's only appropriate for local or throwaway environments like this demo.

7.10 Why inference is a *separate* service from the consumer

Look closely at `consumer.py`: it does **not** load `model.pkl` or import scikit-learn. Instead it makes an HTTP call to the serving API from Ch. 3:

```
API = os.environ.get("MODEL_API", "http://spam-api:8000/predict")
...
r = requests.post(API, json={"text": text}, timeout=90)
```

The docstring states the reasoning outright: keeping inference in the serving API rather than loading the model in the consumer means "the streaming layer and the model service scale independently." This is a deliberate, interview-worthy architecture decision:

- **Independent scaling** - Kafka consumption and model inference have different bottlenecks. If model calls are the slow part, you scale up `spam-api` replicas (Ch. 6 shows exactly this in Kubernetes) *without* touching the consumer, and vice-versa.
- **Decoupling / single source of truth** - one model service is shared by the REST clients, the Kafka pipeline, and anything else. You retrain and redeploy the model in *one* place; every consumer instantly uses the new version. No model file is duplicated into the streaming code.
- **Separation of concerns** - the consumer's job is *stream plumbing* (offsets, deserialization, publishing results). The API's job is *inference*. Each can be developed, tested, and owned separately.

> **Interview Q:** Why call a model API from your Kafka consumer instead of loading the model directly in the consumer for lower latency?
> **A:** In-process loading is lower latency but tightly couples streaming to the model - every model update means redeploying consumers, and you can't scale inference and consumption independently. Externalizing inference behind a service lets each layer scale on its own metric, centralizes the model so all clients share one version, and cleanly separates stream plumbing from ML. If ultra-low latency is critical you can co-locate them, but the default production pattern favors the decoupled service. It's the same trade-off as microservices vs. a monolith.

7.11 Resilience: retry and backoff for startup ordering

Distributed systems must tolerate the fact that services start in an unpredictable order and at different speeds. Kafka takes many seconds to become ready; the model API's first request is slow because the model loads lazily (Ch. 3). The clients handle this with **retry loops and backoff** - try, fail, wait, try again - rather than crashing.

The producer retries until the broker exists:

```
def connect():
    while True:
        try:
            return KafkaProducer(bootstrap_servers=BROKER, ...)
        except NoBrokersAvailable:
```

```
            print("producer: waiting for kafka...", flush=True)
            time.sleep(3)     # backoff, then retry
```

The consumer additionally retries the model API up to 40 times, sleeping 3 seconds between attempts (a generous window for the model's slow cold start), and only gives up after that:

```
def predict(text):
    for _ in range(40):
        try:
            r = requests.post(API, json={"text": text}, timeout=90)
            r.raise_for_status()
            return r.json()
        except requests.RequestException:
            print("consumer: waiting for model api...", flush=True)
            time.sleep(3)
    raise RuntimeError("model API never became available")
```

Compose's `depends_on: [kafka, spam-api]` controls only **start order**, not **readiness** - Docker starts the containers in order but does not wait for Kafka to finish booting or the API to be able to answer. Application-level retries are what actually make the pipeline robust; `depends_on` alone is a well-known insufficient guarantee.

> **Gotcha:** `depends_on` in Compose waits for a container to *start*, not to become *healthy/ready*. Relying on it to guarantee Kafka is accepting connections is a classic mistake - the client will still hit "no brokers available" if it connects too early. The correct fix is retry-with-backoff in the client (as done here), or a healthcheck-based `depends_on: condition: service_healthy`.

7.12 The whole pipeline, end to end

Putting it together, one message's journey:

1. `producer.py` picks a sample text, wraps it as `{"id": i, "text": ...}`, JSON-serializes it, and appends it to the `messages` topic every 2 seconds. `producer.flush()` forces it out immediately rather than buffering.
2. Kafka durably stores that record in a `messages` partition at the next offset.
3. `consumer.py`, a member of group `spam-inference`, reads the record in offset order, deserializes the JSON back to a dict.
4. It POSTs the text to `http://spam-api:8000/predict`; the FastAPI service returns `{"label": ..., "confidence": ...}`.
5. The consumer builds a result record and appends it to the `predictions` topic - a new stream any downstream system can consume.

Topics are created on demand because `KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"` is set (fine for a demo; in production you usually disable auto-create and provision topics explicitly with chosen partition and replication settings). You run the whole thing with `docker compose up --build` and watch classifications stream by with `docker compose logs -f consumer`, where each line reads like `consumer: SPAM (0.98) <- Congratulations you won a free prize claim now`.

Interview Q: Walk me through a real-time ML inference pipeline you've built.
A: A producer publishes incoming messages as JSON to a Kafka `messages` topic. A consumer group reads them in per-partition order with `auto_offset_reset=earliest` for at-least-once processing, and for each message calls a separate model-serving API over HTTP, then publishes the prediction to a `predictions` topic for downstream consumers. Kafka runs in KRaft mode - no ZooKeeper - with advertised listeners set to the broker's Docker hostname so containers can reconnect correctly. Inference is a decoupled service so the streaming layer and the model scale independently, and clients use retry-with-backoff so startup ordering doesn't break the pipeline. That's the shape of the `kafka-streaming` module in this project.

8. Orchestration: Kubernetes

In Ch. 4 you learned to package the spam classifier into a single Docker container, and in Ch. 5 you ran a handful of containers together with Docker Compose. That works beautifully on one laptop. But the moment you want to run in production - across many machines, surviving crashes, handling traffic spikes, upgrading without downtime - you need a bigger tool. That tool is **Kubernetes**. This chapter teaches it from zero, anchored to the two real manifests in `kubernetes/` : `deployment.yaml` and `service.yaml` .

8.1 What "orchestration" means and why it exists

A **container orchestrator** is a system that runs and manages containers for you across a pool of machines. Think of a container as a single musician and the orchestrator as the conductor: you tell the conductor what music you want played (the *desired state*), and the conductor makes sure the right musicians are playing the right parts, replacing anyone who drops out.

Without an orchestrator, *you* are the conductor, and you have to do all of this by hand:

Problem	What you'd do manually	What Kubernetes does for you
Scaling - one container can't handle the load	SSH into servers, start more containers, wire them together	Change `replicas: 2` to `replicas: 10` ; it starts the rest
Self-healing - a container crashes at 3 a.m.	Get paged, log in, restart it	Notices the crash and restarts it automatically
Rollouts - ship a new version safely	Stop old, start new, pray, no easy undo	Gradually replaces old with new; one command to roll back
Scheduling - which machine has room?	Track CPU/RAM in a spreadsheet	Picks a machine with enough free resources
Networking - how do clients find a moving target?	Hard-code IPs that change on every restart	Gives a stable virtual address that follows the pods

Kubernetes (often written **k8s** - "k", then 8 letters, then "s") is the open-source, industry-standard orchestrator, originally from Google. A **cluster** is the whole system: a set of machines (**nodes**) plus a **control plane** (the brain that makes decisions). Your `kubernetes/` module runs against a *local* single-node cluster - Docker Desktop's built-in Kubernetes, or `kind` / `minikube` , per the README - but the exact same YAML would deploy to a 500-node cluster in the cloud. That portability is the whole point.

> **Key insight:** Kubernetes is **declarative**. You don't write a script of steps ("start this, then that"). You write a description of the end state you want, and Kubernetes continuously works to make reality match it. This is the single most important mental model for the entire chapter.

8.2 The object hierarchy: Pod -> ReplicaSet -> Deployment

Kubernetes manages your app through nested **objects** (also called **resources**). You describe each object in YAML and hand it to the cluster. Three of them stack on top of each other.

The Pod - the smallest unit

A **Pod** is the smallest thing Kubernetes runs. It's a wrapper around **one or more containers** that share a network address and storage, and are always scheduled together on the same node. In our project a Pod holds exactly one container: the `mlops-serving:latest` FastAPI image from Ch. 4.

Why not just call it a "container"? Because sometimes you want two containers glued together (an app plus a logging helper, say - a "sidecar"). The Pod is that glue. But 90% of the time, **one Pod = one container**, and that's the case here.

> **Gotcha:** Pods are **ephemeral and disposable** - cattle, not pets. A Pod can die and be replaced at any time, and the replacement gets a **new IP address and a new name**. You should never point a client directly at a Pod's IP; it's a moving target. (That's exactly why Services exist - §8.4.) You almost never create bare Pods by hand either; you let a higher-level object create them for you.

The ReplicaSet - keep N copies alive

A **ReplicaSet** is an object whose one job is: "make sure exactly *N* identical Pods are running at all times." If you ask for 2 and one dies, the ReplicaSet immediately creates a replacement to get back to 2. If somehow 3 exist, it deletes one. This is where **self-healing** and **horizontal scaling** (running more copies) actually live.

The Deployment - manage ReplicaSets over time

A **Deployment** is the object you actually write. It sits on top and manages ReplicaSets *across versions* to give you smooth rollouts and rollbacks. Here's our real file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spam-api
  labels:
    app: spam-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: spam-api
  template:
    metadata:
      labels:
        app: spam-api
    spec:
      containers:
        - name: spam-api
          image: mlops-serving:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
```

Reading this top to bottom:

- `kind: Deployment` - the object type. `apiVersion: apps/v1` says which part of the Kubernetes API defines it.
- `metadata.name: spam-api` - the object's name in the cluster.
- `spec` - the *desired state* ("spec" = specification).
- `replicas: 2` - I want two copies of my Pod running.
- `template` - the **blueprint** for each Pod: this is a mini Pod definition. Every Pod the Deployment creates is stamped out from this template, with the container `spam-api` running image `mlops-serving:latest` and listening on port 8000.

So the chain is: **you create a Deployment -> it creates a ReplicaSet -> the ReplicaSet creates the Pods.** Three layers, each doing one job.

Why the extra layer for rollouts? When you change the template (say, a new image tag), the Deployment doesn't edit the running Pods. Instead it creates a **brand-new ReplicaSet** for the new version and gradually scales the new one up while scaling the old one down - a **rolling update** with zero downtime. The old ReplicaSet isn't deleted; it's kept at 0 replicas as history. That's what makes `kubectl rollout undo` possible: rolling back just means scaling the old ReplicaSet back up and the new one down.

> **Interview Q:** What's the difference between a Pod, a ReplicaSet, and a Deployment?
> **A:** A Pod is the smallest deployable unit - one or more co-located containers sharing a network namespace. A ReplicaSet ensures a specified number of identical Pod replicas are running, replacing any that die. A Deployment manages ReplicaSets to enable declarative rolling updates and rollbacks: on a spec change it spins up a new ReplicaSet and shifts replicas over gradually, keeping the old one at zero so you can undo. In practice you only ever write Deployments; the ReplicaSets and Pods are created for you.

> **Interview Q:** Why would you never run a bare Pod in production?
> **A:** A bare Pod has no controller watching it, so if the node fails or the Pod crashes, nothing recreates it - it's gone. You always wrap Pods in a controller like a Deployment so you get self-healing, scaling, and rolling updates.

8.3 Labels and selectors - how objects find each other

Notice `app: spam-api` appears **four times** across the two files. That's not decoration - it's the mechanism that wires everything together.

A **label** is an arbitrary key-value tag you attach to an object (`app: spam-api` here). A **selector** is a query that says "find all objects with these labels." Kubernetes objects are loosely coupled: they don't reference each other by ID or IP, they reference each other by label match. It's like a coat check - you don't describe your coat, you match a ticket number.

In the Deployment:

```
spec:
  selector:
    matchLabels:
      app: spam-api      # "the Pods I manage are the ones labeled app: spam-api"
  template:
    metadata:
      labels:
        app: spam-api    # "...and here I stamp that label onto every Pod I create"
```

The `selector` and the template `labels` **must match**, or the cluster rejects the Deployment - otherwise a Deployment would create Pods it can't find. The Service (next section) uses the *same* label to find those same Pods.

> **Key insight:** Labels + selectors are the universal glue in Kubernetes. Deployments find their Pods this way; Services route to Pods this way; you filter with them on the command line (`kubectl get pods -l app=spam-api`). There are no hard-wired links - everything is "whatever currently matches this label."

> **Gotcha:** A Service and a Deployment aren't formally connected. The Service just selects on `app: spam-api` and picks up *any* Pod carrying that label. This is powerful (you can point a Service at Pods from two Deployments during a migration) but bites people who typo a label - traffic silently goes nowhere with no error.

8.4 Services - a stable address for moving Pods

Pods come and go with new IPs (§8.2), so we need something stable in front. A **Service** is a Kubernetes object that gives a group of Pods a single, permanent virtual IP and DNS name, and **load-balances** requests across all the matching Pods. It's a receptionist: clients always call the front desk, and the desk forwards to whichever staff member is available, even as staff change.

Our `service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: spam-api
spec:
  type: NodePort
  selector:
    app: spam-api      # send traffic to Pods with this label
  ports:
    - port: 8000        # the Service's own port (inside the cluster)
      targetPort: 8000  # the port on the Pod/container to forward to
      nodePort: 30080   # the port opened on every node (external access)
```

The `selector: app: spam-api` is how the Service finds our two Pods. `targetPort: 8000` matches the `containerPort: 8000` where Uvicorn listens (Ch. 4). Requests hitting the Service get distributed across both replicas automatically - that's built-in load balancing.

The three Service types

Type	Reachable from	Analogy	Use it for
ClusterIP (default)	*Inside* the cluster only	An internal office extension	Service-to-service calls within the cluster
NodePort	Outside, via `<nodeIP>:<nodePort>` on every node	Opening one specific door on the building	Dev/demo, or behind an external load balancer
LoadBalancer	Outside, via a cloud provider's load balancer + public IP	Hiring a valet with a public phone number	Real production on AWS/GCP/Azure

- **ClusterIP** is internal-only. Great for, say, one microservice calling another, but you can't reach it from your laptop's browser.
- **NodePort** - what this repo uses - opens the same port (here **30080**) on *every node* in the cluster. Anything hitting `nodeIP:30080` gets forwarded to the Service, then to a Pod. Ports must be in the 30000-32767 range, which is why it's `30080` and not `80`.
- **LoadBalancer** asks the cloud provider to provision a real external load balancer with a public IP. It only works on a cloud that supports it (not on a plain local cluster), which is why this portfolio uses NodePort.

> **Interview Q:** ClusterIP vs NodePort vs LoadBalancer - when do you use each?
> **A:** ClusterIP is the default and is internal-only - for Pod-to-Pod traffic inside the cluster. NodePort exposes the Service on a static high port on every node's IP, which is handy for local testing or when something upstream (an ingress or external LB) will forward to it. LoadBalancer provisions a cloud provider's external load balancer with a public IP, which is the normal way to expose a Service to the internet in production. They build on each other: NodePort includes a ClusterIP, and LoadBalancer includes a NodePort.

Why `port-forward` for local dev

The README reaches the API like this:

```
kubectl port-forward service/spam-api 8000:8000
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"claim your free prize now"}'
```

`kubectl port-forward` opens a temporary tunnel from a port on *your laptop* (`localhost:8000`) straight into the cluster Service, bypassing all the node-IP/NodePort networking. On a local cluster figuring out the node's IP and hitting `:30080` can be fiddly (Docker Desktop, kind, and minikube each expose it differently), so port-forward is the reliable, no-config way to poke a Service from your machine during development. It's a dev convenience - it runs only while that terminal is open and routes to a single Pod, so it is **not** how you serve real traffic.

> **Gotcha:** `port-forward` is not load-balanced and not for production. It tunnels to one Pod for the life of the command. It's purely a local debugging aid; production traffic goes through the Service (via NodePort/LoadBalancer/Ingress).

8.5 Health probes - readiness vs liveness

Kubernetes can't read your mind about whether the app inside a container is actually *working*. The container process being alive doesn't mean FastAPI has finished loading the model. So you give Kubernetes **probes** - little health checks it runs on a schedule. There are two that matter here, and confusing them is a classic interview trap.

From `deployment.yaml`:

```
readinessProbe:
  httpGet:
    path: /health
    port: 8000
  initialDelaySeconds: 60
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /health
    port: 8000
  initialDelaySeconds: 90
  periodSeconds: 15
```

Both hit the `GET /health` endpoint from Ch. 4 (an HTTP GET returning 200 = healthy). But they answer different questions:

	Readiness probe	Liveness probe
Question it answers	"Is this Pod ready to receive traffic *right now*?"	"Is this Pod still alive, or is it wedged?"
Action on failure	**Remove the Pod from the Service** (stop sending traffic) - but leave it running	**Kill and restart the container**
Analogy	A shop flipping its sign to "Closed" while restocking	Calling maintenance to reboot a frozen machine

- **Readiness** = "should we send traffic?" If it fails, the Service stops routing to that Pod but the Pod keeps running - perfect for a Pod that's still warming up or temporarily overloaded. When our Pod first starts, scikit-learn hasn't imported yet, so `/health` isn't ready; readiness holds traffic back until it is.
- **Liveness** = "should we restart?" If it fails repeatedly, Kubernetes concludes the app is hung and restarts the container. This recovers from deadlocks that a mere "is the process running?" check would miss.

Why the generous `initialDelaySeconds`? `initialDelaySeconds` tells Kubernetes to wait that many seconds after the container starts *before running the probe the first time*. Here readiness waits **60s** and liveness waits **90s**. The reason is called out right in the file's comments: **scikit-learn's first import is slow**, and the model loads lazily on the first request (Ch. 4). If the liveness probe fired immediately, it would find `/health` not-yet-ready, declare the Pod dead, and restart it - over and over, a **crash loop**, so the Pod never gets a chance to finish starting. The delays give the slow Python/scikit-learn startup room to complete.

> **Key insight:** Liveness delay (90s) is deliberately *longer* than readiness delay (60s). You want readiness to gate traffic early, but you must give liveness extra slack so it never kills a Pod that's simply still booting. A liveness probe that's too aggressive is worse than none - it turns a slow start into an infinite restart loop.

> **Interview Q:** What's the difference between a readiness and a liveness probe, and what goes wrong if you misconfigure them?
> **A:** Readiness decides whether a Pod receives traffic - on failure it's pulled out of the Service's rotation but keeps running, so it's for warm-up and transient overload. Liveness decides whether a Pod gets restarted - on failure the container is killed and recreated, so it's for unrecoverable hangs. The classic misconfiguration is a liveness probe with too short an initial delay on a slow-starting app: it kills the container before startup finishes, causing a permanent crash loop. That's exactly why this project sets a 90-second liveness delay for the slow scikit-learn import.

8.6 Resource requests vs limits

Containers share a node's CPU and memory, so you tell Kubernetes how much each Pod needs. From our Deployment:

```
resources:
  requests:
    cpu: "100m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

- A **request** is the amount **guaranteed** to the container. The **scheduler** (the control-plane component that decides which node a Pod runs on) uses requests to place Pods: it only puts a Pod on a node that has at least `100m` CPU and `256Mi` memory free. Requests are about **scheduling** - reserving a seat.
- A **limit** is the **ceiling** the container may not exceed. Limits are about **enforcement** at runtime.

The units:

- **CPU** - `100m` means "100 **millicores**", i.e. 0.1 of one CPU core (1000m = one full core). So this Pod is guaranteed a tenth of a core and may burst up to half a core (`500m`).
- **Memory** - `256Mi` is 256 **mebibytes** (Mi = 1024-based, ~ MB). Guaranteed 256Mi, hard cap 512Mi.

What happens at the limit is different for the two resources, and this is a favorite gotcha:

Resource	Exceed the limit ->	Why
CPU	**Throttled** - the container is slowed down but keeps running	CPU is compressible; you can just give it fewer time-slices
Memory	**OOMKilled** - the container is killed (Out Of Memory) and restarted	Memory is incompressible; you can't "slow down" RAM usage

> **Interview Q:** What's the difference between a resource request and a limit?
> **A:** A request is what the container is guaranteed and is what the scheduler uses to decide which node the Pod fits on - it reserves capacity. A limit is the hard maximum at runtime. The key subtlety is behavior at the limit: exceeding a CPU limit only throttles the container because CPU is compressible, whereas exceeding a memory limit gets the container OOMKilled and restarted because memory can't be reclaimed on the fly. So under-setting a memory limit causes restarts, while under-setting a request causes bad scheduling and contention.

> **Gotcha:** If you set no requests, the scheduler assumes the Pod needs almost nothing and may cram it onto an overloaded node. If you set no limits, one runaway Pod can starve every other Pod on the node. Setting both, as this manifest does, is the responsible default.

8.7 imagePullPolicy: IfNotPresent

```
image: mlops-serving:latest
imagePullPolicy: IfNotPresent
```

imagePullPolicy tells Kubernetes when to fetch the container image from a **registry** (a remote store of images, like Docker Hub). The three values:

- **Always** - pull from the registry every time a Pod starts.
- **IfNotPresent** - use the copy already on the node if it exists; only pull if it's missing.
- **Never** - never pull; fail if it's not already local.

This project builds the image *locally* (`docker build -t mlops-serving:latest ../serving`) and never pushes it to a registry. So `IfNotPresent` is exactly right, as the file's comment says: **use the locally built image; don't try to pull from a registry.** With the default policy of `Always` (which Kubernetes uses whenever the tag is `latest`), the cluster would try to pull `mlops-serving:latest` from Docker Hub, not find it, and the Pods would fail with `ErrImagePull` .

> **Interview Q:** Why does this manifest set `imagePullPolicy: IfNotPresent` explicitly?
> **A:** Because the image is built locally and never pushed to a registry. Kubernetes defaults the pull policy to `Always` for the `:latest` tag, which would make it try to pull from Docker Hub and fail with ImagePullBackOff. `IfNotPresent` tells it to use the image already present on the node. In real production you'd instead push an immutably tagged image (never `:latest`) to a registry and let the cluster pull it.

> **Gotcha:** Even with `IfNotPresent`, a multi-node cluster like `kind` won't have your locally built image on its worker nodes - which is why the README notes `kind load docker-image mlops-serving:latest`. The build happens on your host's Docker; the cluster's nodes are separate.

8.8 Declarative apply and the reconciliation loop

Everything above is glued together by one idea: the **reconciliation loop**. Kubernetes controllers run a never-ending loop - *observe* the current state, *compare* it to the desired state you declared, and *act* to close the gap - forever. Declared 2 replicas but only 1 is running? Start one. A node vanished taking a Pod with it? Reschedule it elsewhere. It's a thermostat: you set 21°C once, and it keeps nudging the heating until the room matches, then keeps watching.

This is why the workflow is **declarative apply**, not imperative scripting:

```
kubectl apply -f deployment.yaml   # "make the cluster match this file"
kubectl apply -f service.yaml
```

`kubectl apply` sends your desired-state YAML to the control plane and says "converge to this." Run it again with an unchanged file and nothing happens (the state already matches - this property is **idempotency**: applying the same thing repeatedly is safe). Edit `replicas` to 5 and re-apply, and Kubernetes adds three Pods. You never say *how*, only *what*.

> **Key insight:** The reconciliation loop is *why* Kubernetes self-heals. Self-healing isn't a special feature bolted on - it's the natural consequence of a control loop that constantly drives actual state toward declared state. Kill a Pod by hand and one reappears, because the ReplicaSet's loop noticed the count dropped below 2.

> **Interview Q:** What does "declarative" mean in Kubernetes, and why does it matter?
> **A:** Declarative means you describe the desired end state in manifests and let Kubernetes figure out the steps to reach and maintain it, rather than issuing imperative commands. Controllers run continuous reconciliation loops that compare actual to desired state and correct any drift. It matters because it gives you idempotency, self-healing, and version-controllable infrastructure - the same YAML in Git always produces the same result, and the cluster keeps itself in that state without babysitting.

8.9 kubectl basics

kubectl ("kube control") is the command-line tool you use to talk to a cluster's control plane. The commands from this module's README, annotated:

```
kubectl cluster-info                    # is a cluster reachable?
kubectl apply -f deployment.yaml        # create/update from a manifest
kubectl apply -f service.yaml
```

```
kubectl get pods -l app=spam-api -w        # list Pods with label app=spam-api; -w = watch live
kubectl port-forward service/spam-api 8000:8000   # tunnel local :8000 -> the Service
kubectl delete -f service.yaml -f deployment.yaml # tear it all down
```

A few more you'll want in an interview:

```
kubectl get deployments,replicasets,pods   # see the whole hierarchy at once
kubectl describe pod <name>                # detailed status + recent events (great for debugging)
kubectl logs <pod>                         # a container's stdout/stderr
kubectl rollout status deployment/spam-api # watch a rolling update progress
kubectl rollout undo   deployment/spam-api # roll back to the previous ReplicaSet
kubectl scale deployment/spam-api --replicas=5   # imperative scale (or edit YAML + apply)
```

The `-l app=spam-api` flag is the same label selector from §8.3 - proof that labels aren't just internal wiring, they're how *you* slice and filter too. When a Pod misbehaves, `kubectl describe pod` and `kubectl logs` are your first two stops: `describe` shows the Events (why it won't schedule, why a probe failed, `ImagePullBackOff`, `OOMKilled`), and `logs` shows what the app itself printed.

> **Interview Q:** A Pod is stuck in `CrashLoopBackOff`. How do you debug it?
> **A:** `CrashLoopBackOff` means the container keeps starting and exiting, so Kubernetes is backing off between restarts. I'd run `kubectl describe pod` to read the Events and the last state/exit code, then `kubectl logs <pod> --previous` to see what the crashed instance printed. Common causes are the app erroring on startup, a missing config/secret, an OOMKill from too low a memory limit, or - very relevant here - a liveness probe with too short an initial delay killing a slow-starting app before it's ready. The fix depends on the cause: raise the memory limit, lengthen `initialDelaySeconds`, or fix the app config.

> **Interview Q:** Walk me through what happens end to end when you run `kubectl apply -f deployment.yaml` for this project.
> **A:** kubectl sends the manifest to the API server, which stores the desired state. The Deployment controller sees a new Deployment and creates a ReplicaSet for `replicas: 2`; the ReplicaSet controller creates two Pods from the template. The scheduler places each Pod on a node with enough free CPU and memory to satisfy the `100m / 256Mi` requests. The kubelet on each node pulls the image - here it uses the local `mlops-serving:latest` because of `IfNotPresent` - and starts the container. For the first 60 seconds the readiness probe is silent, so the Service (once applied) routes no traffic; once `/health` returns 200 the Pod is marked Ready and the Service's `app: spam-api` selector adds it to the load-balanced pool. From then on, controllers keep reconciling: if a Pod dies, the ReplicaSet makes a new one to hold the count at two.

8.10 Chapter summary

Kubernetes takes the container you built in Ch. 4 and runs it *reliably at scale*. You declare a **Deployment** (`replicas: 2`), which creates a **ReplicaSet**, which creates **Pods** - three layers giving you self-healing and safe rolling updates/rollbacks. **Labels and selectors** (`app: spam-api`) wire everything together with no hard-coded links. A **Service** of type **NodePort** (30080) gives the moving Pods a stable, load-balanced address, and **port-forward** is the quick way to reach it locally. **Readiness** ("send traffic?") and **liveness** ("restart?") probes hit `/health`, with generous initial delays (60s/90s) so the slow scikit-learn import doesn't get killed mid-boot. **Requests vs limits** control scheduling vs runtime capping (CPU throttles, memory OOMKills). `imagePullPolicy: IfNotPresent` uses the locally built image. And underneath it all, the **reconciliation loop** - continuously driving actual state toward the declared state - is what makes the whole thing self-healing and declarative. Next, Ch. 9 adds a GenAI explanation layer with AWS Bedrock.

9. Infrastructure as Code: Terraform

So far the spam classifier has been a program (Ch. 5) wrapped in a container (Ch. 6), fed by a streaming pipeline (Ch. 7). But every real ML system needs somewhere to *store* things in the cloud - a place for the trained `model.pkl`, a database for the predictions it makes. That "somewhere" is **infrastructure**: buckets, databases, networks, servers, permissions. This chapter is about how you create and manage that infrastructure with code instead of by clicking around a web console. The tool is **Terraform**, and the module is `mlops-portfolio/terraform-localstack/`.

9.1 What "Infrastructure as Code" means, and why anyone bothers

Infrastructure is the collection of cloud resources your application runs on: storage buckets, databases, virtual machines, load balancers, DNS records, access policies. **Provisioning** infrastructure means creating those resources so they exist and are configured correctly.

There are two ways to provision. The old way is **click-ops**: you log into the AWS web console, click "Create bucket", type a name, tick some boxes, and repeat for every resource. It works once. It is a disaster to reproduce. Nobody remembers exactly which boxes were ticked; the staging environment quietly drifts away from production; and when the person who set it up leaves, the knowledge leaves with them.

Infrastructure as Code (IaC) replaces the clicking with text files. You *describe* the infrastructure you want in a file, commit that file to Git, and a tool reads it and makes reality match the description. Think of it as the difference between assembling furniture by memory versus following (and keeping) the printed instructions.

> **Analogy:** click-ops is cooking without a recipe - it might taste great once, but you can't reliably make it again or tell someone else how. IaC is writing the recipe down: anyone can reproduce the exact dish, you can review changes to it, and you can track its history.

The concrete payoffs, and the words interviewers want to hear:

Benefit	What it means in practice
Repeatable	`terraform apply` builds identical infra every time - dev, staging, prod all from the same code.
Reviewable	Infra changes arrive as a Git pull request your teammates can read and approve before anything happens.
Versioned	Every change is a commit. You can see who changed the database config in March, and roll back.
Self-documenting	The `.tf` files *are* the documentation of what exists. No stale wiki page.
Auditable / recoverable	If a region burns down, you re-run the code and rebuild.

> **Interview Q:** What is Infrastructure as Code and why is it valuable?
> **A:** IaC is managing infrastructure - servers, storage, databases, networking - through declarative machine-readable files kept in version control, instead of manual console clicks or one-off shell scripts. The value is repeatability, peer review, versioning, and disaster recovery: the same code produces the same environment every time, changes go through pull requests, and the definition of your infrastructure lives beside your application code. Terraform is the most common tool for this.

9.2 Terraform, declarative vs. imperative

Terraform is an open-source IaC tool by HashiCorp. You write files ending in `.tf` describing the resources you want; Terraform figures out the API calls needed to make them exist, in the right order, and keeps track of what it created.

The crucial idea is that Terraform is **declarative**, not **imperative**.

- **Imperative** means you write step-by-step instructions: "create bucket, then create table, then set this attribute." A bash script calling the AWS CLI is imperative. *You* own the logic of what to do and in what order, and you have to handle "does it already exist?" yourself.
- **Declarative** means you describe the **desired end state** - "I want a bucket named X and a table named Y" - and the tool computes the difference between what exists now and what you asked for, then does only what's needed to close the gap.

> **Analogy:** imperative is turn-by-turn driving directions ("left, then right, then straight for 2 miles"). Declarative is giving a destination to a GPS and letting it find the route from wherever you currently are. If you're already halfway there, the GPS doesn't start you over.

This is why running `terraform apply` twice is safe: the second time, reality already matches the desired state, so Terraform does nothing. That property is called **idempotency** - applying the same configuration repeatedly yields the same result. Imperative "create bucket" scripts, by contrast, error out on the second run because the bucket already exists.

> **Interview Q:** What's the difference between Terraform and a shell script that calls the cloud CLI?
> **A:** A shell script is imperative - it's a fixed sequence of commands, and you have to manually handle ordering, existing resources, and updates. Terraform is declarative: you describe the desired state, and it diffs that against the current state and makes only the necessary changes. That gives you idempotency, an automatic plan/preview of changes, dependency resolution between resources, and state tracking so it knows what it manages. A script has none of that for free.

> **Key insight:** "Declarative desired state" is the single most important phrase for a Terraform interview. Everything else - plan, state, idempotency - follows from it.

9.3 HCL: the language you write Terraform in

Terraform files are written in **HCL** - **HashiCorp Configuration Language**. It's a purpose-built configuration language designed to be readable by humans and parseable by machines, sitting somewhere between JSON and a real programming language. HCL is built from **blocks** (`type "label" { ... }`) and **arguments** (`name = value`). Let's read the module's `main.tf` top to bottom; every construct you need is in it.

The `terraform {}` block - pinning versions

```
terraform {
  required_version = ">= 1.5"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

This block configures Terraform itself (not any cloud resource). Two jobs:

- `required_version = ">= 1.5"` - the Terraform CLI must be version 1.5 or newer, or it refuses to run. This stops a teammate on an ancient version from silently misbehaving.
- `required_providers` - declares which **providers** this code needs. A **provider** is a plugin that teaches Terraform how to talk to one specific platform's API. `hashicorp/aws` is the plugin for Amazon Web Services; there are providers for Google Cloud, Azure, Kubernetes, GitHub, Datadog, and hundreds more. `source = "hashicorp/aws"` tells Terraform where to download it from (the public Terraform Registry). `version = "~> 5.0"` constrains which version of that plugin is acceptable.

Those `>=` and `~>` symbols are **version constraints**, and interviewers love asking about `~>` specifically:

Constraint	Meaning	Allows	Blocks
`>= 1.5`	at least 1.5	1.5, 1.9, 2.3, ...	1.4
`~> 5.0`	"pessimistic" - allow patch/minor within 5.x	5.0, 5.4, 5.99	6.0
`~> 5.1.0`	allow patch only	5.1.3	5.2.0
`= 5.31.0`	exactly this	5.31.0	everything else

> **Interview Q:** What does the `~>` (pessimistic) version constraint mean?
> **A:** It pins the leftmost specified component and allows the rightmost to increase. `~> 5.0` means "any 5.x version but not 6.0" - you accept new minor and patch releases of the AWS provider automatically but never a major version, because major versions can contain breaking changes. It's the standard way to get bug fixes without surprise breakage.

> **Gotcha:** version constraints in `.tf` files are your *intent*. The exact versions actually installed get recorded in a separate `.terraform.lock.hcl` lock file (created by `terraform init`), which you commit so every teammate and CI run uses byte-identical provider versions. Constraint = allowed range; lock file = the specific pick.

The `provider {}` block - configuring the AWS plugin

```
provider "aws" {
  region                      = "us-east-1"
  access_key                  = "test"
  secret_key                  = "test"
  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true
  s3_use_path_style           = true

  endpoints {
    s3       = "http://localhost:4566"
```

```
    dynamodb = "http://localhost:4566"
  }
}
```

Whereas `required_providers` said *which* plugin to fetch, the `provider "aws"` block *configures* it: which region, which credentials, which API endpoint. Normally that's just `region` and credentials and it talks to real AWS. Here almost every line exists to redirect it at **LocalStack** instead. We'll unpack that in 9.6 - for now, note the shape: a block type (`provider`), a label (`"aws"`), and arguments inside.

`resource {}` blocks - the things you actually want to exist

A **resource** block declares one piece of infrastructure. This is the heart of Terraform. The module declares two.

```
resource "aws_s3_bucket" "model_artifacts" {
  bucket = "mlops-model-artifacts"
}
```

Anatomy of a resource block:

- `resource` - the block type.
- `"aws_s3_bucket"` - the **resource type**. The prefix (`aws_`) tells you the provider; the rest names the thing. An **S3 bucket** is Amazon's object storage - think of it as an infinitely large, durable folder in the cloud where you drop files ("objects"). Here it will hold the trained model artifacts.
- `"model_artifacts"` - the **local name**, a label *you* choose. It's how you refer to this resource elsewhere in the code (e.g. `aws_s3_bucket.model_artifacts.bucket`). It is not the real cloud name.
- `bucket = "mlops-model-artifacts"` - an **argument**. This one sets the actual globally-unique bucket name in AWS.

The second resource is a database table:

```
resource "aws_dynamodb_table" "predictions" {
  name         = "predictions"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "id"

  attribute {
    name = "id"
    type = "S"
  }
}
```

DynamoDB is AWS's fully-managed **NoSQL** database - a key-value / document store, not a table-of-rows-and-columns SQL database. It's designed for huge scale and single-digit-millisecond lookups by key. This table, named `predictions` , will record each prediction the spam model makes. We'll dissect `billing_mode` , `hash_key` , and `attribute` in 9.7.

> **Gotcha:** don't confuse the **resource type** (`aws_dynamodb_table` , fixed by the provider) with the **local name** (`predictions` , chosen by you) with the **real resource name** (`name = "predictions"` , the actual thing in AWS). Three different names that can look alike. In `main.tf` the local name and the AWS name happen to both be `predictions` , which is common but not required.

`output {}` blocks - reporting values back

```
output "bucket_name" {
  value = aws_s3_bucket.model_artifacts.bucket
```

```
}

output "table_name" {
  value = aws_dynamodb_table.predictions.name
}
```

An **output** exposes a value after `apply` finishes - printed to your terminal and readable by other tooling. `aws_s3_bucket.model_artifacts.bucket` is a **reference**: it reads the `bucket` attribute of the resource we labelled `model_artifacts`. Outputs are how you surface the names/IDs/URLs of things Terraform created so a human or a later step (a deploy script, another Terraform module) can use them without hard-coding.

> **Key insight:** references like `aws_dynamodb_table.predictions.name` do double duty. They fetch a value, *and* they tell Terraform "this thing depends on that thing." Terraform reads all references to build an internal **dependency graph** and creates resources in the correct order automatically - you never write "create the bucket before the output." This implicit dependency resolution is a core Terraform feature.

9.4 State: the file that makes Terraform work

Here is the concept that separates people who've *used* Terraform from people who've only read about it. When Terraform creates resources, it records what it made in a **state file** named `terraform.tfstate`. It's a JSON file mapping each resource in your config to the real-world object it created (the bucket's ID, the table's ARN, every attribute).

Why is state necessary? Because Terraform is declarative, it must answer "what already exists?" before it can compute "what to change." It has three sources of truth to reconcile:

1. **Your configuration** - the desired state (`.tf` files).
2. **The state file** - what Terraform believes it created last time.
3. **Reality** - what's actually in the cloud right now.

When you run a command, Terraform performs a **refresh**: it queries the cloud API for each resource in state to update its record of reality. Then it diffs desired-vs-reality to decide the actions.

> **Analogy:** state is Terraform's memory / receipt book. Without it, Terraform would have no idea which of the millions of buckets in your AWS account are the ones *it* is supposed to manage. The state file is the list of "things I own."

Drift is what happens when reality diverges from state behind Terraform's back - for example, someone manually deletes the DynamoDB table in the console, or changes a setting by hand. On the next `plan`, the refresh notices the mismatch and Terraform proposes to fix it (recreate the table, revert the setting) to restore your declared desired state. Detecting and correcting drift is a headline reason to use IaC at all.

> **Interview Q:** What is the Terraform state file and why does it matter?
> **A:** `terraform.tfstate` is a JSON file where Terraform records every resource it manages and maps it to the real cloud object plus its attributes. It matters because Terraform needs it to know what it already created, to compute an accurate plan by diffing desired config against known state and refreshed reality, and to detect drift. Losing state means Terraform forgets it owns your infrastructure. In a team you never keep it local - you use a remote backend like an S3 bucket with locking so state is shared and can't be corrupted by two people applying at once.

> **Gotcha:** the state file often contains **secrets** - database passwords, generated keys - in plaintext. Never commit `terraform.tfstate` to Git. Real teams store it in a **remote backend** (an S3 bucket, Terraform Cloud) with **state locking** so two engineers can't run `apply` simultaneously and corrupt it. This module runs locally so state lands in the working directory, which is fine for a demo but not for production.

9.5 The core loop: init -> plan -> apply -> destroy

You drive Terraform with four commands. This is the workflow the README walks through and the one every interview probes.

```
terraform init      # download providers, set up the working dir
terraform plan      # preview: what would change?
terraform apply     # make it real
terraform destroy   # tear it all down
```

Command	What it does	When
`init`	Reads `required_providers`, downloads the AWS provider plugin into `.terraform/`, writes the lock file, initializes the backend. Run once per new/changed module.	First, and after adding providers.
`plan`	Refreshes state, diffs desired vs. current, and prints an **execution plan**: `+` create, `~` update in place, `-/+` replace, `-` destroy. Changes *nothing*.	Before every apply; in CI on every PR.
`apply`	Shows the plan, waits for you to type `yes`, then makes the API calls to reach desired state and updates the state file.	To provision or change infra.
`destroy`	Deletes every resource Terraform manages (a plan in reverse).	To tear down; great for ephemeral test/demo envs.

The `-auto-approve` flag (used in the README) skips the interactive `yes` prompt - convenient in scripts and CI, and exactly what you'd disable for a human-gated production apply.

> **Interview Q:** Walk me through the Terraform workflow.
> **A:** `terraform init` first - it downloads the required provider plugins and sets up the backend and lock file. Then `terraform plan`, which refreshes state and shows a dry-run diff of exactly what will be created, changed, or destroyed, without touching anything. You review that plan - in a team it's attached to the pull request. Then `terraform apply` executes the plan and updates state. When you're done, `terraform destroy` removes everything it created. The golden rule is always plan before you apply.

> **Key insight:** `plan` is Terraform's superpower and the answer to "how is this safer than a script?" You get a precise, reviewable preview of every change *before* it happens. A bash script just runs.

> **Gotcha:** a plan showing `-/+ destroy and then create` (a **replacement**) is a red flag in production - it means the resource can't be changed in place and will be recreated, which for a database means **data loss**. Always read whether a change is in-place (`~`) or a replace (`-/+`).

9.6 Retargeting the provider: how one config hits LocalStack instead of AWS

Recall the `provider "aws"` block. Ordinarily Terraform's AWS provider talks to Amazon's real API endpoints and needs real credentials, which would create real (billable) resources. This project deliberately avoids that - per the project's own rule, no real cloud credentials are ever used. Instead it points at **LocalStack**, covered fully in Ch. 10: a program that runs on your laptop and *emulates* AWS's APIs on `http://localhost:4566`. To your Terraform code, LocalStack looks just like AWS.

Every non-`region` line in the provider block exists to pull off that redirection:

Argument	Why it's here
`access_key = "test"`, `secret_key = "test"`	Dummy credentials. The AWS provider *demands* credentials, but LocalStack accepts anything, so `"test"` / `"test"` satisfy the plugin without any real AWS account.
`skip_credentials_validation = true`	Don't phone real AWS STS to check the creds are valid - they aren't.
`skip_metadata_api_check = true`	Don't look for the EC2 instance-metadata service (we're not on EC2).
`skip_requesting_account_id = true`	Don't try to resolve a real AWS account number.
`s3_use_path_style = true`	Use `localhost:4566/bucket-name` URLs instead of AWS's default `bucket-name.s3.amazonaws.com` **virtual-host style**, which doesn't resolve for a local emulator.
`endpoints { s3 = "...:4566" dynamodb = "...:4566" }`	The actual redirect: send all S3 and DynamoDB API calls to LocalStack instead of AWS.

The `endpoints {}` block is the star. It overrides the URL Terraform uses per-service. Change those URLs and dummy creds to real ones, and this identical `main.tf` provisions against real AWS - nothing in the resource or output blocks changes.

> **Key insight:** the exact same mechanism you use here to hit LocalStack is what you use in real life to point one config at **different accounts or regions** - change `region`, swap in credentials via environment variables or an assumed IAM role, and the resource definitions stay untouched. The provider block is the "where"; the resource blocks are the "what," and they're cleanly separated.

> **Interview Q:** How would you make the same Terraform code run against LocalStack for testing and real AWS for production?
> **A:** Keep the resource definitions identical and vary only the provider configuration. For LocalStack you override the service `endpoints` to `localhost:4566`, supply dummy credentials, enable `s3_use_path_style`, and skip the credential/metadata checks. For real AWS you drop the endpoint overrides and skips and supply real credentials - typically via environment variables or an assumed role rather than hard-coding them - and set the real region. Because the "where" (provider) is separated from the "what" (resources), you get to test the exact same infrastructure code locally for free before it ever touches a billed account.

9.7 DynamoDB specifics: billing mode, hash key, attributes

The `predictions` table exercises three DynamoDB concepts worth understanding cold.

`billing_mode = "PAY_PER_REQUEST"`. DynamoDB has two capacity/pricing modes:

- `PAY_PER_REQUEST` (a.k.a. **on-demand**): you pay per read and per write request, and the table auto-scales to whatever traffic arrives. No capacity planning. Ideal for unpredictable or spiky workloads - like a demo, or a new app whose traffic you can't forecast.
- `PROVISIONED`: you pre-declare a fixed number of **read capacity units (RCUs)** and **write capacity units (WCUs)** per second. Cheaper at steady, predictable high volume, but you must guess right - under-provision and requests get **throttled** (rejected); over-provision and you pay for idle capacity.

> **Interview Q:** When would you choose on-demand vs. provisioned DynamoDB capacity?
> **A:** On-demand (`PAY_PER_REQUEST`) when traffic is spiky, unpredictable, or new - it auto-scales and you pay per request with zero capacity planning, so you never throttle from under-provisioning. Provisioned when traffic is steady and you can forecast it - you pre-buy read/write capacity units, which is significantly cheaper per request at sustained high volume, and you can add auto-scaling to widen the band. For this project the table uses on-demand because a demo's load is unpredictable and you want zero tuning.

`hash_key = "id"`. In DynamoDB the **hash key** - officially the **partition key** - is the primary key that identifies each item and determines which physical partition stores it. Here every prediction is keyed by its `id`. (DynamoDB can also have an optional **sort key**, the "range key," to allow multiple items under one partition key; this table has none.)

The `attribute {}` block.

```
attribute {
  name = "id"
  type = "S"
}
```

This is a **nested block** (a block inside a resource). A subtle-but-important DynamoDB rule: you only declare attributes that are part of a **key**. `id` is the hash key, so it must be declared, with `type = "S"` meaning **String** (`N` = number, `B` = binary). You do *not* declare the other fields of a prediction (the message text, the label, a timestamp) - DynamoDB is **schemaless** for non-key attributes, so items can carry any fields you like without Terraform or the table knowing about them in advance.

> **Gotcha:** a classic Terraform-DynamoDB mistake is declaring an `attribute` block for a column that isn't a key. Terraform will error. The rule is: define an `attribute` only for something used as a `hash_key` , `range_key` , or an index key - nothing else.

9.8 Putting it together

The whole `terraform-localstack` module is deliberately small - one provider, two resources, two outputs, ~50 lines - but it demonstrates the full IaC discipline the project cares about: declarative desired state written in HCL, version-pinned providers, a clean separation of "where" (the retargetable provider) from "what" (the resources), and the `init -> plan -> apply -> destroy` lifecycle. The resources it creates - an S3 bucket for model artifacts and a DynamoDB table for predictions - are exactly the two persistence pieces the spam-classifier system needs, and it creates them for free against LocalStack (Ch. 10) with no cloud bill and no real credentials.

Interview Q: This is a tiny Terraform file - what does it actually prove you can do?
A: It proves the fundamentals end to end: I can author HCL with pinned provider versions and version constraints, declare cloud resources (S3, DynamoDB) declaratively with correct DynamoDB key modeling and on-demand billing, expose outputs, and separate provider configuration from resource definitions so the same code targets a local emulator or real AWS by changing only endpoints and credentials. And I understand the runtime model - state, drift, refresh, and the plan-before-apply loop - which is what actually keeps Terraform safe in production.

10. Cloud & AWS Emulation: LocalStack, S3, DynamoDB, IAM

This project needs cloud infrastructure - a place to store the trained model file and a database to record predictions - but it deliberately uses **no real cloud account and spends no money**. It does this with LocalStack, a program that pretends to be Amazon Web Services on your own laptop. This chapter teaches cloud fundamentals from zero: what "the cloud" and AWS are, what S3 and DynamoDB do, how AWS logins (credentials and IAM) work, and how LocalStack lets you run genuine `terraform apply` (see Ch. 9 for Terraform itself) against a fake AWS that behaves like the real one.

10.1 What "the cloud" and AWS actually are

The cloud is just *someone else's computers that you rent over the internet, by the minute or by the request*. Instead of buying a physical server, plugging it in, and maintaining it, you ask a cloud provider for a server (or storage, or a database) and it appears in seconds. You pay only for what you use.

AWS (Amazon Web Services) is the largest cloud provider. It is not one product but a menu of ~200 separate **services**, each doing one job:

- **S3** - store files (objects).
- **DynamoDB** - a database.
- **EC2** - rent virtual servers.
- **IAM** - manage who is allowed to do what.
- ...and so on.

You talk to every AWS service the same way: your computer sends an **HTTPS API request** (a web request) to an AWS **endpoint** (a URL like `https://s3.us-east-1.amazonaws.com`), signed with your credentials. Tools like Terraform and the `aws` command-line tool are just convenient wrappers that build and send those web requests for you.

> **Key insight:** Every AWS action is ultimately an authenticated HTTP request to a service endpoint. Once you internalize that, LocalStack stops being magic - it is simply a server that answers those same requests on `localhost` instead of on Amazon's network.

10.2 What LocalStack is and why it is valuable

LocalStack is a program (shipped as a Docker image - see Ch. 5 for Docker) that **emulates the AWS APIs on your own machine**. "Emulate" means it listens for the exact same HTTP requests real AWS listens for, and it replies with the exact same shape of responses - but everything actually happens locally in a container, with no Amazon account involved.

Here is how this project starts it, in `terraform-localstack/docker-compose.yml`:

```
name: terraform-localstack
services:
  localstack:
    image: localstack/localstack:3
    ports:
      - "4566:4566"
    environment:
      SERVICES: s3,dynamodb
      DEBUG: "0"
```

Line by line:

- `image: localstack/localstack:3` - the official LocalStack image, major version 3.
- `ports: "4566:4566"` - LocalStack exposes **one** port, `4566`, which is the single front door for *all* emulated services. (Real AWS uses many hostnames; LocalStack multiplexes them behind this one port.)
- `SERVICES: s3,dynamodb` - only start the two services we need, so startup is fast and lightweight.
- `DEBUG: "0"` - quiet logging.

Why is this valuable? Four concrete reasons an interviewer wants to hear:

Benefit	Explanation
No cost	You run `terraform apply`, create buckets and tables, and destroy them - $0. Real AWS bills you.
No account / no risk	New engineers can practice IaC with zero onboarding and zero chance of touching production.
Fast feedback	A local container answers in milliseconds; no network round-trips to Amazon.
CI-friendly	Automated tests can spin LocalStack up, provision infra, assert it exists, and tear it down - fully offline and reproducible.

> **Interview Q:** What problem does LocalStack solve?
> **A:** It lets you develop and test AWS infrastructure code - Terraform, the AWS CLI, boto3 - against a local emulator of the AWS APIs. You get real end-to-end `init -> plan -> apply -> destroy` cycles with no cloud account, no credentials, no cost, and no blast radius, which makes it ideal for learning, local development, and CI pipelines.

> **Gotcha:** LocalStack emulates AWS *behavior*, not AWS *scale, latency, or every edge case*. It is excellent for testing your infra definitions and application logic, but it is not a substitute for a staging environment on real AWS before production. Some services and advanced features live only in LocalStack's paid Pro tier.

10.3 The endpoint override - how you point tools at LocalStack

The single trick that makes LocalStack work is the **endpoint override**. Normally an AWS tool computes the endpoint URL for itself - e.g. it sends S3 requests to `https://s3.us-east-1.amazonaws.com`. To use LocalStack, you tell the tool: *send those requests to* `http://localhost:4566` *instead.*

Terraform does this in `main.tf` with an `endpoints {}` block inside the provider:

```
provider "aws" {
  region                      = "us-east-1"
  access_key                  = "test"
  secret_key                  = "test"
  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true
  s3_use_path_style           = true

  endpoints {
    s3       = "http://localhost:4566"
```

```
    dynamodb = "http://localhost:4566"
  }
}
```

The `endpoints` block redirects each named service to LocalStack's port. Notice **both** S3 and DynamoDB point at the *same* URL - that is the one-port multiplexing from §10.2.

The command-line equivalent is the `--endpoint-url` flag, shown in the README's verification step:

```
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 dynamodb list-tables --region us-east-1
```

`aws` is the **AWS CLI**, Amazon's official command-line program for driving AWS. Without `--endpoint-url` it talks to real AWS; with it, it talks to LocalStack. `s3 ls` lists buckets; `dynamodb list-tables` lists tables. Because Terraform created these resources *inside* LocalStack, the CLI - pointed at the same endpoint - can see them. This proves the resources genuinely exist and were not faked by Terraform.

> **Key insight:** The endpoint override is not a LocalStack-only hack. The exact same `endpoints` / `--endpoint-url` mechanism is how you target a different AWS region, a VPC endpoint, a GovCloud partition, or an S3-compatible store like MinIO. LocalStack just reuses a first-class AWS feature. The README makes this point explicitly: it is "the same technique used to target different AWS accounts/regions."

10.4 The skip_* flags - telling Terraform not to phone home

Real AWS providers do several things at startup that assume they're talking to Amazon: they validate your credentials, they read instance metadata, and they call `sts:GetCallerIdentity` to discover your account ID. Against LocalStack those calls are pointless, so the provider disables them:

Flag	What it turns off	Why for LocalStack
`skip_credentials_validation`	Checking creds against AWS STS	The creds are fake; there's nothing to validate.
`skip_metadata_api_check`	Reading the EC2 instance-metadata endpoint	You're not on an EC2 instance.
`skip_requesting_account_id`	Looking up your 12-digit account ID	LocalStack has no real account.

Without these, the provider would hang or error trying to reach services LocalStack isn't emulating.

10.5 AWS S3 - object storage for the model artifacts

S3 (Simple Storage Service) is AWS's **object storage**: a service for storing files of any size and type, retrieved over HTTP. Think of it as an infinite hard drive you access by URL rather than by mounting a disk.

Core vocabulary:

- **Object** - one stored file (the bytes) plus metadata. In this project, the object is the trained spam-classifier `model.pkl` (see Ch. 4).

- **Bucket** - a named container for objects. Bucket names are **globally unique across all of AWS** (no two customers can both own `mlops-model-artifacts`) and live in a specific region. Analogy: a bucket is a top-level folder, but its name is claimed worldwide like a domain name.
- **Key** - the object's full name/path within the bucket, e.g. `models/v1/model.pkl`. Bucket + key uniquely identifies an object. S3 has no real folders; the "path" is just a key string with `/` in it.

This project creates a bucket in `main.tf`:

```
resource "aws_s3_bucket" "model_artifacts" {
  bucket = "mlops-model-artifacts"
}
```

Why S3 for model artifacts? A trained model is a binary file you need to store durably and pull down wherever you serve it. S3 is the canonical MLOps home for such artifacts: durable, versionable, cheap, and reachable from anywhere. The serving container (Ch. 4) or a Kubernetes pod (Ch. 8) could download `model.pkl` from this bucket at startup instead of baking it into the image.

> **Interview Q:** What is S3 and when would you use it in an ML system?
> **A:** S3 is AWS's object storage - durable, virtually unlimited file storage addressed by bucket and key over HTTP. In ML it's the standard place for model artifacts, training datasets, feature files, and batch prediction outputs. It's durable (11 nines), cheap, versioned, and integrates with nearly every AWS service, so it's the default artifact store in MLOps pipelines.

Path-style vs virtual-hosted addressing - why `s3_use_path_style`

S3 objects can be addressed two ways:

- **Virtual-hosted style (the modern default):** the bucket name is a **subdomain** -

`https://mlops-model-artifacts.s3.us-east-1.amazonaws.com/models/model.pkl`

- **Path-style (older):** the bucket name is the **first path segment** -

`https://s3.us-east-1.amazonaws.com/mlops-model-artifacts/models/model.pkl`

Virtual-hosted style requires DNS to resolve a unique subdomain *per bucket*. LocalStack runs on a single host, `localhost:4566`, and can't easily give every bucket its own working subdomain. So we force the older path-style, where the bucket is part of the URL path and the hostname stays constant:

```
s3_use_path_style = true
```

That one line is why S3 works against LocalStack.

> **Gotcha:** Forgetting `s3_use_path_style = true` is the classic "my Terraform hangs / DNS errors against LocalStack (or MinIO)" bug. Any single-host S3-compatible endpoint needs path-style addressing because per-bucket subdomains don't resolve there.

10.6 AWS DynamoDB - a managed NoSQL database for predictions

DynamoDB is AWS's **managed NoSQL key-value database**. Unpacking each word:

- **Managed** - AWS runs the servers, patching, scaling, and backups. You never touch a machine; you just create a table and read/write items. (Contrast with running your own PostgreSQL, where you own the box.)

- **NoSQL** - "not a traditional relational SQL database." There are no fixed columns you must declare up front and no `JOIN`s across tables. Each row can have different fields.
- **Key-value** - you fetch data primarily by a **key**, and get back a **value** (the record). It's optimized for "give me the item with this id," extremely fast, at any scale.

Vocabulary:

- **Table** - a collection of items (rough equivalent of a SQL table).
- **Item** - one record (rough equivalent of a row), stored as a set of attribute name/value pairs (like a JSON object).
- **Attribute** - one field of an item.
- **Partition key (a.k.a. hash key)** - the mandatory primary key that identifies each item and decides which physical partition (server) stores it. In Terraform's AWS provider this is called `hash_key`.

This project's table, from `main.tf`:

```
resource "aws_dynamodb_table" "predictions" {
  name         = "predictions"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "id"

  attribute {
    name = "id"
    type = "S"
  }
}
```

Reading it:

- `name = "predictions"` - the table stores prediction records made by the spam model.
- `hash_key = "id"` - each item is identified by an attribute called `id`.
- The `attribute` block declares `id` with type `"S"` (String). **You only declare attributes that are part of a key.** DynamoDB is schemaless for everything else, so you do *not* list `message`, `label`, `timestamp`, etc. here - items can carry any other fields freely. (Types: `S` = string, `N` = number, `B` = binary.)

> **Interview Q:** Why do you only define the `id` attribute in the Terraform, when a prediction record clearly has more fields?
> **A:** DynamoDB only requires you to declare attributes that participate in a key - the partition key here, plus any sort or index keys. Everything else is schemaless and attached per item at write time. So you declare `id` (the hash key) and nothing else; each prediction item can then carry message text, label, score, and timestamp without any schema change.

PAY*PER*REQUEST (on-demand) vs provisioned capacity

DynamoDB has two billing/capacity modes:

Mode	How you pay	Best for
`PAY_PER_REQUEST` (on-demand)	Per read and per write actually performed; capacity auto-scales instantly from zero.	Spiky, unpredictable, or low/dev traffic - like this project. Nothing to size; no cost when idle.
`PROVISIONED`	You pre-declare read/write capacity units per second and pay for that reserved throughput whether used or not.	Steady, predictable, high-volume workloads where reserving capacity is cheaper.

This project uses `PAY_PER_REQUEST` because it's a portfolio/demo with bursty, unpredictable usage - you never have to guess a throughput number, and an idle table costs nothing.

> **Interview Q:** When would you choose on-demand over provisioned DynamoDB capacity?
> **A:** On-demand (`PAY_PER_REQUEST`) when traffic is spiky, unpredictable, or low - dev, demos, new features - because it scales instantly and you pay only for actual requests with zero idle cost and no capacity planning. Provisioned when traffic is high and predictable, since reserving capacity (optionally with auto-scaling) is significantly cheaper per request at steady volume.

> **Key insight:** DynamoDB is a natural pairing with S3 in ML: bulky artifacts (the model, datasets) go in S3; small, fast-lookup structured records (each prediction, keyed by `id`) go in DynamoDB. Right tool per data shape.

10.7 AWS credentials & IAM - who are you, and what may you do

Every real AWS request must prove **who is making it** and be checked against **what they're allowed to do**. Two concepts:

Credentials answer *who are you?* The basic form is an **access key pair**:

- **Access Key ID** - a public-ish identifier, like a username (e.g. `AKIA...`).
- **Secret Access Key** - the secret half, like a password. It's used to cryptographically **sign** each request so AWS knows the request is genuine and untampered.

IAM (Identity and Access Management) answers *what are you allowed to do?* IAM is the AWS service where you define:

- **Users / roles** - identities.
- **Policies** - JSON documents granting or denying specific actions on specific resources (e.g. "allow `s3:GetObject` on `mlops-model-artifacts/*` ").

AWS is **deny-by-default**: an identity can do nothing until a policy explicitly allows it. Good practice is **least privilege** - grant only the exact actions a component needs.

Why dummy "test" credentials work against LocalStack

Look again at `main.tf` :

```
access_key = "test"
secret_key = "test"
```

These are obviously fake. They work because **LocalStack does not enforce IAM by default** - it accepts any credentials and doesn't check permissions. There's no real account, no billing identity, and nothing to protect, so authentication is a no-op. The provider still has to *supply* some credentials (the AWS SDK refuses to build a request with none), so we hand it the throwaway string `"test"` . This is exactly why `skip_credentials_validation = true` is set - there's no AWS STS to validate them against.

> **Interview Q:** Why can you use `access_key = "test"` with LocalStack but never with real AWS?
> **A:** LocalStack emulates the AWS APIs but doesn't enforce IAM by default, so it accepts any credentials - they're just placeholders the SDK needs to sign a request. Real AWS validates the signature against a real access key and checks IAM policies on every call, so fake creds are rejected and, worse, real creds in code are a serious security leak.

Why real credentials must come from the environment or roles - never in code

The single most important cloud-security rule for a candidate to state clearly:

> **Never hard-code real AWS credentials in source, config, or a Docker image.**

Committed secrets get pushed to GitHub, copied into logs, and baked into image layers, where anyone with the repo or image can extract them and run up charges or steal data. Instead, real credentials should come from **outside the code**:

- **Environment variables / a credentials file** - `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`, read by the SDK at runtime, never committed. (This is exactly how the Bedrock module in Ch. 11 supplies its credentials.)
- **IAM roles** (the best option in production) - when code runs on AWS (EC2, ECS, Lambda, EKS) or via OIDC federation, AWS injects **short-lived, auto-rotating temporary credentials** into the environment. There is *no long-lived secret to leak at all*.

The fake `"test"` values here are safe to commit *precisely because they are meaningless* - they authenticate against nothing real. That's a deliberate teaching pattern: the repo demonstrates the wiring without ever containing a real secret.

> **Key insight:** The provider block in this repo is safe to publish on GitHub *only because* the endpoint points at LocalStack and the creds are dummies. Swap in a real endpoint and this file would be exactly where you must *not* put real keys - you'd delete the `access_key` / `secret_key` lines and let the SDK pick creds up from the environment or an IAM role.

> **Gotcha:** Even a *deleted* hard-coded secret lives forever in git history and in cached Docker image layers. If a real key is ever committed, rotating (revoking and reissuing) it is mandatory - removing the line is not enough.

10.8 Regions and endpoints

An AWS **region** is a geographic cluster of data centers - e.g. `us-east-1` (N. Virginia), `eu-west-1` (Ireland). You choose a region for latency (close to users), data-residency laws, and cost. Resources mostly live in one region and each region has its own endpoints.

This project sets `region = "us-east-1"`, and the verification command repeats it: `dynamodb list-tables --region us-east-1`. Against LocalStack the region is largely cosmetic - everything is one local container - but the field is still required, and keeping it consistent avoids "resource not found" confusion (a bucket created in one region isn't visible when you query another).

An **endpoint** is the actual URL a request is sent to. Region + service normally determine the endpoint (`dynamodb.us-east-1.amazonaws.com`); the `endpoints{}` block (§10.3) overrides that to `http://localhost:4566`.

> **Interview Q:** What's the difference between a region and an endpoint?
> **A:** A region is the physical location - a group of data centers like `us-east-1` - where your resources live. An endpoint is the concrete URL your client sends API calls to, normally derived from the region and service (`dynamodb.us-east-1.amazonaws.com`). You can override the endpoint to retarget the same client at a different environment - a VPC endpoint, another partition, or a local emulator like LocalStack on `localhost:4566` - without changing anything else.

10.9 The full picture

Putting it together, the flow this module demonstrates is:

1. `docker compose up -d` starts LocalStack listening on `:4566`, emulating S3 and DynamoDB.
2. Terraform (Ch. 9), pointed by its `endpoints` block and dummy `"test"` creds at LocalStack, runs `init -> plan -> apply` and really creates a bucket and a table *inside* the emulator.
3. `aws --endpoint-url=http://localhost:4566 ...` independently confirms the bucket and table exist.
4. `terraform destroy` removes them; `docker compose down` stops LocalStack. Total cost: nothing.

You have exercised the exact same S3, DynamoDB, IAM-credential, region, and endpoint concepts you'd use on real AWS - the *only* differences are the endpoint URL and the fact that the credentials are dummies. That is the whole value proposition: **learn and test real cloud skills with zero account, zero risk, and zero cost.**

11. The GenAI Layer: AWS Bedrock, boto3, and LLMs

So far the project has done one thing five ways: take a text message and decide **spam or ham** (see Ch. 1 for the model, Ch. 2 for the serving API). That decision is a single word plus a confidence number. It is useful, but it is opaque - a human looking at the label `spam` has no idea *why* the machine thought so. The `bedrock-explainer/` module bolts on a final layer that answers the "why": it hands the message and its label to a **Large Language Model (LLM)** and asks the LLM to explain the decision in plain English. This chapter teaches, from zero, what an LLM is, what it means to call one as a *managed cloud service* (AWS Bedrock), and how the tiny `explain.py` script wires it all together.

11.1 What a Large Language Model is (from scratch)

A **Large Language Model** is a program that has been trained on an enormous amount of text (books, code, websites) to do one deceptively simple thing: **predict the next chunk of text given the text so far.** You give it some words (a *prompt*), and it produces words that plausibly continue them. That's it at the core. Everything impressive - answering questions, writing code, explaining a spam classification - emerges from doing that prediction extremely well.

A few terms you must be able to define out loud:

- **Token** - the unit an LLM reads and writes. Not quite a word: roughly 3-4 characters of English. "spam" might be one token; "Congratulations" might be two or three. Models are billed and limited by tokens, not words. Think of tokens as the syllables the model counts.
- **Foundation model** - a single large model trained once, at great expense, on broad general data, then reused for *many* different tasks without retraining. It is a "foundation" you build applications on top of. Anthropic's Claude, OpenAI's GPT, Meta's Llama are foundation models. Contrast with the project's spam classifier (Ch. 1), which is a *narrow* model trained for exactly one task.
- **Inference** - running a trained model to get an answer (as opposed to *training*, which builds the model). When we call Bedrock, we are doing inference.
- **Prompt** - the input text you send. **Completion / response** - the text the model sends back.

> **Key insight:** The spam classifier and the LLM are both "machine learning," but they sit at opposite ends of a spectrum. The classifier is small, cheap, deterministic-ish, and does exactly one thing (classify). The LLM is huge, general, and does open-ended language. This project deliberately uses each for what it's best at: the **classifier decides**, the **LLM explains**. Being able to articulate that division of labor is a strong interview moment.

11.2 Managed hosting vs self-hosting - and what AWS Bedrock is

To *use* an LLM you need the model's weights (gigabytes of numbers) running on hardware powerful enough to do inference - typically expensive GPUs. You have two broad options.

Self-hosting. You download an open-weight model (say Llama), rent or buy GPU servers, install the serving stack, keep it patched, scale it under load, and pay for the hardware whether it's busy or idle. You have full control and your data never leaves your machines, but you own all the operational pain. This is the LLM equivalent of running your own Kubernetes cluster (Ch. 10) instead of using a managed one.

Managed hosting (a hosted API). A cloud provider runs the model for you. You send an HTTP request with your prompt and get a response. You never see a GPU. You pay per token used. **AWS Bedrock** is exactly this: a single managed AWS service that gives you one consistent API to call many foundation models from different providers - Anthropic (Claude), Meta (Llama), Amazon (Titan/Nova), Mistral, Cohere, and others - without provisioning any infrastructure.

Here is the comparison an interviewer wants:

Concern	Self-hosting an LLM	AWS Bedrock (managed)
Infrastructure	You provision & run GPUs	None - AWS runs it
Scaling	Your problem	Handled by AWS
Cost model	Pay for hardware (idle or busy)	Pay per token (usage-based)
Model updates/patching	You do it	AWS does it
Data control	Stays on your machines	Sent to AWS (governed by their data terms)
Access control	You build it	AWS IAM + per-model "model access"
Time to first call	Days	Minutes

> **Interview Q:** What is AWS Bedrock and why would you use it instead of self-hosting?
> **A:** Bedrock is a fully managed AWS service that exposes many foundation models - Anthropic Claude, Llama, Amazon Titan, Mistral - behind one unified API. You call it with boto3 or HTTP, you never manage GPUs, and you pay per token. You'd choose it over self-hosting when you want to ship fast, don't want the operational burden of running GPU inference, have spiky or low volume where usage-based pricing wins, and want IAM-governed access to multiple models. You'd lean toward self-hosting when you need full data residency control, extreme cost efficiency at very high sustained volume, or a custom/fine-tuned model you must keep in-house.

> **Gotcha:** In Bedrock, having an AWS account is not enough to call a model. You must explicitly request **model access** for each foundation model (e.g. Claude 3 Haiku) in the Bedrock console, per region. First-time callers frequently get an `AccessDeniedException` not because of bad credentials but because model access was never granted. The module's README calls this out directly: "Model access granted for the chosen model ... in the AWS Bedrock console, in your region."

11.3 boto3 and the bedrock-runtime client

boto3 is the official **AWS SDK for Python** - SDK meaning "software development kit," a library that wraps AWS's raw HTTP APIs in normal Python function calls. Instead of hand-crafting signed HTTP requests to AWS, you write `boto3.client(...)` and call methods. (Ch. 5's Terraform provisions AWS resources declaratively; boto3 is the *imperative* counterpart - you call AWS from code, step by step.)

boto3 splits AWS into two kinds of client:

- A **control-plane** client manages the *existence and configuration* of resources. For Bedrock that's the `bedrock` client (list models, manage model access).

- A **data-plane** client does the *actual work*. For Bedrock that's the `bedrock-runtime` client - the one that runs inference. `explain.py` uses `bedrock-runtime`.

```
import boto3
client = boto3.client("bedrock-runtime", region_name=REGION)
```

The first argument names the service. `region_name` says which AWS region to talk to (more on regions in 11.7). Notice what's *not* here: no access key, no secret key, no password. That is deliberate and important - covered in 11.6.

11.4 invoke_model and the Anthropic messages shape

The workhorse is `client.invoke_model(...)`. You give it two things: **which model** to run, and a **body** describing the request. The body's exact shape depends on the model provider - Bedrock is a common front door, but each provider defines its own request/response format. Because this project uses an Anthropic Claude model, the body follows the **Anthropic messages** schema.

Here is the real code from `explain.py`:

```
def explain(text: str, label: str, confidence: float) -> str:
    client = boto3.client("bedrock-runtime", region_name=REGION)
    prompt = (
        f"A spam classifier labeled this message '{label}' with "
        f"{confidence:.0%} confidence:\n\n\"{text}\"\n\n"
        "In two sentences, explain to a non-technical user why it may have been "
        "classified this way."
    )
    body = {
        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": 200,
        "messages": [{"role": "user", "content": prompt}],
    }
    resp = client.invoke_model(modelId=MODEL_ID, body=json.dumps(body))
    payload = json.loads(resp["body"].read())
    return payload["content"][0]["text"]
```

Let's decode the **request body** field by field:

- `anthropic_version` - `"bedrock-2023-05-31"`. A version string that pins the Anthropic-on-Bedrock API contract, so your request stays interpreted the same way even as the platform evolves. It is not the model version; it's the *request format* version.
- `max_tokens` - `200`. The maximum number of tokens the model is allowed to generate in its reply. This is a hard cap and a cost/latency control: more tokens = slower and more expensive. 200 tokens is plenty for a two-sentence explanation. Required by the Anthropic schema.
- `messages` - a list of turn objects, each with a `role` (`"user"` or `"assistant"`) and `content`. This is the modern "chat" format: a conversation is an array of messages. Here there is exactly one user message - the prompt. (A multi-turn conversation would append the model's `assistant` replies and further `user` messages to this list.)

Two easy-to-miss mechanical details, both interview bait:

1. `body=json.dumps(body)` - `invoke_model` wants the body as a **JSON string (bytes)**, not a Python dict. `json.dumps` serializes the dict to a JSON-formatted string. Forget this and you get a serialization error.
2. `json.loads(resp["body"].read())` - the response's `body` is a **streaming object**, not a plain dict. You call `.read()` to pull the raw bytes, then `json.loads` to parse them back into a Python dict.

Now the **response shape**. An Anthropic messages response is a JSON object whose `content` is a *list of content blocks* (a model could return several blocks - text, tool-use, etc.). For a simple text reply, the generated sentence lives at:

```
payload["content"][0]["text"]
```

That is: the first content block's `text` field. That string - the plain-language explanation - is what `explain()` returns.

> **Interview Q:** Walk me through calling a Claude model on Bedrock in Python.
> **A:** Create a `bedrock-runtime` client with boto3 for your region. Build a request body in the Anthropic messages format: `anthropic_version`, a `max_tokens` cap, and a `messages` list of `{role, content}` turns. Call `invoke_model(modelId=..., body=json.dumps(body))` - the body must be a JSON string. The response's `body` is a stream, so you `.read()` it and `json.loads` it, then pull the text out of `content[0]["text"]`. Model id and region come from environment variables, and credentials come from the environment via boto3's default chain - never hard-coded.

> **Gotcha:** `invoke_model` is the classic per-provider API - each model family has its own body schema, so switching from Claude to, say, Llama means rewriting the body. Bedrock also offers a newer `converse` API that gives one *provider-agnostic* message format across models. This project uses `invoke_model` with the Anthropic schema, which is the most widely-seen pattern in interviews; knowing `converse` exists as the portable alternative is a bonus point.

11.5 Prompt design: how the explanation is steered

The quality of an LLM's answer is mostly decided by the **prompt** - the instructions you give it. This is **prompt engineering**: shaping the input so the output is what you want. Look at how `explain.py` constructs its prompt and notice every choice is doing a job:

```
A spam classifier labeled this message 'spam' with 68% confidence:

"Congratulations you won a free prize claim now send your bank details"

In two sentences, explain to a non-technical user why it may have been
classified this way.
```

- **Context first.** It tells the model the *label* and the *confidence* before anything else, so the LLM knows the verdict it must justify. The `{confidence:.0%}` format turns `0.68` into a friendly `68%`.
- **The exact input in quotes.** The message being explained is embedded verbatim, quoted, so the model reasons about the real text.
- **A length constraint.** "In two sentences" bounds the answer - pairs naturally with `max_tokens: 200`.
- **An audience constraint.** "explain to a non-technical user" sets the *register*: no jargon, plain language. This is the whole point of the feature.
- **Careful hedging.** "why it *may* have been classified this way" - because the LLM did **not** make the decision and cannot truly read the classifier's internals; it is producing a *plausible* explanation, not ground truth. The wording keeps the model honest.

> **Key insight:** The LLM here is a **post-hoc explainer**, not the decision-maker. It never sees the model's weights or the TF-IDF features (Ch. 1). It infers a likely reason from the text and the label. That's a genuine limitation to state plainly: these explanations are persuasive narratives, not a faithful trace of the classifier's math. For a *faithful* explanation you'd reach for techniques like SHAP or LIME on the classifier itself. Knowing that distinction shows real depth.

11.6 Credentials from the environment - never hard-coded

Nowhere in `explain.py` is there an AWS access key or secret. That is a security requirement, not an oversight. When you write `boto3.client("bedrock-runtime", ...)` with no credentials, boto3 walks its **default credential provider chain** - it looks, in order, at environment variables (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`), then a shared credentials file (`~/.aws/credentials`, populated by `aws configure` or an SSO login), then, on AWS infrastructure, an IAM role attached to the compute (EC2/ECS/EKS). The code stays credential-free; the *environment* supplies the secret at runtime.

Why this matters, stated the way an interviewer wants to hear it:

- **Hard-coded secrets leak.** Code gets committed to Git, shared, screenshotted, pushed to public GitHub. A key in source is a key in everyone's hands - and one that can spend real money or exfiltrate data.
- **Secrets must rotate.** Keys should be rotatable without editing and redeploying code. Externalizing them means rotation is a config change, not a code change.
- **Least privilege & environment separation.** The same code can run in dev, staging, and prod with *different* credentials and permissions, just by changing the environment. On real AWS you'd use an **IAM role** so there's no long-lived key at all - the machine gets short-lived, auto-rotating credentials.

The README states the rule twice, bluntly: "Never hard-code keys." and "Why credentials come from the environment, never the code."

> **Interview Q:** How should an app authenticate to AWS, and why never put keys in code?
> **A:** Let the SDK's default credential chain resolve them from the environment: env vars locally, a shared profile or SSO for developers, and - best of all in production - an IAM role attached to the compute so there are no long-lived keys at all, just short-lived auto-rotating credentials. Never hard-code, because source code gets committed, shared, and leaked; because keys must be rotatable without a redeploy; and because externalizing credentials lets one codebase run in dev/staging/prod under least-privilege permissions.

> **Gotcha:** This module deliberately does **not** run against a fake/emulated backend. Unlike Ch. 5's Terraform against LocalStack, there is no "Bedrock emulator." The README is explicit: it "requires real AWS Bedrock access to run" and "goes on the résumé only once you have actually run it against Bedrock ... no faking." Honesty about what has and hasn't been verified is itself a professional signal.

11.7 Regions and model IDs - the two things you must get right

Two configuration values decide *what* you call and *where*, and `explain.py` reads both from the environment with sensible defaults:

```
REGION = os.environ.get("AWS_REGION", "us-east-1")
MODEL_ID = os.environ.get("BEDROCK_MODEL_ID", "anthropic.claude-3-haiku-20240307-v1:0")
```

Region. AWS runs in many geographic **regions** (`us-east-1` is Northern Virginia, `eu-west-1` is Ireland, etc.), each a cluster of data centers. Bedrock and specific models are only available in *some* regions, and model access is granted **per region**. Region also affects latency (call one near your users) and data residency (which country your data is processed in). Default here: `us-east-1` , the most feature-complete region.

Model ID. The exact identifier of the foundation model to invoke. `anthropic.claude-3-haiku-20240307-v1:0` decodes as: provider `anthropic` , family `claude-3` , tier `haiku` , a training-snapshot date, and a version suffix `v1:0` . **Haiku** is Anthropic's fastest, cheapest Claude 3 tier - a smart choice for short, high-volume explanations where you don't need the heavier reasoning of Sonnet or Opus. Because both values are environment variables, you can point the same code at a different region or upgrade to a different model **without touching the source** - the same externalize-your-config discipline as the credentials.

> **Interview Q:** Why parameterize region and model id via environment variables instead of hard-coding them?
> **A:** Because they're deployment concerns, not logic. Different environments and users hit different regions, model availability is per-region, and you want to swap or upgrade models - Haiku to Sonnet, or a newer snapshot - without a code change or redeploy. Externalizing them (with safe defaults) keeps the code portable and turns "which model, which region" into configuration you can vary per environment. Choosing Haiku specifically reflects picking the cheapest, fastest tier that's good enough for a two-sentence explanation.

11.8 Composing classical ML with GenAI - Explainable AI

Step back and see the whole pipeline the portfolio now forms:

1. A message arrives.
2. The **TF-IDF + Logistic Regression classifier** (Ch. 1) labels it `spam` / `ham` with a confidence - fast, cheap, narrow, and the actual *decision*.
3. `explain.py` passes the message, label, and confidence to a **Bedrock-hosted LLM**, which returns a plain-language *justification*.

This two-stage design is a small instance of **Explainable AI (XAI)** - the practice of making a model's outputs understandable to humans. The classifier alone gives an answer; the GenAI layer gives an answer *a person can act on and trust*. Real-world value: a user told "this is spam, because it promises a free prize and asks for your bank details - classic phishing" is far better served than one who just sees the word `spam` .

> **Key insight:** The pattern "small specialized model does the task, LLM makes it human-friendly" is one of the most common and defensible ways to add GenAI to an existing system. You don't replace your working classifier with an expensive LLM (which would be slower, costlier, and less reliable at pure classification). You keep the right tool for the decision and add the LLM only where language is the value. That architectural restraint is exactly what senior interviewers look for.

Interview Q: Where does an LLM add value in this project, and where would using it be the wrong tool?
A: The LLM adds value in *explanation* - turning a terse label and score into trustworthy plain language for a non-technical user, which is genuine Explainable-AI value. It would be the wrong tool for the *classification itself*: the TF-IDF + logistic-regression model is faster, cheaper, deterministic, and easily retrained on labeled spam, whereas an LLM would be slower, pricier per call, and harder to evaluate for a simple binary decision. The design keeps classical ML for the verdict and reserves GenAI for the language - the right tool for each job.

Gotcha: These LLM explanations are generated, so they can be **plausible-but-wrong** (a "hallucination") - the model may invent a reason the classifier didn't actually use, because it never sees the classifier's features. In anything user-facing you'd hedge the wording (the prompt already says "may have been"), keep the explanation advisory, and for true fidelity use a feature-attribution method (SHAP/LIME) on the classifier rather than a narrative from a separate model. Naming that limitation unprompted marks you as someone who ships GenAI responsibly.

12. Putting It All Together: Architecture, Flow, and Running It

By now you have met each tool on its own (Chapters 3-11). This chapter zooms all the way out and looks at the whole system as **one picture**: how a message travels through it, why the pieces are split the way they are, how to run each module, what is genuinely production-grade versus a local demo, and how you would harden it for real production. This is the chapter that turns a pile of tools into a *system-design story* you can tell in an interview.

Throughout, keep one sentence in your head: **the machine-learning model is one small part of the system.** The spam classifier itself (Ch. 2) is maybe 30 lines of scikit-learn. Everything else in this repo - serving, streaming, infrastructure, orchestration, explanation - is the *engineering around* that model. That ratio is the whole point of MLOps. "MLOps" (Machine Learning Operations) is the discipline of running ML models reliably in production, and as you will see, it is mostly ordinary software/DevOps engineering applied to a model.

> **Key insight:** In real ML products, the model is a few percent of the code and effort. The other 95% - serving, data pipelines, infrastructure, deployment, monitoring - is what keeps it alive. A candidate who understands this is far ahead of one who only knows how to `model.fit()`.

12.1 The one-model, five-ways idea

There is exactly **one** trained artifact in this entire repo: a TF-IDF + Logistic Regression spam classifier (a `model.pkl` file, built by `serving/train.py`). Every module operates that same model from a different angle:

Module	Angle	The question it answers
`serving/`	The model as a REST API	"How do I let *anything* call the model over HTTP?"
`kafka-streaming/`	The model in a real-time pipeline	"How do I classify a never-ending *stream* of messages?"
`terraform-localstack/`	Infrastructure for the model	"How do I create the cloud resources it needs, reproducibly?"
`kubernetes/`	The model, orchestrated	"How do I run *many copies* that heal and scale themselves?"
`bedrock-explainer/`	The model, explained	"How do I explain *why* a message was flagged, in plain English?"

Notice that four of the five never re-implement inference - they all funnel back to the **serving API**. That single shared surface (`POST /predict`) is the seam that holds the system together. Learn to point that out; it demonstrates you understand interfaces and reuse.

12.2 The end-to-end architecture

Here is the whole system in one diagram, drawn from the repo's own `README.md` and the module code:

```
                producer.py
                    |  publishes {id, text}
```

```
            v
      +----------------------+
      |  Kafka topic:        |   real-time buffer
      |     "messages"       |   (KRaft mode, no ZooKeeper)
      +----------+-----------+
                 |  consumer.py reads each record
                 v
      +----------------------+
      |  serving API         |   FastAPI + Uvicorn, in Docker
      |  POST /predict       |<-- SAME image deployed by Kubernetes
      |  -> {label, confidence} |       (Deployment: 2 replicas + Service)
      +----------+-----------+
                 |  consumer.py publishes the result
                 v
      +----------------------+
      |  Kafka topic:        |
      |    "predictions"     |
      +----------------------+

Terraform --provisions--> S3 bucket (model artifacts) + DynamoDB (predictions)
                          against LocalStack (local AWS emulator)

Bedrock explainer --after a label--> Claude 3 Haiku --> plain-English "why"
```

Two distinct kinds of arrows are worth naming explicitly, because interviewers probe this:

- **Data flow** - the actual messages and predictions moving: producer -> `messages` -> consumer -> serving -> `predictions`.
- **Control / deployment flow** - how the pieces come to *exist*: Terraform stands up infrastructure; Kubernetes deploys the serving image; Docker builds every container. These do not carry request data; they set the stage.

> **Interview Q:** Walk me through what happens to a single message end-to-end.
> **A:** The producer serializes `{id, text}` to JSON and sends it to the Kafka `messages` topic. The consumer, part of a consumer group, reads that record, extracts the text, and does an HTTP `POST /predict` to the serving API. The FastAPI app runs TF-IDF vectorization plus logistic regression and returns `{label, confidence}`. The consumer wraps that with the original id and text and publishes it to the `predictions` topic, where any downstream system can consume it. Kafka decouples the fast producer from the slower model, and the model runs as its own service so it can scale independently of the pipeline.

12.3 The "same image everywhere" principle

This is the single most important architectural fact in the repo, and it is easy to miss. The serving module builds a Docker image. That *exact* image is what:

- `kafka-streaming/docker-compose.yml` runs as the `spam-api` service - note `build: ../serving`, pointing straight at the serving module.
- `kubernetes/deployment.yaml` runs across 2 replicas - `image: mlops-serving:latest`.
- `serving/README.md` runs standalone with `docker run -p 8000:8000`.

A **Docker image** is a frozen, self-contained snapshot of an application plus everything it needs to run (Python, libraries, the trained `model.pkl`). Because the image is identical in all three contexts, the model behaves identically whether you run it by hand, stream through it, or orchestrate it. This is what people mean by **reproducibility**: "it works on my machine" stops being a risk because *the machine is the image*.

> **Interview Q:** How do you guarantee the model behaves the same in dev and prod?
> **A:** Ship the trained model *inside* an immutable Docker image and run that same image everywhere - locally, in the streaming compose stack, and in Kubernetes. There is no separate "install dependencies on the server" step that could drift. The image is the unit of deployment, so dev/prod parity is structural, not a matter of discipline.

12.4 The recurring design principles

The same five or six ideas show up in every module. Recognizing them as *patterns* - not module-specific tricks - is what separates a strong candidate from someone who memorized commands.

1. Service decoupling

Independent pieces talk over well-defined interfaces (HTTP, Kafka topics) instead of being fused into one program. The consumer does **not** load the model; it calls the serving API (`consumer.py` even documents this: "the streaming layer and the model service scale independently"). If predictions get slow, you scale the model service without touching the pipeline, and vice versa.

> **Interview Q:** Why put inference behind an HTTP call instead of importing the model into the consumer?
> **A:** Decoupling. The consumer and the model then have separate failure domains, deploy cycles, and scaling knobs. You can run 2 replicas of the model behind a load balancer while keeping one consumer, upgrade the model without redeploying the pipeline, and reuse the same API from Kubernetes, curl, or anything else. The cost is network latency and one more moving part - usually a good trade.

2. Configuration via environment variables

Nothing that changes between environments is hard-coded. The consumer reads `KAFKA_BROKER`, `MODEL_API`, `MESSAGES_TOPIC`, `PREDICTIONS_TOPIC` from `os.environ` with sensible defaults; compose injects `KAFKA_BROKER: kafka:9092` and `MODEL_API: http://spam-api:8000/predict`; Bedrock reads `AWS_REGION` and `BEDROCK_MODEL_ID` from the environment and **never** hard-codes credentials. This is straight out of the "Twelve-Factor App" playbook - a widely-cited set of rules for building portable services - whose third factor is "store config in the environment."

> **Gotcha:** Notice the broker is advertised as `kafka:9092`, not `localhost:9092`. Inside the Docker network, containers reach each other by *service name*. Hard-coding `localhost` there is the single most common Kafka-in-Docker mistake - a container's `localhost` is itself, not the broker.

3. Health checks and probes everywhere

The serving app exposes `GET /health`. The serving Dockerfile has a `HEALTHCHECK`. The Kubernetes deployment defines **both** a readiness probe and a liveness probe hitting `/health` (Ch. 8). Every layer has a way to ask "are you alive and ready?" A **readiness probe** decides whether to send traffic to a pod; a **liveness probe** decides whether to restart it.

> **Interview Q:** Why does the readiness probe use `initialDelaySeconds: 60` and liveness `90` ?
> **A:** scikit-learn's first import and the model's lazy first load are slow (~40s). Without a generous initial delay, the liveness probe would declare the pod dead and restart it before it ever finished booting - a crash loop. Readiness at 60s keeps traffic away until the model is warm; liveness at 90s gives even more slack so a slow-but-healthy start is never mistaken for a hang. The deployment file even comments this.

4. Retries and backoff for startup ordering

Distributed systems start in an unpredictable order. Kafka takes longer to boot than the Python clients, and the model API's first request is slow. So `producer.py` and `consumer.py` retry: they loop on `NoBrokersAvailable`, sleeping 3s, until Kafka answers; the consumer's `predict()` retries the model API up to 40 times. This is **backoff** - wait and try again rather than crash on the first failure.

> **Key insight:** `depends_on` in compose only waits for a container to *start*, not to be *ready*. That is exactly why the code retries in application logic. Saying this in an interview shows you know the difference between "process launched" and "service ready."

5. Reproducibility via containers

Every runnable module is a container or a stack of them. No "install Java 17 and Kafka, then Python 3.11 and these pip packages" instructions that rot. `docker compose up --build` and `kubectl apply` reproduce the entire environment from declarative files.

6. The model is one small part

Restated because it is a *design principle*, not just an observation: the architecture is dominated by serving, buffering, orchestration, and infrastructure. The model is a dependency that gets *operated*, like a database.

12.5 Running each module (command recap)

A concise, correct recap. Each module also has its own README with the full walkthrough.

serving/ - the model as an API:

```
docker build -t mlops-serving ./serving
docker run -d -p 8000:8000 --name spam-api mlops-serving
curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "Congratulations you won a free prize claim now"}'
docker rm -f spam-api
```

kafka-streaming/ - the real-time pipeline (brings up kafka, spam-api, producer, consumer):

```
cd kafka-streaming
docker compose up --build
docker compose logs -f consumer   # watch predictions flow (first ones take ~1-2 min)
docker compose down
```

terraform-localstack/ - infrastructure as code against a local AWS emulator:

```
cd terraform-localstack
docker compose up -d                # start LocalStack on :4566
```

```
terraform init
terraform plan
terraform apply -auto-approve        # creates S3 bucket + DynamoDB table
aws --endpoint-url=http://localhost:4566 s3 ls
terraform destroy -auto-approve
docker compose down
```

kubernetes/ - orchestration (needs a local cluster, e.g. Docker Desktop's Kubernetes):

```
docker build -t mlops-serving:latest ./serving   # build the image the cluster runs
kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/service.yaml
kubectl get pods -l app=spam-api -w              # wait for Ready (~1-2 min)
kubectl port-forward service/spam-api 8000:8000
kubectl delete -f kubernetes/service.yaml -f kubernetes/deployment.yaml
```

bedrock-explainer/ - LLM explanations (requires real AWS Bedrock access):

```
cd bedrock-explainer
pip install -r requirements.txt
export AWS_REGION=us-east-1
export BEDROCK_MODEL_ID=anthropic.claude-3-haiku-20240307-v1:0
python explain.py
```

> **Gotcha:** For `kind` clusters the locally-built image is invisible until you run `kind load docker-image mlops-serving:latest`. The deployment sets `imagePullPolicy: IfNotPresent` precisely so Kubernetes uses your local image instead of trying (and failing) to pull `mlops-serving:latest` from a registry that has no such image.

12.6 What is real vs. what is demo

Being honest about this is a *strength* in interviews - it proves you know what production actually demands. The repo's own READMEs are careful about it, and so should you be.

Aspect	In this repo (demo)	Real production
Kafka	Single broker, `replication.factor=1`, `auto.create.topics=true`	3+ brokers, replication >=3, explicit topic creation, auth (SASL/TLS)
Cloud infra	LocalStack (a local emulator of AWS on `:4566`)	Real AWS: real S3, real DynamoDB, real IAM
Credentials	Dummy creds for LocalStack; Bedrock uses real env creds	Real IAM roles, least privilege, short-lived tokens
Kubernetes	Local single-node cluster, `NodePort`, `port-forward`	Managed cluster (EKS/GKE), Ingress + real load balancer + TLS
Serving	2 replicas, no autoscaling, no persistence	Horizontal Pod Autoscaler, request logging, a model registry
Bedrock	Only runs with real AWS Bedrock access	Same, plus cost controls, rate limits, prompt/response logging

The Kafka config makes the demo/real gap concrete: `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1` and friends are set to `1` because there is only one broker. **Replication factor** is how many copies of each partition Kafka keeps; `1` means *zero redundancy* - lose the broker, lose the data. Fine for a laptop, unacceptable for production, where you would run at least three brokers and a replication factor of three.

> **Interview Q:** Is this production-ready? Where would it break?
> **A:** No - and it's designed to teach the patterns, not to be prod. The Kafka cluster is a single broker with replication factor 1, so it has no fault tolerance. Infra runs against LocalStack, an emulator, not real AWS. The serving tier has fixed replicas and no autoscaling, and there's no monitoring, no model registry, no CI/CD, and no auth on the API. The *shapes* are right - decoupled services, health probes, IaC, containers - so hardening it is about depth, not redesign.

12.7 How you would harden it for production (the system-design bridge)

This is where the whole guide pays off: you can turn each demo into a production design. A crisp answer per layer:

- **Serving:** Put the API behind a real load balancer with TLS. Add a **Horizontal Pod Autoscaler** (scales replica count by CPU or request rate). Emit structured request/prediction logs and metrics (Prometheus/Grafana). Store the model in a **model registry** (e.g. MLflow) and load a versioned artifact at boot instead of baking one specific `model.pkl` into the image - or keep baking it but tag the image with the model version. Add authentication (an API key or JWT) so not just anyone can call `/predict`.
- **Streaming (Kafka):** 3+ brokers, replication factor 3, `min.insync.replicas=2`. Disable `auto.create.topics`; create topics deliberately with a chosen partition count (partitions are the unit of parallelism). Add a **schema registry** so producers and consumers agree on message shape. Add a **dead-letter topic** for messages that fail prediction, and enable SASL/TLS for auth and encryption.
- **Infrastructure (Terraform):** Swap LocalStack for real AWS by changing the provider's `endpoints`/credentials. Move Terraform **state** to a remote backend (an S3 bucket with a DynamoDB lock table) so a team can collaborate without corrupting state. Run `plan` in CI and require review before `apply`. Use least-privilege IAM roles, not root creds.
- **Kubernetes:** Use a managed cluster (EKS/GKE). Replace `NodePort` + `port-forward` with an **Ingress** and a cloud load balancer. Add resource-based autoscaling, `PodDisruptionBudgets`, and rolling-update strategy tuning. Pull images from a private registry by digest, not the mutable `:latest` tag.
- **Explainability (Bedrock):** Add caching (identical messages get identical explanations), rate limiting and cost caps, and logging of prompts/responses for audit. Keep credentials in the environment / IAM roles, never in code.
- **Cross-cutting:** A CI/CD pipeline that builds the image, runs tests, scans for vulnerabilities, and deploys. Monitoring and alerting on latency, error rate, and - crucially for ML - **data drift** (the incoming messages slowly stop resembling the training data, so accuracy silently decays).

> **Interview Q:** You have this running locally. What are the first three things you do to make it production-ready?
> **A:** First, remove single points of failure: a real multi-broker Kafka cluster with replication, and real AWS behind Terraform with remote, locked state. Second, observability: structured logs, latency/error metrics, and alerting - you cannot operate what you cannot see, and for ML you specifically watch for data drift. Third, a CI/CD pipeline so every change is built, tested, scanned, and deployed the same way, with the image as the immutable unit. After that: autoscaling, auth on the API, and a model registry for versioned rollouts and rollbacks.

12.8 Tying the modules together

A final set of connective questions - the kind that separate "I did a tutorial" from "I understand systems."

> **Interview Q:** How do the five modules relate - is this five projects or one?
> **A:** One system, one model, five operational concerns. There's a single trained classifier exposed as one HTTP interface, `POST /predict`. Streaming, Kubernetes, and standalone Docker are three ways to *run that same image*; Terraform provisions the infrastructure it would live on; Bedrock adds explanation on top of its output. The serving API is the hub every other module connects through.

> **Interview Q:** Where would a prediction be stored, and how does that tie the modules together?
> **A:** Terraform provisions a DynamoDB `predictions` table (and an S3 bucket for model artifacts). In a full build, the consumer - or the serving API - would write each `{id, text, label, confidence}` record into that table instead of only publishing to the `predictions` Kafka topic. That closes the loop: the streaming layer produces predictions, the infrastructure layer stores them, and downstream analytics or the Bedrock explainer can read them back.

> **Interview Q:** If the serving API goes down, what happens to the pipeline?
> **A:** The consumer's `predict()` retries the API up to 40 times with a 3-second backoff, so a brief outage just delays predictions rather than dropping messages - the messages sit safely in the Kafka `messages` topic (Kafka persists them; the consumer's committed offset doesn't advance past an unprocessed message). If the API stays down past the retry budget, the consumer raises and stops, and unprocessed messages remain in Kafka to be handled once it recovers. In production you'd add a dead-letter topic so poison messages don't block the stream indefinitely.

> **Key insight:** The strongest thing you can say about this project is that it treats the model like any other production dependency - versioned, containerized, health-checked, decoupled, and provisioned with infrastructure-as-code. That mindset, more than any single tool, is what "MLOps engineer" means.

You now have the whole picture: one model, operated five ways, held together by a shared API, built on a handful of design principles that repeat at every layer - and a clear, honest story about what's real, what's demo, and how you'd take it to production. That is exactly the arc an interviewer wants to hear.

13. Interview Prep I - Tool Question Bank (Zero to Advanced)

This chapter is a drill deck. Every tool this project touches gets its own bank of interview questions, ordered from "explain it to a five-year-old" (ELI5) through core competency to senior deep-dives. Each question has a **model answer** written the way you should actually say it out loud in a room: confident, correct, and concise. Read the answer, then close the page and try to reproduce it in your own words. If you can do that for every question here, you can hold your own on any question about this stack.

A quick note on how to *use* an answer in an interview: state the definition first, then the "why it exists" (the problem it solves), then one concrete detail from real experience. Interviewers reward the third part because it proves you have actually used the tool, not just read about it. Where possible the answers below anchor to this repo (the spam-classifier served five ways) so you have that concrete detail ready.

13.1 Docker

> **Interview Q (ELI5):** What is Docker?
> **A:** Docker packages an application together with everything it needs to run - the code, the Python interpreter, the libraries, the OS-level files - into a single sealed box called a **container**. Because the box carries its own environment, it runs the same way on my laptop, a teammate's laptop, and a production server. It kills the "works on my machine" problem.

Before going further, three terms you must never confuse:

Term	Plain meaning	Analogy
Image	A read-only template/blueprint. Built once, stored, shipped.	A class in code, or a cake recipe
Container	A running instance of an image.	An object, or an actual baked cake
Dockerfile	A text script of steps to build an image.	The recipe's written instructions

> **Interview Q (core):** What's the difference between a container and a virtual machine?
> **A:** A virtual machine virtualizes hardware and runs a full guest operating system with its own kernel, so it's heavy - gigabytes and tens of seconds to boot. A container virtualizes at the OS level: all containers on a host share the host's Linux kernel and isolate only the process, filesystem, and network using kernel features (namespaces and cgroups). So a container is megabytes, starts in milliseconds, but is less isolated than a VM.

> **Interview Q (core):** Walk me through a Dockerfile you've written.
> **A:** In this project the serving image starts `FROM python:3.11-slim`, sets a working directory, copies `requirements.txt` *first* and runs `pip install`, then copies the source, trains the model at build time, exposes port 8000, adds a HEALTHCHECK, and ends with `CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]`. The key move is copying requirements before source code - that's for layer caching.

> **Key insight - layer caching:** Each Dockerfile instruction creates a **layer** (a cached filesystem diff). Docker reuses a layer if that instruction and everything above it are unchanged. Dependencies change rarely; source code changes constantly. By copying `requirements.txt` and running `pip install` *before* copying the app, you only re-run the slow `pip install` when dependencies actually change. Reverse the order and every one-line code edit reinstalls every package.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .          # deps first...
RUN pip install --no-cache-dir -r requirements.txt
COPY . .                         # ...source last
EXPOSE 8000
HEALTHCHECK CMD curl -f http://localhost:8000/health || exit 1
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

> **Interview Q (core):** `CMD` vs `ENTRYPOINT` ? `COPY` vs `ADD` ? `EXPOSE` vs `-p` ?
> **A:** `CMD` sets the default command, easily overridden at `docker run .` `ENTRYPOINT` sets the fixed executable and treats `CMD` /run args as its arguments - use it when the container *is* one tool. `COPY` just copies local files; `ADD` also unpacks tarballs and can fetch URLs, so best practice is `COPY` unless you specifically need `ADD` . `EXPOSE` is documentation of which port the app listens on; it does **not** publish anything - you actually open the port to the host with `-p 8000:8000` at run time.

> **Interview Q (senior):** What is a multi-stage build and why use one?
> **A:** A multi-stage build uses multiple `FROM` statements. You compile or build in a heavy "builder" stage, then copy only the finished artifacts into a slim final stage, discarding compilers and dev dependencies. The result is a smaller, more secure image (less installed = smaller attack surface). For a Python app you might build wheels in stage one and install just those in stage two.

> **Interview Q (senior):** How do you keep a Docker image small and secure?
> **A:** Start from a slim or distroless base; use multi-stage builds; combine `RUN` steps and clean package caches in the same layer (`--no-cache-dir` for pip, `rm -rf /var/lib/apt/lists/*` for apt); add a `.dockerignore` so build context stays lean; pin base image versions rather than `latest` ; run as a non-root `USER` ; and scan the image (Trivy, `docker scan`) for CVEs.

> **Gotcha:** Data written inside a container is ephemeral - it dies with the container. To persist data you mount a **volume** (`-v`). And each `RUN rm` in a *later* layer doesn't shrink the image, because the file still exists in the earlier layer; you must delete within the same `RUN` that created it.

> **Interview Q (senior):** What are namespaces and cgroups?
> **A:** They're the Linux kernel features that make containers possible. **Namespaces** isolate what a process can *see* - its own PID tree, network interfaces, mounts, hostname. **cgroups** (control groups) limit what a process can *use* - CPU, memory, I/O. Docker is essentially a friendly UX over these primitives plus an image format and registry.

13.2 Docker Compose & Container Networking

Interview Q (ELI5): What is Docker Compose?
A: Compose lets you define and run a group of containers together from one YAML file. Instead of typing many `docker run` commands, you write `docker-compose.yml` describing each service and run `docker compose up`. This project uses it to bring up Kafka plus the API, producer, and consumer as one stack.

Interview Q (core): How do containers talk to each other in Compose?
A: Compose puts all services on a shared user-defined bridge network and gives each an internal DNS name equal to its **service name**. So the consumer reaches the API at `http://spam-api:8000` and both reach the broker at `kafka:9092` - no IP addresses needed. Docker's embedded DNS resolves the service name to the container's current IP.

Key insight - localhost trap: Inside a container, `localhost` means *that container*, not the host and not another container. A classic bug is a consumer trying `http://localhost:8000` to reach the API in a different container. The fix is to use the service name (`http://spam-api:8000`). This is one of the most common real-world debugging moments in a Compose/Kafka setup like this one.

Interview Q (core): What Docker network drivers exist?
A: `bridge` is the default for standalone containers on one host (private internal network, NAT to outside). `host` removes network isolation - the container shares the host's network stack directly (fast, no port mapping, less isolated). `none` disables networking. `overlay` spans multiple hosts for Swarm/multi-host clusters.

Interview Q (senior): How do you express startup order and readiness in Compose?
A: `depends_on` controls *start* order but not *readiness* - it doesn't wait for a service to be healthy. To wait for readiness you add a `healthcheck` to the dependency and use `depends_on:` with `condition: service_healthy`, or the consumer retries its connection with backoff. In this project the consumer must tolerate Kafka and the API not being ready the instant it starts, so retry logic matters.

```
services:
  kafka:
    image: apache/kafka:3.9.0
  spam-api:
    build: ../serving
  consumer:
    build: .
    depends_on: [kafka, spam-api]   # start order only, not readiness
```

13.3 FastAPI / Uvicorn / Pydantic & REST

Interview Q (ELI5): What is a REST API?
A: A REST API is a way for programs to talk over HTTP using URLs and standard verbs. You send a request to an endpoint (a URL) with a method - GET to read, POST to send data - and get back a response, usually JSON. The spam service exposes `GET /health` to check it's alive and `POST /predict` to classify a message.

Interview Q (core): What is FastAPI and why choose it for model serving?
A: FastAPI is a modern Python web framework for building APIs. It's fast (built on the ASGI async standard), gives you automatic request validation and interactive Swagger docs from your type hints, and is minimal to write. For ML serving it's ideal: you declare a Pydantic request model, FastAPI validates incoming JSON, and you return predictions - all with auto-generated `/docs`.

Interview Q (core): What are Uvicorn and Pydantic, and how do they relate to FastAPI?
A: They're two different layers. **Uvicorn** is the ASGI *server* - the process that actually listens on the socket, accepts HTTP connections, and calls your app. FastAPI is the *framework* you write handlers in; Uvicorn runs it (`uvicorn app:app`). **Pydantic** is the *data-validation* library: you define a class with typed fields, and Pydantic parses and validates incoming JSON, rejecting bad input with a clear 422 error automatically.

```
from pydantic import BaseModel

class Message(BaseModel):
    text: str            # request auto-validated: missing/wrong type -> 422

@app.post("/predict")
def predict(msg: Message):
    label = get_model().predict([msg.text])[0]
    return {"label": label}
```

Interview Q (core): WSGI vs ASGI?
A: WSGI is the older synchronous Python web-server interface (Flask, Django classic) - one request blocks a worker until done. ASGI is the async successor that supports `async` / `await`, WebSockets, and high-concurrency I/O. FastAPI is ASGI, and Uvicorn is an ASGI server. For an I/O-bound service that waits on model calls or network, async lets one worker juggle many requests.

Interview Q (senior): In the serving app, why load the model lazily?
A: The model is loaded on the *first* prediction rather than at import time. This keeps startup fast so the container passes its readiness/health check quickly, and it avoids doing heavy work if the process is only being health-checked. The pattern is a module-level `_model = None` with a `get_model()` that loads `model.pkl` via joblib once and caches it. The trade-off is a slightly slower first request (a "cold start").

Interview Q (senior): Common HTTP status codes an API should return?
A: 200 OK for success; 201 Created after a POST that creates a resource; 400 Bad Request for malformed input; 422 Unprocessable Entity (FastAPI's validation failure); 401 Unauthorized (not authenticated) vs 403 Forbidden (authenticated but not allowed); 404 Not Found; 429 Too Many Requests (rate limited); 500 Internal Server Error for an unhandled exception; 503 Service Unavailable when a dependency is down.

Gotcha: `GET` should be safe and idempotent - never mutate state in a GET handler. Health checks are GET for exactly this reason. Prediction is POST because you're sending a request body, even though it doesn't persist anything.

13.4 scikit-learn / ML Serving

Interview Q (ELI5): What does the model in this project actually do?
A: It reads a text message and predicts "spam" or "ham" (ham = legitimate). It's trained on labeled examples so it learns which words signal spam. Under the hood it's a TF-IDF vectorizer feeding a Logistic Regression classifier.

Interview Q (core): Explain TF-IDF and Logistic Regression in one breath each.
A: TF-IDF (Term Frequency-Inverse Document Frequency) turns text into numbers: a word scores high if it appears often in *this* message but rarely across *all* messages, so common words like "the" get down-weighted and distinctive words like "winner" stand out. **Logistic Regression** is a linear classifier that takes those numeric features, computes a weighted sum, and squashes it through a sigmoid into a probability between 0 and 1, which becomes spam-vs-ham above/below a threshold.

Interview Q (core): What is a scikit-learn Pipeline and why use one?
A: A Pipeline chains preprocessing and the estimator into one object - here TF-IDF vectorizer then Logistic Regression. It matters because the *exact same* transformation must apply at training and at inference. Bundling them means you serialize one artifact and can't accidentally vectorize differently in production. You call `.fit()` once and `.predict()` later on raw text.

Interview Q (core): How is the model persisted and loaded here?
A: `train.py` fits the pipeline and dumps it with `joblib.dump(model, "model.pkl")` at Docker build time; `app.py` loads it with `joblib.load`. joblib is preferred over plain pickle for scikit-learn because it serializes large NumPy arrays efficiently.

Interview Q (senior): What are the risks of pickle/joblib model files in production?
A: Two big ones. First, **security**: unpickling executes arbitrary code, so you must never load a model file from an untrusted source. Second, **version fragility**: a pickle is tied to the library versions it was created with - load it with a different scikit-learn/NumPy version and it may break or silently misbehave. Mitigations: pin exact versions in the image, and for cross-platform sharing consider ONNX or a documented schema.

Interview Q (senior): What is training-serving skew and how do you prevent it?
A: It's when the data or preprocessing at inference differs from training, so live accuracy drops. Prevention: reuse the *same* pipeline object (not a reimplemented transform) at both stages - exactly why the TF-IDF and classifier live in one serialized Pipeline here - pin dependency versions, and validate inputs. Related is **data drift**, where production input distribution shifts over time; you monitor prediction distributions and periodically retrain.

Gotcha: Accuracy is a misleading metric for spam because classes are imbalanced - if 95% of messages are ham, a model that always says "ham" scores 95%. Report precision, recall, and F1, and think about which error is worse (a spam getting through vs a real message flagged).

13.5 Apache Kafka

> **Interview Q (ELI5):** What is Kafka?
> **A:** Kafka is a system for streaming events between programs. Producers write messages onto named channels called **topics**; consumers read from them. It acts like a durable, replayable log in the middle, so senders and receivers are decoupled - neither has to be online at the same moment. Here a producer streams messages to a `messages` topic and a consumer reads them, classifies each, and writes results to a `predictions` topic.

Key vocabulary you must own:

Term	Meaning
Topic	A named stream of messages (like `messages`, `predictions`)
Partition	A topic is split into partitions for parallelism and ordering
Offset	A message's sequential position within a partition
Producer	Writes messages to a topic
Consumer	Reads messages from a topic
Consumer group	A set of consumers sharing the work of a topic
Broker	A Kafka server node storing partitions

> **Interview Q (core):** What guarantees does Kafka give about ordering?
> **A:** Ordering is guaranteed **within a single partition**, not across a topic. Messages with the same key go to the same partition, so if you need per-user or per-key order you set the key. Across partitions there's no global order - that's the price of horizontal scalability.

> **Interview Q (core):** What is a consumer group and how does it scale consumption?
> **A:** A consumer group is consumers that cooperatively read one topic; Kafka assigns each partition to exactly one consumer in the group, so work is spread out. Add consumers up to the number of partitions to increase throughput. Two *different* groups each get the *full* stream independently - that's how you fan out the same events to multiple applications.

> **Interview Q (core):** What is KRaft mode? This project uses Kafka 3.9 in KRaft, no ZooKeeper.
> **A:** Historically Kafka needed a separate **ZooKeeper** cluster to manage metadata and leader election - an extra system to run. **KRaft** (Kafka Raft) moves that metadata management into Kafka itself using the Raft consensus protocol, so you run one system instead of two. It's simpler to operate, starts faster, and scales metadata better - which is exactly why a local demo uses it: a single Kafka container, no ZooKeeper.

> **Interview Q (senior):** Delivery semantics - at-most-once, at-least-once, exactly-once?
> **A: At-most-once**: commit the offset before processing, so a crash loses the message but never duplicates. **At-least-once**: process first, then commit - a crash may reprocess, so you can get duplicates (the common default). **Exactly-once**: Kafka supports it via idempotent producers and transactions, but end-to-end exactly-once also needs your consumer's side effects to be idempotent or transactional. In practice most pipelines choose at-least-once plus idempotent handlers.

> **Interview Q (senior):** How does Kafka achieve durability and high throughput?
> **A:** Messages are appended to an on-disk commit log per partition and replicated across brokers (replication factor); a message is "committed" once the in-sync replicas have it. Throughput comes from sequential disk writes, zero-copy transfer to consumers, batching, and compression. Consumers pull at their own pace and track offsets, so a slow consumer never blocks producers - back-pressure is handled by the log's retention, not by dropping.

> **Gotcha:** Kafka is not a database or a request/response queue - it's an append-only log with a retention window. Consumers can replay from an old offset, but old messages expire per the retention policy. And Kafka doesn't push; consumers **poll**.

13.6 Kubernetes

> **Interview Q (ELI5):** What is Kubernetes?
> **A:** Kubernetes (K8s) is a system that runs and manages containers across a cluster of machines for you. You declare the desired state - "run 2 copies of this API, keep them healthy, expose them on this port" - and Kubernetes continuously makes reality match, restarting crashed containers and rescheduling them if a machine dies.

Core objects, which map directly onto this repo's two YAML files:

Object	Role
Pod	Smallest unit: one (or few) containers that share network/storage
Deployment	Declares desired replica count and manages/updates Pods
ReplicaSet	Created by a Deployment; keeps N Pods running
Service	Stable network endpoint / load-balancer in front of Pods
Node	A worker machine in the cluster

> **Interview Q (core):** What does a Deployment give you over running a bare Pod?
> **A:** Self-healing and scaling and rolling updates. A Deployment holds a desired replica count (here `replicas: 2`); if a Pod dies the controller recreates it. Change the image and it performs a **rolling update** - spinning up new Pods and draining old ones with zero downtime - and can roll back. A bare Pod has none of that; kill it and it's gone.

> **Interview Q (core):** Explain readiness vs liveness probes - this app defines both on `/health`.
> **A:** A **liveness** probe answers "is this container broken and needs a restart?" - if it fails, Kubernetes kills and restarts the Pod. A **readiness** probe answers "is this container ready to receive traffic?" - if it fails, the Pod is removed from the Service's load-balancer but *not* restarted. Readiness handles temporary unavailability (still warming up), liveness handles deadlocks. Both hit `GET /health` here, with a generous `initialDelaySeconds` so the app has time to start before probes count failures.

```
readinessProbe:
  httpGet: { path: /health, port: 8000 }
```

```
  initialDelaySeconds: 15
livenessProbe:
  httpGet: { path: /health, port: 8000 }
  initialDelaySeconds: 20
```

> **Interview Q (core):** What are resource requests and limits?
> **A:** A **request** is what the scheduler reserves for a container and uses to place it on a node; a **limit** is the hard ceiling. Exceed the CPU limit and you're throttled; exceed the memory limit and the container is OOM-killed. Setting both (as this Deployment does) prevents one Pod from starving neighbors and helps the scheduler pack nodes.

> **Interview Q (core):** What Service types are there, and why NodePort here?
> **A: ClusterIP** (default) exposes the Service only inside the cluster; **NodePort** opens a port on every node (30000-32767 range) so you can reach it from outside a local cluster; **LoadBalancer** provisions a cloud load balancer with a public IP. This project uses **NodePort 30080 -> 8000** because it's a local cluster with no cloud LB - it's the simplest way to hit the API from your laptop (often combined with `kubectl port-forward`).

> **Interview Q (senior):** What is `imagePullPolicy: IfNotPresent` doing here and why does it matter locally?
> **A:** It tells the kubelet to use the locally cached image if it exists and only pull from a registry otherwise. That's essential for a locally *built* image that was never pushed to a registry - with the default `Always` (which applies to `:latest` tags), Kubernetes would try to pull and fail with `ImagePullBackOff`. `IfNotPresent` lets the cluster run the image you built on your machine.

> **Interview Q (senior):** How does a Service actually route to the right Pods, and how does the reconciliation loop work?
> **A:** A Service uses a **label selector**; it dynamically tracks all Pods whose labels match and load-balances across their IPs (via kube-proxy / endpoints), so Pods can come and go and the Service endpoint stays stable. More broadly, Kubernetes is a set of **controllers** running reconciliation loops: each continuously compares desired state (your YAML in etcd) to observed state and takes action to close the gap. That control-loop model is why K8s is self-healing.

> **Gotcha:** `ImagePullBackOff` and `CrashLoopBackOff` are the two errors you'll be asked to debug. The first is an image the kubelet can't pull (wrong name/tag, missing registry auth, or `Always` policy on a local image). The second is a container that keeps starting and exiting - check `kubectl logs` and probe/config/missing-dependency issues.

13.7 Terraform / Infrastructure as Code

> **Interview Q (ELI5):** What is Terraform?
> **A:** Terraform lets you write your cloud infrastructure as code. Instead of clicking around a console to create a storage bucket or database, you describe what you want in files, and Terraform creates, changes, or deletes real resources to match. Here it declares an S3 bucket and a DynamoDB table.

Interview Q (core): What is Infrastructure as Code and why does it matter?
A: IaC means defining infrastructure in version-controlled text rather than manual clicks. Benefits: it's reproducible (spin up an identical environment anywhere), reviewable (infra changes go through pull requests), auditable (git history), and self-documenting. It eliminates configuration drift and "what did someone change in the console last week?"

Interview Q (core): Declarative vs imperative - which is Terraform?
A: Terraform is **declarative**: you describe the desired end state and Terraform figures out the actions to reach it. Imperative (a bash script) specifies the exact steps. Declarative is idempotent - running it again when nothing changed does nothing - which is why `terraform apply` twice is safe.

Interview Q (core): Walk me through the core workflow.
A: `terraform init` downloads the providers (here `hashicorp/aws ~> 5.0`) and sets up the backend. `terraform plan` computes and shows the diff between desired config and current state - a dry run. `terraform apply` executes that plan and updates state. `terraform destroy` tears everything down. This project loops init -> plan -> apply -> destroy against LocalStack.

Interview Q (core): What is Terraform state and why is it critical?
A: The **state file** (`terraform.tfstate`) is Terraform's record mapping your config to the real resources it created (IDs, attributes). Terraform needs it to know what already exists so `plan` can compute an accurate diff. It's critical - and sensitive, since it can contain secrets. In a team you store it in a **remote backend** (S3 + DynamoDB lock, or Terraform Cloud) with **state locking** so two people can't apply simultaneously and corrupt it.

Interview Q (senior): How does Terraform target LocalStack instead of real AWS?
A: Through the provider config. The AWS provider gets an `endpoints {}` block pointing every service (s3, dynamodb) at `http://localhost:4566`, dummy access/secret keys, a fixed region, and `s3_use_path_style = true`. Path-style addressing is required because virtual-host-style S3 URLs (`bucket.s3...`) don't resolve against a local endpoint. Skipping credential/metadata validation lets the fake creds through. Same `.tf` code, different endpoint - that's the whole trick.

```
provider "aws" {
  access_key                  = "test"
  secret_key                  = "test"
  region                      = "us-east-1"
  s3_use_path_style           = true
  skip_credentials_validation = true
  endpoints {
    s3       = "http://localhost:4566"
    dynamodb = "http://localhost:4566"
  }
}
```

Interview Q (senior): `count` vs `for_each`, and what's a module?
A: Both create multiple instances of a resource. `count` gives an integer and index-addressed instances (`[0]`, `[1]`) - good for identical copies, but removing a middle one re-indexes and churns the rest. `for_each` iterates a map/set and addresses instances by key - stable when the collection changes, so it's preferred for distinct resources. A **module** is a reusable, parameterized bundle of resources you call with inputs and outputs - the unit of DRY infrastructure.

Gotcha: Never hand-edit the state file or change resources in the console out-of-band - that causes **drift**, and the next `plan` will try to "fix" reality back to the config (or error). Use `terraform import` to bring existing resources under management, and `terraform state` commands for surgical changes.

13.8 AWS (S3, DynamoDB, IAM, Bedrock) & LocalStack

Interview Q (ELI5): What is AWS, and what are S3 and DynamoDB?
A: AWS is Amazon's cloud - you rent computing, storage, and services on demand. **S3** (Simple Storage Service) is object storage: you put files ("objects") in "buckets" and get them back by key - great for model artifacts, which is what this project stores. **DynamoDB** is a fully managed NoSQL key-value/document database - fast lookups by key, scales automatically - here it stores predictions.

Interview Q (core): Why store the model in S3 and predictions in DynamoDB - why not the reverse?
A: They fit different shapes. A model file is a large binary blob you read/write whole - that's object storage, S3's job. Predictions are many small structured records you write and look up by an id key at high rate - that's DynamoDB's job. The Terraform here reflects that: an S3 bucket for artifacts, a DynamoDB table keyed on `id`.

Interview Q (core): What is DynamoDB's PAY*PER*REQUEST / on-demand mode, and what's a hash key?
A: DynamoDB billing is either **provisioned** (you pre-buy read/write capacity) or **on-demand / PAY*PER*REQUEST** (you pay per request, no capacity planning, scales instantly). This table uses PAY*PER*REQUEST - ideal for spiky or unknown workloads like a demo. The **hash key** (partition key) is the primary key that determines which partition a row lives in; here it's `id`. Choosing a high-cardinality partition key avoids "hot partitions."

Interview Q (core): What is IAM?
A: IAM (Identity and Access Management) controls *who* can do *what* in AWS. You define **policies** (JSON documents granting/denying specific actions on specific resources) and attach them to users, groups, or **roles**. A **role** is an identity that services or apps *assume* temporarily - the preferred way to grant an app permissions without long-lived keys. The governing principle is **least privilege**: grant only the actions needed.

Interview Q (core): What is LocalStack and why use it?
A: LocalStack is a local emulator of AWS services running in a Docker container (`localstack/localstack:3` , here with `SERVICES=s3,dynamodb` on port 4566). It lets you develop and test IaC and AWS code offline, fast, and free - no cloud account, no bill, no risk. You point your SDK/Terraform at `http://localhost:4566` with dummy creds and it responds like AWS.

Interview Q (core): What is AWS Bedrock and how does this project use it?
A: Bedrock is AWS's managed service for foundation models (LLMs) behind one API - no infrastructure to run. The `bedrock-explainer` module uses **boto3** (the AWS Python SDK) to call `bedrock-runtime` 's `invoke_model` with **Anthropic Claude 3 Haiku**, sending the Anthropic messages format, so the LLM explains in plain language *why* a message was classified spam or ham. Credentials come from the environment, never hard-coded.

Interview Q (senior): How should an app authenticate to AWS - and what's wrong with hard-coded keys?
A: Never hard-code access keys in source; they leak via git and can't rotate easily. Use the **default credential provider chain**: environment variables, shared config files, or - best in production - an **IAM role** attached to the compute (EC2 instance profile, ECS task role, or EKS IRSA), which delivers short-lived, auto-rotated credentials. This project follows the rule: Bedrock creds come from the environment, and the LocalStack modules use dummy keys precisely because nothing real is touched.

Key insight: The boto3 credential chain checks, in order: explicit params -> environment variables (`AWS_ACCESS_KEY_ID` etc.) -> shared credentials file (`~/.aws/credentials`) -> IAM role via instance/container metadata. Knowing this order lets you debug "why is it using the wrong account?" instantly.

Interview Q (senior): S3 consistency and durability guarantees?
A: S3 now provides **strong read-after-write consistency** for all operations - a successful write is immediately readable (it used to be eventually consistent). Durability is famously "eleven nines" (99.999999999%) because objects are redundantly stored across devices and availability zones. That durability is why it's the right home for irreplaceable artifacts like trained models.

Gotcha: LocalStack emulates the *API surface*, not every behavior or limit of real AWS, and IAM enforcement is loose by default. So passing tests against LocalStack doesn't guarantee correct IAM in production - validate permissions against real AWS or with strict IAM enforcement enabled.

13.9 General MLOps

Interview Q (ELI5): What is MLOps?
A: MLOps is DevOps applied to machine learning: the practices and tooling to take a model out of a notebook and run it reliably in production - training it reproducibly, serving it, deploying it, monitoring it, and updating it. It's about the *system* around the model, not just the model.

Interview Q (core): How is MLOps different from traditional DevOps?
A: Traditional software behavior is defined by code alone; ML behavior depends on **code + data + model**, all of which change. So MLOps adds data versioning, model versioning, experiment tracking, and monitoring for *statistical* failures (data drift, accuracy decay) on top of ordinary CI/CD. A deploy can pass all tests and still degrade because the world changed - unique to ML.

Interview Q (core): What does a model lifecycle look like end to end?
A: Data collection and validation -> feature engineering -> training and experiment tracking -> evaluation against metrics -> packaging (containerize the model + server) -> deployment (serve behind an API, orchestrate on K8s) -> monitoring (latency, throughput, drift, accuracy) -> retraining trigger, looping back. This portfolio walks the middle-to-late stages concretely: train (scikit-learn) -> package (Docker) -> serve (FastAPI) -> stream (Kafka) -> provision (Terraform) -> orchestrate (Kubernetes) -> explain (Bedrock).

Interview Q (senior): How do you monitor a model in production, and what's model drift?
A: Monitor two layers. **Operational**: latency, error rate, throughput, resource use - standard SLOs. **Model-quality**: input feature distributions, prediction distribution, and where you have delayed labels, live accuracy/precision/recall. **Drift** is the gap growing between training-time and production data (data drift) or the input-output relationship changing (concept drift), which quietly erodes accuracy. You alert on distribution shifts and trigger retraining when metrics cross a threshold.

Interview Q (senior): What deployment strategies reduce risk when shipping a new model?
A: Shadow / canary: send a small slice of traffic (or a mirrored copy) to the new model and compare before full rollout. **Blue-green**: run old and new side by side and switch traffic atomically, with instant rollback. **A/B testing**: split traffic to measure business metrics. All of these need the reproducible, containerized, orchestrated foundation this project builds - you can't safely canary a model you can't version and roll back.

Interview Q (senior): How does reproducibility get enforced across this stack? (Great synthesis answer.)
A: Layer by layer: Docker pins the runtime and dependencies into an image; the model is trained at build time and serialized with joblib so the artifact is fixed; Terraform pins the provider and declares infra as code; Kubernetes declares desired state; and everything is in git. Given the same repo and Docker, anyone reproduces the identical system. That end-to-end reproducibility - code, environment, model, and infra all versioned - is the core promise of MLOps, and it's why "container-first" is the project's organizing principle.

Key insight - the one-liner to close any interview on this project: "It's a single spam classifier operated five ways - served as a FastAPI container, streamed through Kafka, provisioned with Terraform against LocalStack, orchestrated on Kubernetes, and explained by a Bedrock LLM - so it demonstrates the whole MLOps lifecycle on one model, entirely in Docker for reproducibility." Memorize that sentence; it frames every follow-up.

14. Interview Prep II - Project Deep-Dive, System Design, War Stories, Glossary

Chapter 13 drilled the individual tools; this chapter puts the *whole project* in front of an interviewer. It gives you rehearsed scripts for "walk me through your project," a bank of project-specific Q&A, system-design answers that scale the repo up to real cloud, behavioral "war stories" drawn from actual bugs in this codebase, a one-page cheat sheet, and a complete glossary. Read it out loud until the answers feel like *yours*.

14.1 The 30-second, 2-minute, and 5-minute pitch

Interviewers almost always open with "tell me about a project." Have three lengths ready and pick based on their body language and follow-up appetite.

30-second version (elevator):

> "I built an MLOps portfolio around one machine-learning model - a spam classifier - and I operate it five different ways, each in its own module. I serve it as a REST API in Docker, stream real-time predictions through Apache Kafka, provision cloud infrastructure for it with Terraform, deploy it on Kubernetes with two replicas and health probes, and add a GenAI layer where AWS Bedrock explains *why* a message was flagged. The point wasn't the model - it's a simple TF-IDF plus logistic regression - the point was to show I can take *any* model to production across the whole tool stack."

2-minute version (adds the "how" and one insight):

> "The core is a scikit-learn spam classifier - TF-IDF vectorizer feeding a logistic regression. That's deliberately boring so the focus stays on operations. Module one wraps it in a FastAPI service: `GET /health` for probes and `POST /predict` for inference, packaged in Docker where `train.py` trains the model at *build time* into `model.pkl`, and the model loads *lazily* on the first prediction so startup and health checks stay fast.
>
> Module two is a Kafka streaming pipeline in KRaft mode - no ZooKeeper. A producer streams messages to a `messages` topic; a consumer reads each one, calls the serving API over HTTP, and publishes the result to a `predictions` topic. Crucially the consumer does *not* load the model itself - inference stays behind the API so the streaming layer and the model service scale independently.
>
> Module three is Terraform against LocalStack, a local AWS emulator, provisioning an S3 bucket for model artifacts and a DynamoDB table for predictions - so I can run a real `terraform apply` with no cloud bill. Module four deploys the container to Kubernetes: a Deployment with two replicas, readiness and liveness probes, and resource requests and limits, fronted by a NodePort service. Module five uses boto3 to call AWS Bedrock - Claude 3 Haiku - to explain predictions in plain English.
>
> Everything runs in Docker so it's reproducible on any machine."

5-minute version: deliver the 2-minute pitch, then add (a) one *design decision* - "I decoupled inference from the consumer so I could scale twenty consumers against three API replicas without twenty copies of the model in memory"; (b) one *war story* - the Kafka `advertised.listeners` bug (14.4); and (c) *where I'd take it next* - EKS, MSK, ECR, real Bedrock, plus monitoring and CI/CD

(14.3). Ending on "what's next" signals you think about production, not just demos.

> **Key insight:** The strongest framing is "one model, five operational patterns." It turns a small project into a *breadth* demonstration and lets the interviewer steer toward whichever tool they care about.

14.2 Project-specific Q&A

> **Interview Q:** Why five separate modules instead of one integrated app?
> **A:** Each module isolates one production concern so it can be understood, run, and scaled on its own. Serving is about packaging and HTTP; Kafka is about async, high-throughput ingestion; Terraform is about reproducible infrastructure; Kubernetes is about orchestration and self-healing; Bedrock is about a GenAI layer. In the real world these are owned by different teams and scale on different axes. Keeping them separate mirrors that and keeps each module a clean, teachable unit rather than a tangled monolith.

> **Interview Q:** Why decouple inference from the Kafka consumer instead of loading the model inside the consumer?
> **A:** Two reasons - scaling and separation of concerns. If the model lived in the consumer, every consumer instance would carry its own copy of the model in memory, and scaling ingestion would force me to scale the model too. By keeping inference behind the FastAPI service, I can run, say, twenty lightweight consumers feeding three model replicas - or the reverse - and tune each independently. It also means one model deployment serves *every* client - the streaming pipeline, a web app, a batch job - with one place to version and roll back. The consumer's job is just transport: read from `messages`, POST to `/predict`, write to `predictions`.

> **Interview Q:** Why load the model lazily on the first prediction rather than at startup?
> **A:** So the process starts fast and the liveness probe never blocks on model loading. `scikit-learn`'s first import is genuinely slow, and joblib-loading the pickle adds more. If I loaded eagerly at import, the container would look "down" to Docker and Kubernetes for that whole window and might get killed and restarted in a loop. With lazy loading, `/health` responds instantly, the readiness probe can pass, and only the *first* real prediction pays the load cost - which I hide behind client-side retries. The tradeoff is that first prediction is slow; a production alternative is loading eagerly but tuning probe `initialDelaySeconds` and healthcheck `start-period` to cover it.

> **Interview Q:** Why train the model at Docker build time instead of at runtime or fetching it from storage?
> **A:** Reproducibility and immutability. Training at build time bakes `model.pkl` into the image, so the image *is* the model - the exact same artifact runs on my laptop, in Kubernetes, anywhere, with no runtime dependency on a training job or an artifact store being reachable. It also keeps the demo self-contained. The honest downside is that model and code are now coupled in one image and you rebuild to retrain; a production system would train separately, version the artifact in S3 or a model registry, and have the service pull a pinned version at startup - which is exactly what the S3 bucket in the Terraform module foreshadows.

Interview Q: Why NodePort and `kubectl port-forward` for local Kubernetes instead of a LoadBalancer or Ingress?
A: Because there's no cloud load balancer locally. A `LoadBalancer` service type asks the cloud provider to provision an external LB - on Docker Desktop or minikube that either doesn't work or hangs in `<pending>` . NodePort exposes the service on a static port on the node (30080 mapping to the container's 8000), which works on any cluster, and `port-forward` tunnels straight from my laptop to a Pod for quick testing. In real AWS I'd switch to an Ingress controller or a `LoadBalancer` service backed by an ALB/NLB.

Interview Q: Why LocalStack instead of a real AWS account for the Terraform module?
A: So anyone can run a genuine `terraform init / plan / apply / destroy` cycle with zero cloud credentials and zero cost, and so the demo is deterministic and safe to tear down. LocalStack emulates S3 and DynamoDB on `localhost:4566` . The Terraform code is real HashiCorp AWS-provider code - the only difference from production is the provider configuration, so the skills transfer directly.

Interview Q: What does the `endpoints {}` block in the provider actually do, and why the extra flags?
A: The `endpoints` block redirects each AWS service's API calls to LocalStack's address (`http://localhost:4566`) instead of the real AWS endpoints. Alongside it I set dummy `access_key` / `secret_key` of `test` , and `skip_credentials_validation` , `skip_metadata_api_check` , and `skip_requesting_account_id` so the provider doesn't try to phone home to real AWS to verify the fake creds. I also set `s3_use_path_style = true` because LocalStack serves buckets as `localhost:4566/bucket-name` (path-style) rather than the virtual-hosted `bucket-name.s3.amazonaws.com` style, which doesn't resolve locally. Strip those flags and point `endpoints` at real AWS and the same code provisions real infrastructure.

Interview Q: Why is everything configured through environment variables?
A: The twelve-factor principle: config that varies between environments lives in the environment, not the code. The consumer reads `KAFKA_BROKER` , `MESSAGES_TOPIC` , `PREDICTIONS_TOPIC` , and `MODEL_API` from env vars with sensible defaults like `kafka:9092` and `http://spam-api:8000/predict` . That means the *same image* runs locally, in Docker Compose, and in Kubernetes just by changing env values - no rebuild. It also keeps secrets out of the image, which is exactly how the Bedrock module takes AWS credentials from the environment and never hard-codes them.

Interview Q: Why DynamoDB `PAY_PER_REQUEST` and a hash key of `id` ?
A: `PAY_PER_REQUEST` (on-demand) billing means you don't have to provision read/write capacity up front - it auto-scales and you pay per request, which is ideal for spiky or unknown prediction traffic and for a demo where I don't want to guess capacity. The hash key `id` is the partition key; each prediction gets a unique id, which spreads writes evenly across partitions and lets me look a prediction up directly by id.

14.3 System-design questions built on the repo

Interviewers love to say "now productionize it." Here are structured answers.

"Design a production real-time ML inference pipeline."

Walk the data path and name the AWS-managed equivalent of each local piece:

Local (this repo)	Production equivalent	Why
kafka-python producer/consumer	**Amazon MSK** (Managed Streaming for Kafka)	managed brokers, no ops
Kafka topic `messages`	MSK topic, partitioned	partitions give parallelism
FastAPI in Docker	container on **EKS** behind an ALB	orchestrated, autoscaled
local image	**ECR** (Elastic Container Registry)	versioned, pull-on-deploy
DynamoDB (LocalStack)	real **DynamoDB**	serverless prediction store
S3 (LocalStack)	real **S3** for model artifacts	model registry / versioning

Design narrative: producers write to an MSK topic partitioned by, say, user id so ordering is preserved per key. A consumer group of N consumers reads partitions in parallel - one partition per consumer max. Each consumer calls the model service (an EKS Deployment behind an internal ALB) which is horizontally autoscaled on CPU/latency. Predictions go to a `predictions` topic *and* are persisted to DynamoDB. Add a dead-letter topic for messages that fail after retries, and backpressure via consumer lag monitoring.

> **Key insight:** The number that governs streaming throughput is *partitions*. A topic with P partitions supports at most P consumers doing useful work in a group. Say "I'd size partitions to my target consumer parallelism plus headroom" and you sound like you've run Kafka.

"How would you take this to real AWS?"

1. **Images:** push `mlops-serving` to **ECR** with an immutable tag (git SHA), not `latest`.
2. **Compute:** provision an **EKS** cluster with Terraform; convert the Deployment/Service into a Helm chart; use a `LoadBalancer` /Ingress instead of NodePort.
3. **Streaming:** replace local Kafka with **MSK**; the consumer code barely changes - just point `KAFKA_BROKER` at the MSK bootstrap brokers (thank you, env-var config).
4. **State:** real **S3** + **DynamoDB**; remove the `endpoints` block and dummy creds; use **IAM roles for service accounts (IRSA)** so Pods get scoped credentials with no static keys.
5. **GenAI:** point the Bedrock client at the real `bedrock-runtime` endpoint with an IAM role granting `bedrock:InvokeModel`.
6. **Terraform state:** move state to an S3 backend with DynamoDB state locking so a team can collaborate safely.

"How do you scale the consumers and the API independently?"

Consumers scale up to the number of topic partitions (add partitions to go higher). The API scales on its own metrics via a **HorizontalPodAutoscaler** targeting CPU or p95 latency. Because inference is decoupled (14.2), these two knobs are independent - that decoupling is the whole point.

"Monitoring and observability?"

Three pillars:

- **Metrics:** expose Prometheus metrics from FastAPI (request rate, latency, error rate - the RED method), plus Kafka consumer *lag* (how far behind real-time the consumers are), and business metrics like spam/ham ratio. Visualize in Grafana.
- **Logs:** structured JSON logs to CloudWatch or an ELK/Loki stack; correlate with a request/trace id.

- **Model-specific: data drift** (input distribution shifting away from training data) and **prediction drift** (label mix changing), plus, when ground-truth labels arrive later, accuracy/precision/recall over time. Alert when drift crosses a threshold - that's your signal to retrain.

"CI/CD?" On every push: lint, run unit tests on `train.py` / `app.py`, build the image, scan it for vulnerabilities, push to ECR with the git SHA tag, then deploy to a staging EKS namespace and run a smoke test against `/health` and `/predict`; promote to prod on approval. Terraform runs in its own pipeline with `plan` on PR and `apply` on merge.

"Security and secrets?" No static credentials - IRSA/IAM roles for AWS access; secrets in AWS Secrets Manager or Kubernetes Secrets, never in images or git; the Bedrock module already models this by reading creds from the environment. Least-privilege IAM: the service can write to *its* DynamoDB table and read *its* S3 prefix, nothing more. Network policies to restrict Pod-to-Pod traffic.

"Model versioning and rollback?" Version the artifact in S3 (or a registry like MLflow) with an immutable version id; the service loads a *pinned* version via env var, not "latest." Deploy new models behind a flag; do canary or shadow traffic; if metrics regress, roll back by pointing the env var at the previous version and redeploying - no rebuild. Keep the image and the model version pinned together in the deploy manifest so a rollback is one revert.

14.4 Behavioral "war stories" (real gotchas from this repo)

Behavioral questions ("hardest bug," "what did you learn") are won with *specific* stories. Each of these is true of this codebase. Use STAR: Situation, Task, Action, Result.

War story 1 - Kafka advertised listeners.

> "My producer and consumer kept failing with `NoBrokersAvailable` even though the broker was clearly up. The cause was Kafka's `advertised.listeners`: when a client connects, the broker replies with the address clients should *actually* use for data - and if that's set to `localhost:9092`, a container that connects via the service name `kafka` gets told to reconnect to `localhost`, which inside its own container is itself, not the broker. The fix was advertising `kafka:9092` - the Docker network service name - so containers reconnect to the right host. I learned that Kafka's bootstrap connection and its *data* connection are two separate steps, and the advertised listener governs the second."

War story 2 - the slow scikit-learn first import.

> "In Kubernetes my Pods kept getting killed and restarted right after starting. The liveness probe was firing before the app was ready because `scikit-learn`'s first import plus loading the pickle is slow. I fixed it on three fronts: I load the model *lazily* so `/health` responds instantly and startup isn't blocked; I set generous probe delays - `initialDelaySeconds: 60` for readiness and `90` for liveness - so Kubernetes waits before judging the Pod; and I gave the Kafka consumer client-side retries - up to 40 attempts with a 90-second request timeout - so it tolerates the model API's slow first response instead of crashing. The lesson: startup latency is a first-class design concern, and probes plus retries have to be tuned to the *real* cold-start time."

War story 3 - making LocalStack behave like AWS.

> "Terraform kept failing to reach S3 until I got the provider config exactly right. LocalStack needs three things: an `endpoints` block redirecting S3 and DynamoDB to `localhost:4566`, dummy `test` / `test` credentials with the credential-validation checks skipped so the provider doesn't try to authenticate against real AWS, and `s3_use_path_style = true` because LocalStack serves buckets path-style (`localhost:4566/bucket`) rather than the virtual-hosted subdomain style. Once those lined up, a real `terraform apply` worked with no cloud account. It taught me how the AWS provider actually resolves endpoints and addresses buckets under the hood."

War story 4 - imagePullPolicy on a local image.

> "My Kubernetes Deployment kept failing with `ImagePullBackOff` - it was trying to pull `mlops-serving:latest` from Docker Hub, which doesn't exist there; I'd built it locally. Setting `imagePullPolicy: IfNotPresent` told Kubernetes to use the local image if present instead of always pulling. It reinforced that `:latest` implicitly means `imagePullPolicy: Always`, and that local dev clusters need explicit handling for locally-built images."

> **Gotcha:** If asked "what would you do differently," don't say "nothing." Say: "I'd version images with git SHAs instead of `:latest`, train the model separately and pull a pinned artifact from S3 instead of baking it into the image, and add Prometheus metrics from day one." That shows growth.

14.5 One-page cheat sheet

Docker / serving

```
docker build -t mlops-serving:latest ./serving      # train.py runs at build time
docker run -p 8000:8000 mlops-serving:latest
curl localhost:8000/health
curl -X POST localhost:8000/predict -H 'Content-Type: application/json' \
     -d '{"text":"win a free prize now"}'
```

Facts: FastAPI + Uvicorn (ASGI); lazy model load; Dockerfile installs deps *before* copying source (layer caching); `EXPOSE 8000`; `HEALTHCHECK` with `start-period`.

Kafka streaming

```
docker compose up --build          # kafka, spam-api, producer, consumer
```

Facts: Kafka 3.9 **KRaft** (no ZooKeeper); topics `messages` -> `predictions`; consumer group `spam-inference`; `advertised.listeners = kafka:9092`; consumer POSTs to the API, doesn't load the model.

Terraform / LocalStack

```
terraform init && terraform plan && terraform apply -auto-approve
terraform destroy -auto-approve
```

Facts: `hashicorp/aws ~> 5.0`; `endpoints{}` -> `localhost:4566`; dummy `test` / `test` creds; `s3_use_path_style`; S3 bucket + DynamoDB (`PAY_PER_REQUEST`, hash key `id`).

Kubernetes

```
kubectl apply -f kubernetes/
kubectl get pods
kubectl port-forward svc/spam-api 8000:8000
```

Facts: Deployment, **2 replicas**; readiness (`initialDelay 60`) + liveness (`90`) probes on `/health` ; requests `100m` / `256Mi` , limits `500m` / `512Mi` ; `imagePullPolicy: IfNotPresent` ; NodePort `30080 -> 8000` .

Bedrock Facts: boto3 `bedrock-runtime` ; **Claude 3 Haiku**; `invoke_model` with the Anthropic *messages* format; creds from env, never hard-coded.

14.6 Comprehensive glossary

Containers & Docker

- **Container:** a lightweight, isolated process running from an image; like a shipping container - same box runs anywhere.
- **Image:** a read-only, layered template a container is created from; includes app, deps, and (here) the trained model.
- **Layer:** one filesystem diff in an image; installing deps *before* copying source means the deps layer is cached and reused across code changes.
- **Dockerfile:** the recipe that builds an image.
- **Registry / ECR:** a store of images you push to and pull from; ECR is AWS's private registry.
- **HEALTHCHECK / start-period:** Docker's built-in probe; `start-period` is a grace window before failures count.
- **imagePullPolicy:** when Kubernetes fetches an image; `IfNotPresent` uses a local copy, `Always` re-pulls (implied by `:latest`).

Serving / web

- **REST API:** an HTTP interface where URLs (endpoints) represent operations.
- **FastAPI:** a Python web framework for building APIs with automatic validation and docs.
- **Uvicorn / ASGI:** ASGI is the async server interface for Python web apps; Uvicorn is the server that runs FastAPI.
- **Pydantic:** validates/parses request bodies against a typed schema (`Message.text`).
- **joblib / pickle:** serialization; `model.pkl` is the saved scikit-learn model, loaded with `joblib.load` .
- **Lazy loading:** deferring an expensive step (model load) until first use.
- **Endpoint:** a callable URL path, e.g. `/health` , `/predict` .

ML

- **TF-IDF:** Term Frequency-Inverse Document Frequency - turns text into numbers weighting words by how distinctive they are.
- **Logistic Regression:** a linear model outputting a probability per class.
- **Spam/ham classifier:** ham = legitimate, spam = junk.
- **Confidence:** here, the max predicted class probability.
- **Data drift / prediction drift:** input or output distributions shifting away from training; a retrain signal.

Kafka

- **Broker:** a Kafka server that stores and serves messages.
- **Topic:** a named, append-only log of messages (`messages` , `predictions`).
- **Partition:** a topic is split into partitions for parallelism; ordering is guaranteed *within* a partition.
- **Offset:** a message's position in a partition.
- **Consumer group:** consumers sharing a group id (`spam-inference`) split partitions among themselves.
- **Producer / Consumer:** writes to / reads from topics.

- **KRaft:** Kafka Raft mode - Kafka manages its own metadata, replacing ZooKeeper.
- **advertised.listeners:** the address the broker tells clients to connect to for data.
- **Bootstrap servers:** the initial broker address a client contacts to discover the cluster.
- **Consumer lag:** how many messages behind real-time the consumers are.

Kubernetes

- **Pod:** the smallest deployable unit; one or more containers sharing a network.
- **ReplicaSet:** keeps N identical Pods running.
- **Deployment:** manages ReplicaSets and rollouts (here, 2 replicas).
- **Service:** a stable network address load-balancing across Pods.
- **NodePort:** a Service type exposing a static port on every node (30080).
- **Readiness probe:** "can this Pod receive traffic yet?" - fails route traffic away.
- **Liveness probe:** "is this Pod still healthy?" - fails restart it.
- **Resource requests/limits:** guaranteed vs. maximum CPU/memory.
- **HPA (HorizontalPodAutoscaler):** scales Pod count on metrics.
- **Ingress:** HTTP routing into the cluster (production alternative to NodePort).
- **kubectl / port-forward:** the CLI; port-forward tunnels a local port to a Pod.

Infrastructure as Code

- **IaC:** managing infrastructure through version-controlled code.
- **Terraform:** HashiCorp's declarative IaC tool.
- **HCL:** HashiCorp Configuration Language.
- **Provider:** a plugin Terraform uses to talk to an API (here `hashicorp/aws`).
- **Resource:** a piece of infra Terraform manages (an S3 bucket, a DynamoDB table).
- **State:** Terraform's record of what it created and its real-world mapping.
- **State locking / backend:** where state lives and how concurrent writes are prevented (S3 + DynamoDB in prod).
- **plan / apply / destroy:** preview / create / tear down.
- **LocalStack:** a local emulator of AWS services on `localhost:4566`.
- **Endpoints block:** provider config redirecting API calls to a custom host.
- **Path-style S3:** `host/bucket` addressing vs. virtual-hosted `bucket.host`.

AWS

- **S3:** object storage - buckets of files (model artifacts).
- **DynamoDB:** managed NoSQL key-value store; **PAY*PER*REQUEST** = on-demand billing; **hash key** = partition key.
- **IAM:** Identity and Access Management - users, roles, permissions.
- **IRSA:** IAM Roles for Service Accounts - gives Pods scoped AWS creds with no static keys.
- **EKS / MSK / ECR:** managed Kubernetes / managed Kafka / container registry.
- **Bedrock:** AWS's managed foundation-model service; `bedrock-runtime` is the inference endpoint.
- **boto3:** the AWS SDK for Python.
- **Claude 3 Haiku:** the fast, small Anthropic model used to explain predictions via `invoke_model` in the messages format.

General

- **Twelve-factor / env-var config:** environment-specific settings live in the environment, not the code.
- **CI/CD:** Continuous Integration / Continuous Delivery - automated build, test, deploy.
- **RED method:** Rate, Errors, Duration - core request metrics.
- **Prometheus / Grafana:** metrics collection / dashboards.

- **STAR:** Situation, Task, Action, Result - the framing for behavioral answers.

> **Interview Q:** In one sentence, what is this project about?
> **A:** "It's a container-first MLOps portfolio that takes a single spam-classifier model and operates it five ways - served, streamed, provisioned, orchestrated, and explained - to prove I can put any model into production across the whole modern tool stack." (See Ch. 1 for the overview and Ch. 13 for tool-by-tool prep.)